

SYSTEM DESIGN SERIES

60 Days of System Design Questions

A question-a-day workbook — scenario, four options, and the reasoning behind the right call.

AUTHOR	Joud Awad
CONTENTS	60 questions · 240 explanations
FORMAT	Attempt the question, turn the page for the breakdown
FOLLOW	LinkedIn · YouTube · Medium · Substack

Contents

Day 01 Decoupling Mobile from Backend Services

Day 03 Rate Limiting Without Boundary Bursts

Day 05 Choosing a Database Sharding Strategy

Day 07 Event Ordering in a Message Queue

Day 09 Splitting Read and Write Data Models

Day 11 Handling Webhook Retries Idempotently

Day 13 Managing a Shared Connection Pool

Day 15 Fast “Has the User Seen This?” Checks

Day 17 Backpressure on an Overwhelmed Consumer

Day 19 Read-Your-Writes with Read Replicas

Day 21 Choosing a Real-Time Streaming Transport

Day 23 Feed Fanout for Celebrity Accounts

Day 25 Queue Backpressure Under a Traffic Spike

Day 27 Keeping an LLM’s Answers Up to Date

Day 29 Coordinating a Multi-Agent Workflow

Day 31 Cutting Cross-Region Latency

Day 33 Reconstructing State with Event Sourcing

Day 35 Geospatial “Find Nearby Drivers” at Scale

Day 37 Multi-Writer Conflicts in Collaborative Editing

Day 39 Preventing Concurrent Balance Overspend

Day 41 Moving from Batch to Real-Time Streaming

Day 43 Reliable CDN Cache Invalidation on Deploy

Day 45 Scaling Full-Text Search

Day 47 Incrementally Strangling a Monolith

Day 02 Killing the N+1 Query Problem

Day 04 Preventing Duplicate Payment Charges

Day 06 Safe Distributed Locks for Cron Jobs

Day 08 Keeping Cache and Database in Sync

Day 10 Distributed Transactions Across Services

Day 12 Indexing a High-Ingest Table

Day 14 Safely Rolling Out a Risky Change

Day 16 Taming a Hot Partition Key

Day 18 Surviving a Cache Stampede

Day 20 Containing a Failing Downstream Dependency

Day 22 Reliable Messaging Across Services

Day 24 Paginating Large Result Sets Efficiently

Day 26 Write-Path Cache Consistency

Day 28 Choosing a Vector Store for Semantic Search

Day 30 Choosing a File Storage Backend

Day 32 Managing Secrets and Credentials

Day 34 Improving LLM Classification Accuracy

Day 36 Choosing HTTP/3 vs HTTP/2 at the Edge

Day 38 Fitting a Large Document into an LLM

Day 40 Offloading Heavy Work Off the Main Thread

Day 42 Designing Memory for an AI Agent

Day 44 Syncing Offline Edits Without Data Loss

Day 46 Enforcing Structured Output from an LLM

Day 48 How an Agent Chooses Which Tools to Call

Day 49 Controlling Data-Warehouse Query Costs

Day 51 Isolating Noisy Tenants in Multi-Tenant SaaS

Day 53 Zero-Downtime Schema Migrations

Day 55 Parallel Agents with Shared State

Day 57 Consistency for Payment-Confirmation Reads

Day 59 Designing Idempotency Keys for Payments

Day 50 Serving ML Models at High Throughput

Day 52 Versioning an API Without Breaking Clients

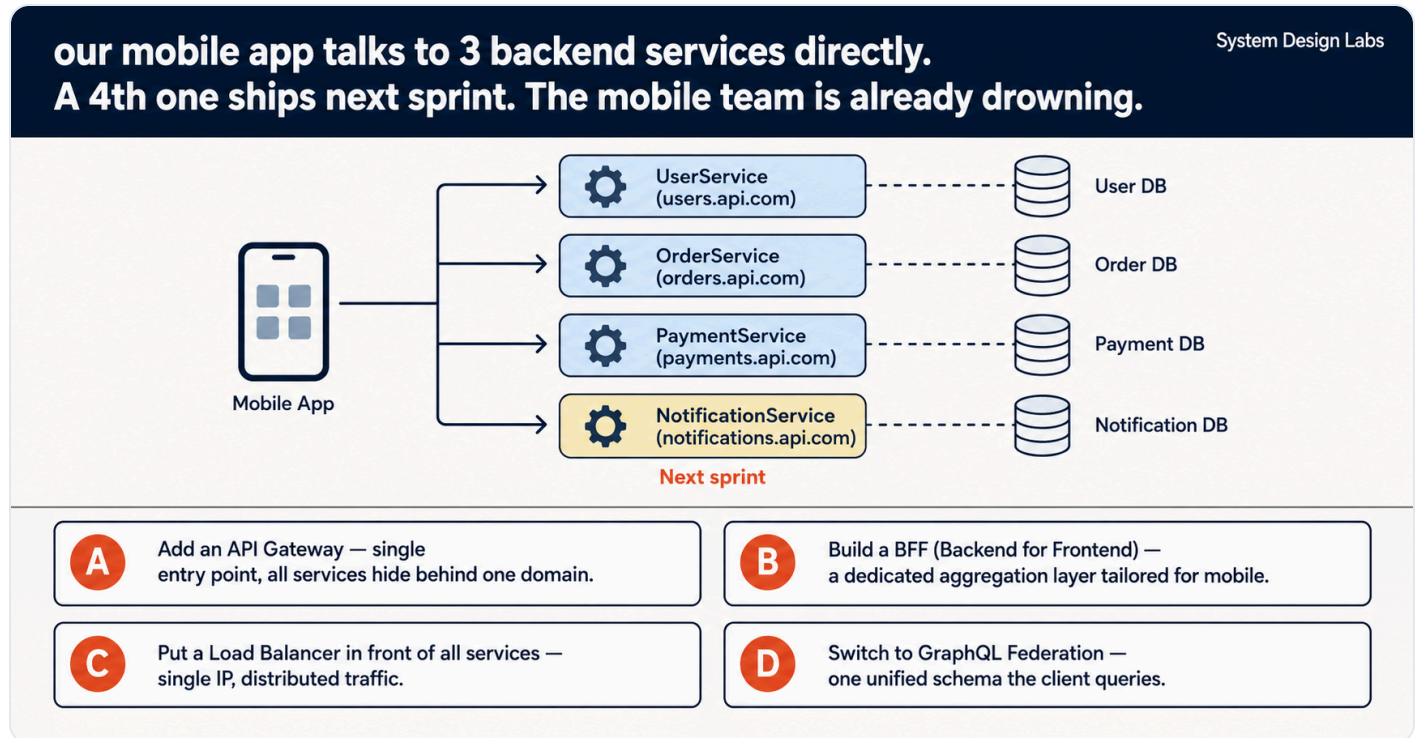
Day 54 Handling Embedding Drift in RAG

Day 56 Tracking Long-Running Async Jobs

Day 58 Observability for LLM Agent Pipelines

Day 60 Architecting a SaaS Platform from Scratch

Decoupling Mobile from Backend Services



our mobile app talks to 3 backend services directly.

A 4th one ships next sprint. The mobile team is already drowning.

Every new service means a new domain to whitelist, a new auth scheme to wire, and a new error shape to parse. You're asked to reduce coupling before NotificationService lands.

Here's the setup:

Mobile → UserService (users.api.com)

Mobile → OrderService (orders.api.com)

Mobile → PaymentService (payments.api.com)

...and NotificationService next sprint.

The client is doing routing the backend should be doing. What do you do?

A Add an API Gateway — single entry point, all services hide behind one domain.

B Build a BFF (Backend for Frontend) — a dedicated aggregation layer tailored for mobile.

C Put a Load Balancer in front of all services — single IP, distributed traffic.

D Switch to GraphQL Federation — one unified schema the client queries.

Answer 1

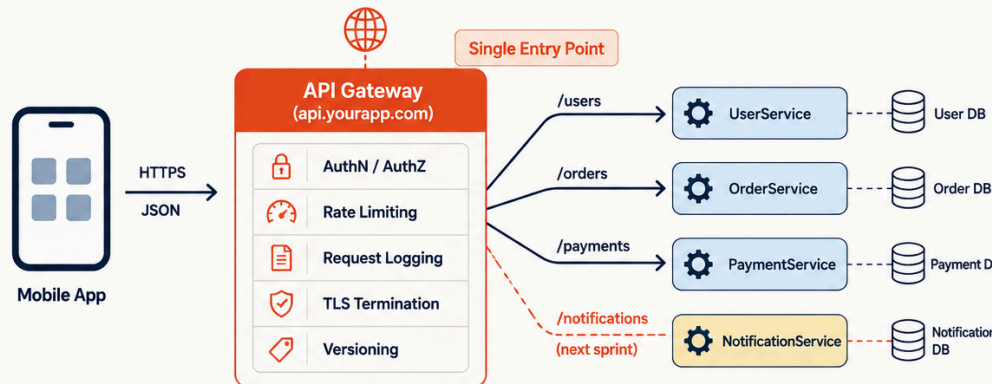
✓ CORRECT ANSWER

Why A wins (API Gateway)

One domain. One auth flow. One error contract. The client stops caring that UserService, OrderService, and PaymentService live on different hosts — the gateway hides them behind **api.yourapp.com/users, /orders, /payments**.

When NotificationService ships next sprint, you add one route.

The mobile app ships **zero changes**.



Centralizes What Matters

- Security**
AuthN/AuthZ in one place.
- Performance**
Rate limiting & traffic control.
- Observability**
Consistent logging & monitoring.
- Reliability**
TLS termination & fail-fast.
- Consistency**
Versioning & error contract enforcement.



That's what "future-proof" actually means at the transport layer:
new services cost nothing on the client side.

Battle-tested at scale:
Netflix, Stripe, and every mature fintech run some form of this.



Why A wins (API Gateway):

One domain. One auth flow. One error contract. The client stops caring that UserService, OrderService, and PaymentService live on different hosts — the gateway hides them behind **api.yourapp.com/users, /orders, /payments**. When NotificationService ships next sprint, you add one route. The mobile app ships **zero changes**. That's what "future-proof" actually means at the transport layer: new services cost nothing on the client side.

It also centralizes the things you don't want scattered across 4 codebases: authN/authZ, rate limiting, request logging, TLS termination, versioning. Netflix, Stripe, and every mature fintech run some form of this.

Answer 2

Why B is the trap answer (BFF):

BFF is a legitimate pattern — and it's close enough to fool a lot of senior engineers. But BFF solves a different problem: *data shape per client type*. Mobile needs a slim payload, web needs a richer one. BFF gives each client its own backend that aggregates and reshapes data.

In this scenario, the pain isn't payload shape — it's domain sprawl and coupling. Putting a BFF here trades 4 URLs for 1 URL + 1 extra service you own end-to-end. That's the wrong tradeoff.

Answer 3

Why C is wrong (Load Balancer):

A load balancer distributes traffic across **identical instances of the same service**. It doesn't route between *different* services on different domains. Putting an LB in front of UserService + OrderService doesn't make sense — it has no concept of "send /orders here and /users there." That's application-layer routing, which is exactly what an API Gateway does.

Answer 4

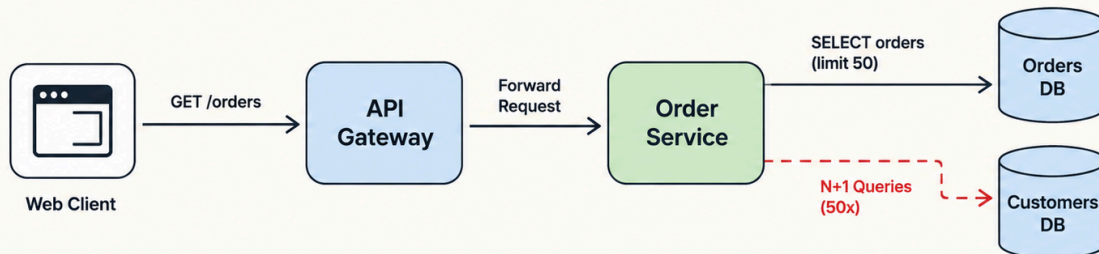
Why D is wrong here (GraphQL Federation):

GraphQL Federation solves **schema unification**, not transport-layer coupling. You'd still need a gateway in front of the federation, and you'd rewrite every backend service to expose a GraphQL subgraph. That's a multi-quarter migration to solve a problem you can fix in a sprint with a gateway.

Killing the N+1 Query Problem

**Your /orders endpoint loads 50 orders on a page.
P95 is 2.4s. The DB's fine. The app server's fine. Nothing's on fire.**

System Design Labs



A Eager-load the relation — include: { customer: true } on the Prisma query. One JOIN, done.

B Add a DataLoader in front of the customer lookup — batches the 50 IDs into one WHERE id IN (...) behind the scenes.

C Cache the customer by ID in Redis — every lookup hits cache first, DB only on miss.

D Denormalize customer_name onto the orders table — read it straight from the orders row, no join, no second query.

Your /orders endpoint loads 50 orders on a page.

P95 is 2.4s. The DB's fine. The app server's fine. Nothing's on fire.

Then you open the query log.

51 queries per request. One SELECT for the orders list. Then 50 more — one per order — to fetch the customer. The ORM is doing lazy-load on order.customer inside your map.

Classic N+1. You've seen it before. The fix is "obvious" — until the team meeting, where three engineers propose three different things and everyone thinks they're right.

Here's what's on the table:

A Eager-load the relation — include: { customer: true } on the Prisma query. One JOIN, done.

B Add a DataLoader in front of the customer lookup — batches the 50 IDs into one WHERE id IN (...) behind the scenes.

C Cache the customer by ID in Redis — every lookup hits cache first, DB only on miss.



Denormalize customer_name onto the orders table — read it straight from the orders row, no join, no second query.

Answer 1

✓ CORRECT ANSWER

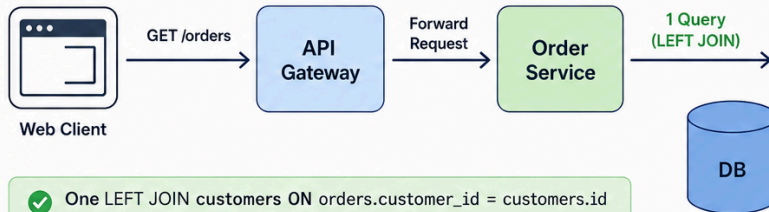


Why A wins:

Eager-load the relation — include: { customer: true }
on the Prisma query. **One JOIN, done.**

A

✓ What changes



SQL shape (conceptual)

```

SELECT o.id, o.created_at, o.total,
       c.id AS customer_id, c.name AS customer_name,
       c.email AS customer_email
FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.id
WHERE o.user_id = ?
ORDER BY o.created_at DESC
LIMIT 50 OFFSET 0;
  
```



Performance

P95 drops from 2.4s
to ~80ms.



Simplicity

No new infrastructure.
No new failure modes.



Reliability

Fewer moving parts.
Fewer things to break.



The real lesson

The N+1 problem in an ORM almost always has the same root cause: a **lazy relation** the developer didn't realize was lazy. Prisma's include, Sequelize's include, TypeORM's relations — **every ORM has this**. Reach for fancier patterns only when JOIN genuinely can't solve it.

Why A wins: One LEFT JOIN customers ON orders.customer_id = [customers.id](#) returns the whole page in a single round trip. P95 drops from 2.4s to ~80ms. No new infrastructure. No new failure modes. The N+1 problem in an ORM almost always has the same root cause: a lazy relation the developer didn't realize was lazy. Prisma's include, Sequelize's include, TypeORM's relations — every ORM has this. Reach for fancier patterns only when JOIN genuinely can't solve it.

Answer 2

Why B is the trap (DataLoader): Beautiful pattern — but earns its keep in GraphQL where the resolver graph fans out in ways a single JOIN can't express. Here you have one list endpoint with one relation. A JOIN is strictly simpler and strictly faster. You're solving a problem you don't have and paying the complexity tax forever.

Answer 3

Why C is wrong (Redis): Cache-first fixes the symptom, not the query pattern. You still issue 50 cache GETs per request, now own a cache invalidation problem every time a customer updates, and cold-hit every ID after a deploy. Cache is a scaling tool, not an N+1 tool.

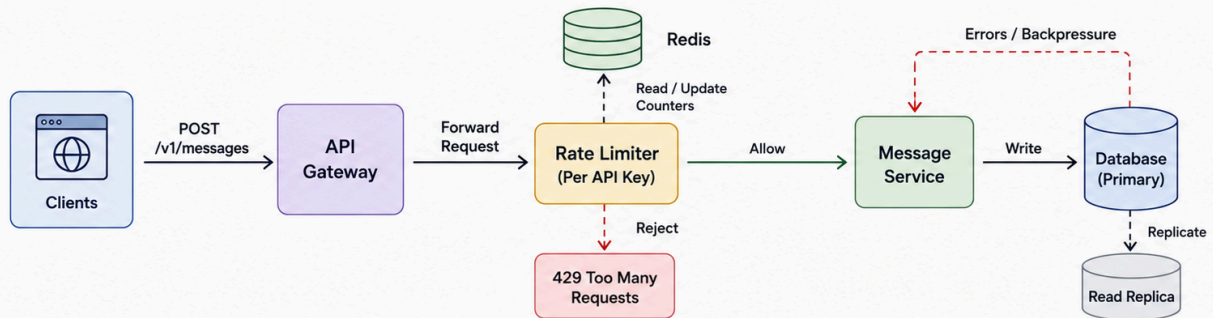
Answer 4

Why D is the career-long argument (denormalize): Legitimate at Instagram scale. Also how you end up with a customer's maiden name three years after they got married because nobody wrote the update path. Denormalization is a write-amplification decision masquerading as a read-optimization. Do it when the JOIN genuinely can't hit your latency SLO at your scale — not before.

Rate Limiting Without Boundary Bursts

You run a SaaS API. POST /v1/messages is the money endpoint. Customers keep getting 429s at minute boundaries.

System Design Labs



A

Fixed Window — bucket per (api_key, minute). Increment, check, reject. Simple, fast, one Redis key per user.

B

Sliding Window Log — store every request timestamp in Redis. Count requests in the last 60s on every call. Exact to the millisecond.

C

Token Bucket — 100 tokens per key, refill at 100/60 per second. Each request takes a token. Bursts up to 100, then shaped.

D

Leaky Bucket — queue incoming requests, dispatch at a constant 1.66/sec. Excess requests wait or get dropped.

You run a SaaS API. POST /v1/messages is the money endpoint.

Plan limit: 100 requests/minute per API key. Customers keep getting 429s at minute boundaries even though their average RPS is well under the cap.

Here's what's happening in prod:

- 12:59:58 → customer sends 90 requests (batched job kicked off)
- 13:00:02 → same customer sends 90 more (next tick)
- Your counter resets at 13:00:00
- Both bursts pass. 180 requests in 4 seconds. Your downstream database starts crying.

Now Support is on your back about 429s, and SRE is on your back about traffic spikes. You need to replace the limiter this week.

FOUR OPTIONS ON THE TABLE:

A

Fixed Window — bucket per (api_key, minute). Increment, check, reject. Simple, fast, one Redis key per user.

B Sliding Window Log — store every request timestamp in Redis. Count requests in the last 60s on every call. Exact to the millisecond.

C Token Bucket — 100 tokens per key, refill at 100/60 per second. Each request takes a token. Bursts up to 100, then shaped.

D Leaky Bucket — queue incoming requests, dispatch at a constant 1.66/sec. Excess requests wait or get dropped.

Answer 1

✓ CORRECT ANSWER



Answer: C — Token Bucket ✓

System Design Labs

Token Bucket allows bursts up to the limit, then enforces a steady rate.

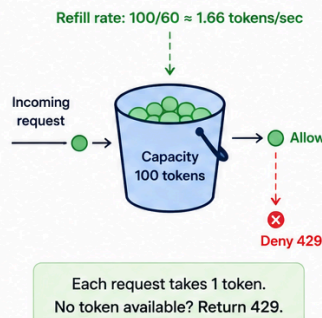
No boundary resets. Smooth traffic. Happy customers. Happy DB.



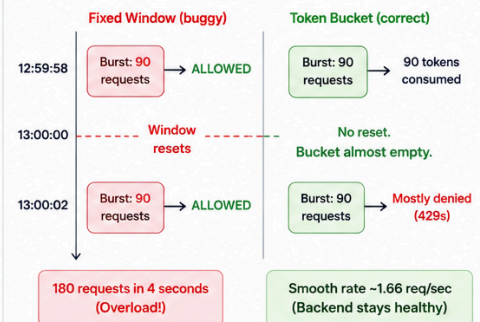
Why C wins (Token Bucket):

- ✓ Token Bucket is what AWS API Gateway, Stripe, GitHub, and basically every mature public API runs. Each API key gets a bucket of 100 tokens, refilling at ~1.66 tokens/sec. Every request takes a token. Empty bucket = 429.
- ✓ This fixes the boundary-burst bug exactly: no window resets. The 12:59:58 burst drains the bucket, and 13:00:02 finds an almost-empty bucket — not a shiny new 100-request allowance.
- ✓ O(1) memory per key, O(1) check, trivially distributed.

How Token Bucket works



Why it fixes the boundary-burst



Why the other three trip up good engineers



A Fixed Window

- ✗ Resets at boundaries → bursts
- ✗ Allows 2x limit in a short time
- ✗ Unfair and spiky traffic

Good for simple, non-critical limits.
Not for money endpoints.



B Sliding Window Log

- ✓ Accurate to the millisecond
- ✗ Stores every request timestamp
- ✗ High memory & CPU at scale
- ✗ Expensive in Redis

Overkill cost for little extra benefit.



C Token Bucket

- ✓ Handles bursts up to limit
- ✓ Smooths traffic over time
- ✓ O(1) memory, O(1) check
- ✓ Easy to distribute
- ✓ Industry standard

Right balance of fairness, performance, and simplicity.



D Leaky Bucket

- ✓ Smooth output rate
- ✗ Adds queuing delay
- ✗ Increases tail latency
- ✗ Complex under load

Good for shaping internal traffic, not for API rate limits.

Answer: C — Token Bucket ✓

Here's why, and why the other three trip up good engineers:

Why C wins (Token Bucket):

Token Bucket is what AWS API Gateway, Stripe, GitHub, and basically every mature public API runs. Each API key gets a bucket of 100 tokens, refilling at ~1.66 tokens/sec. Every request takes a token. Empty bucket = 429.

This fixes the boundary-burst bug exactly: no window resets. The 12:59:58 burst drains the bucket, and 13:00:02 finds an almost-empty bucket — not a shiny new 100-request allowance. O(1) memory per key, O(1) check, trivially distributed.

Answer 2

Why B is the trap (Sliding Window Log):

Mathematically perfect — but stores one Redis entry *per request per key*. At 100k active keys you're looking at real memory pressure and p99 latency spikes. Smart engineers pick it without thinking about the memory cost. That's the tell.

Answer 3

Why A is wrong (Fixed Window):

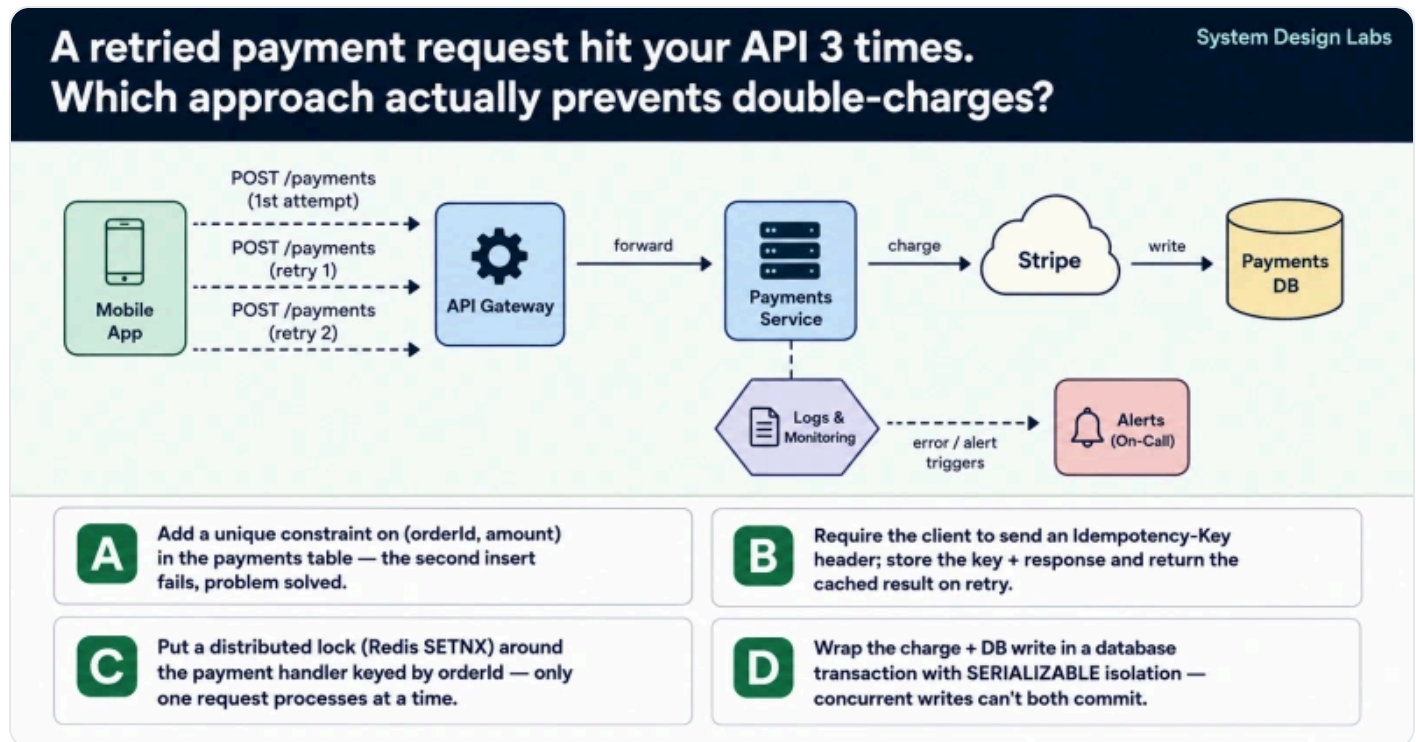
Literally the bug in the post. Counter resets at the minute boundary → customers can burst 2x their limit in a 2-second window. Still the most common production mistake.

Answer 4

Why D is wrong here (Leaky Bucket):

Solves a different problem — shapes *output* by queuing requests at a constant dispatch rate. Great for protecting a fragile downstream. Wrong for a customer-facing API because it adds latency to legitimate bursts. Token Bucket rejects excess; Leaky Bucket delays it. For a public API, reject > delay.

Preventing Duplicate Payment Charges



A user taps "Pay \$499" on checkout.

The spinner hangs. They tap it again. And once more.

Three requests hit your Payments API. Two of them actually succeed. The customer is charged \$1,497 and opens a support ticket before your on-call even sees the alert.

Here's the setup:

Mobile → POST /payments { orderId: "ord_8821", amount: 499 }

Network stalls on the first request → client retries twice

Your service receives 3 near-identical POSTs within 4 seconds

Stripe gets charged 3 times. Your DB writes 3 payment rows.

You have one sprint to make sure a retried request never double-charges again. What do you do?

- A** Add a unique constraint on (orderId, amount) in the payments table — the second insert fails, problem solved.

- B** Require the client to send an Idempotency-Key header; store the key + response and return the cached result on retry.
- C** Put a distributed lock (Redis SETNX) around the payment handler keyed by orderId — only one request processes at a time.
- D** Wrap the charge + DB write in a database transaction with SERIALIZABLE isolation — concurrent writes can't both commit.

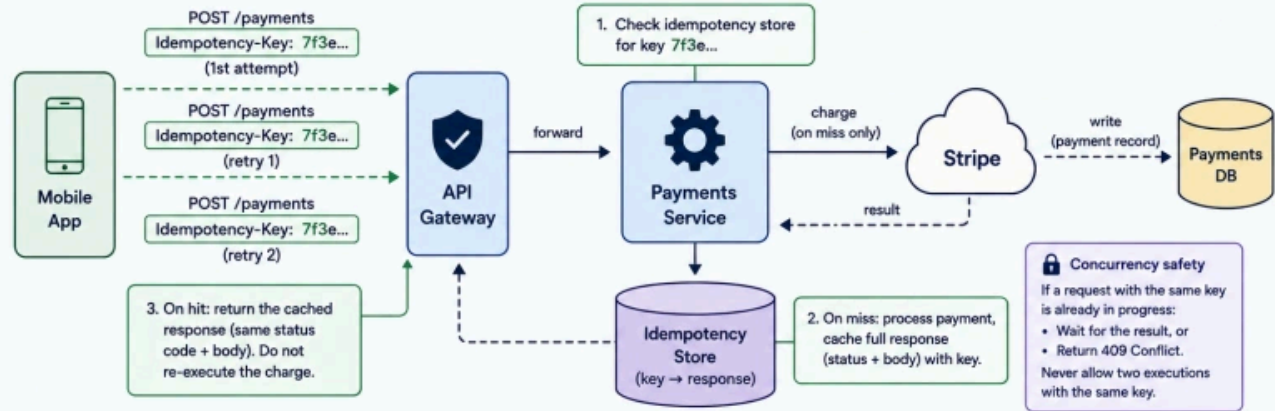
Answer 1

✓ CORRECT ANSWER

Answer: B — Idempotency-Key header ✓

System Design Labs

The server does the work once, stores {key → response}, and returns the same response on retries.



Answer: B — Idempotency-Key header ✓

Here's why, and why the other three are the exact mistakes I've seen in real post-mortems:

Why B wins (Idempotency-Key):

The client generates a UUID *before* the first request and attaches it as `Idempotency-Key: 7f3e...` on every retry of that same logical operation. Your server does the work once, stores {key → response} in a durable store (Postgres, DynamoDB, Redis with persistence — not plain Redis), and on any retry with the same key, it returns the *original* response without re-executing the charge.

Critical detail most people miss: you cache the **full response** (status code + body), not just "I saw this key." If retry #2 arrives while retry #1 is still processing, you make it wait or return a 409 — never let two executions race with the same key.

This is exactly what Stripe (Idempotency-Key), PayPal (PayPal-Request-Id), Shopify, and AWS (ClientRequestToken on half their APIs) all ship. It's the only approach that handles the full failure matrix: network retries, client-triggered retries, load balancer retries, and "the request succeeded but the response got lost" — which is the nastiest case and the one the other three options silently fail on.

Answer 2

Why A is the trap answer (unique constraint on orderId):

This is the answer I've seen in 4 different production codebases, and it's wrong for a subtle reason.

A unique constraint stops the *second insert* from committing — good. But what about the *first charge to Stripe*? In most implementations, you call Stripe's API first, then insert the payment row. So request #1 charges Stripe, DB insert succeeds. Request #2 charges Stripe *again*, then the DB insert fails with a constraint violation. The customer is now charged twice. Your DB looks clean. Your logs say "duplicate order prevented."

Even if you flip the order (insert first, charge second), you've just moved the race condition — now retry #2 fails at the DB, but if retry #1 crashes between insert and charge, you have a payment row with no actual charge. Worse than double-charging. Silent money loss.

A unique constraint is a great *defense-in-depth* layer. It is not the primary mechanism.

Answer 3

Why C is wrong (distributed lock):

A Redis lock on orderId serializes the 3 retries so only one runs at a time. Sounds clean. But it only solves the *concurrent* case. It doesn't help with sequential retries that arrive after the lock has already been released and the response was lost in transit.

Request #1 acquires lock, charges Stripe, releases lock, returns 200 — but the 200 never reaches the client because the connection died. Client retries 30 seconds later. The lock is long gone. You charge again.

Distributed locks are for preventing concurrent execution. Idempotency is for preventing *repeated* execution across time. Different problems.

Also: distributed locks are famously hard to get right. Martin Kleppmann's "How to do distributed locking" post exists for a reason. Don't reach for one unless you actually need the concurrency guarantee.

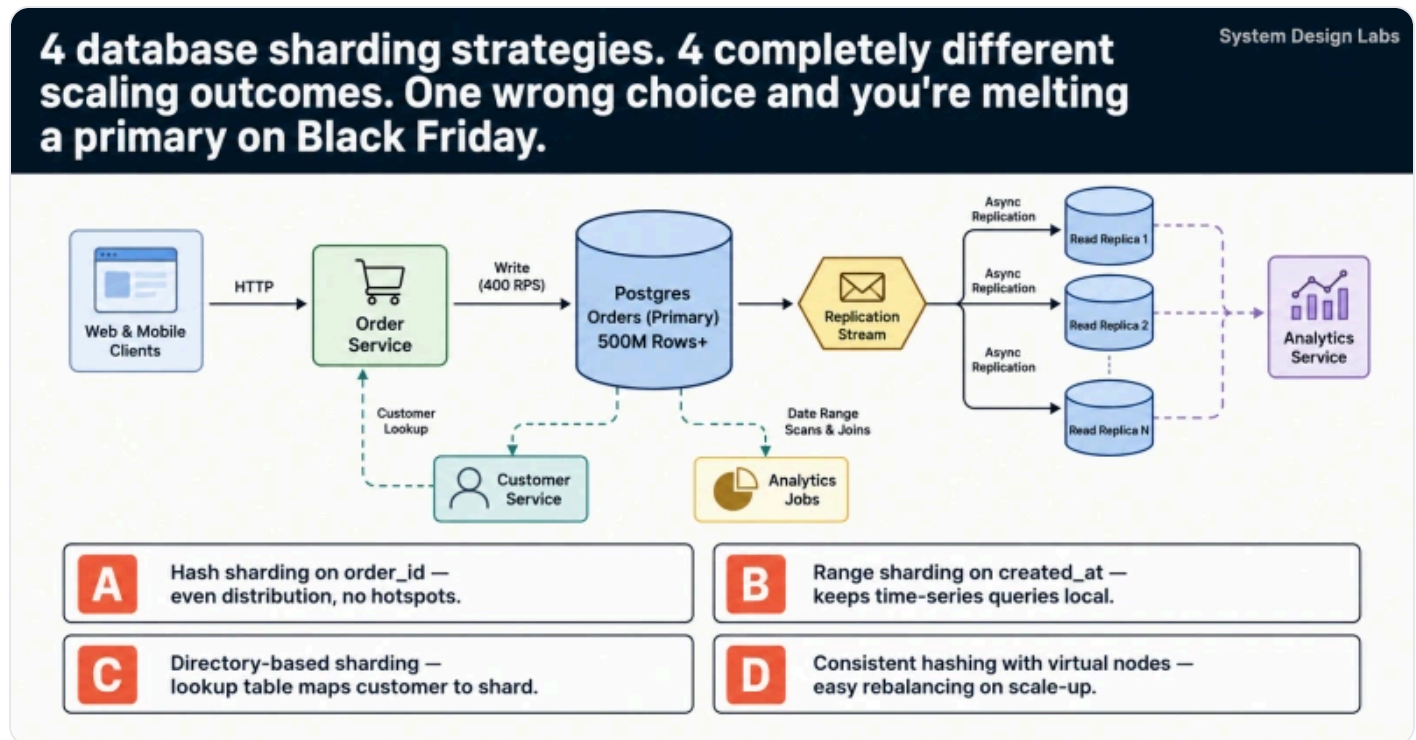
Answer 4

Why D is wrong (SERIALIZABLE transaction):

SERIALIZABLE isolation is about protecting the *database* from concurrent-write anomalies. It has nothing to say about the *external* side effect — the charge to Stripe. Your transaction can roll back and your DB can be pristine, but the HTTP POST to Stripe already went out, and that money is gone.

This is the most common conceptual bug in the "make payments safe" category: treating the database as the boundary of correctness when the real boundary is the external side effect. The moment you call Stripe, the universe has changed. Your DB transaction can't un-change it.

Choosing a Database Sharding Strategy



4 database sharding strategies. 4 completely different scaling outcomes. One wrong choice and you're melting a primary on Black Friday.

Your Postgres orders table just crossed 500M rows.

Range scans that used to take 40ms are creeping past 800ms.

Vertical scaling is dead. You need to shard.

Here is the workload:

- orders = 500M rows, growing 3M/week
- 80% reads = "customer X's last 30 days of orders"
- 15% reads = analytics joins on date ranges
- 5% writes = new orders (steady 400 RPS, 2x on sale days)

Which strategy do you pick?

A Hash sharding on `order_id` — even distribution, no hotspots.

B Range sharding on `created_at` — keeps time-series queries local.

C Directory-based sharding — lookup table maps customer to shard.

D Consistent hashing with virtual nodes — easy rebalancing on scale-up.

Answer 1

✓ CORRECT ANSWER

Answer: C — Directory-based sharding ✓

Here is why the other three trick smart engineers:

Why C wins (Directory-based):

80% of reads are "customer X's orders." You want every order for a customer on exactly one shard. A customer-scoped read hits one node, no scatter-gather fan-out.

You keep a `customer_shard_map` table (cached in Redis). If shard 3 gets hot, you migrate specific heavy customers (whales) without rehashing anything. Targeted rebalancing is the superpower. Figma, Notion, and Slack do this.

Answer 2

Why A is the trap (Hash on `order_id`):

The classic tutorial answer: "Hash the primary key!" It wrecks you here. A single customer's 200 orders scatter across all shards. Every read becomes a 4-shard fan-out gated by your slowest node. P99 latency goes up.

Answer 3

Why B is wrong (Range on `created_at`):

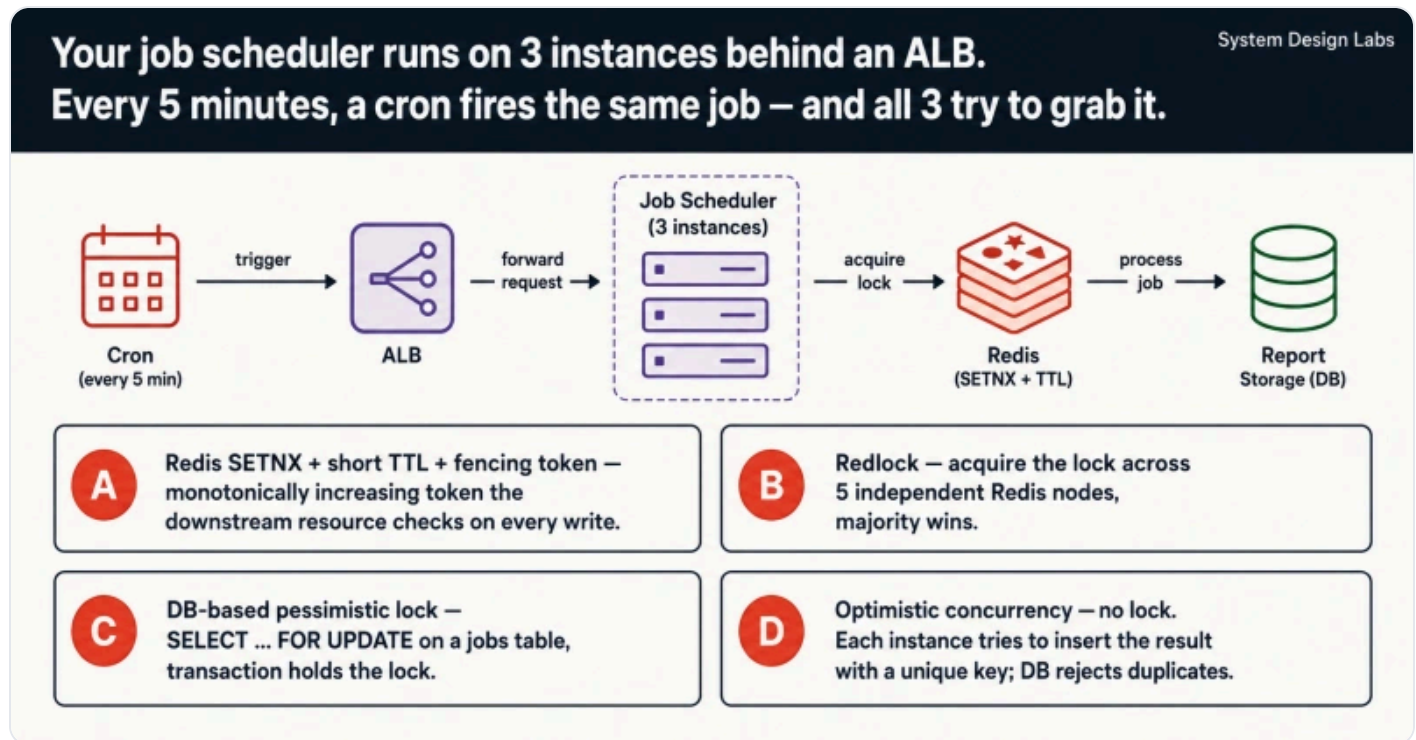
Creates a hot shard by design. Every new order writes to the same shard. Your newest shard absorbs 100% of writes and 70% of reads. On Black Friday, this shard melts while others sit idle.

Answer 4

Why D is wrong (Consistent hashing):

Great for uniform data, but you can't migrate specific heavy tenants. If four whales land on the same shard, you are stuck.

Safe Distributed Locks for Cron Jobs



Your job scheduler runs on 3 instances behind an ALB.

Every 5 minutes, a cron fires the same "generate-daily-report" job — and all 3 instances try to grab it.

You reach for the classic fix: a Redis SETNX lock.

SET job:daily-report locked NX EX 300 → only one instance wins. Done.

Works perfectly for a month. Until 2am on a Saturday.

Instance B wins the lock. Starts the job. 60 seconds in, the pod gets OOM-killed. The lock is still there. TTL says 300 seconds. The report runs once every 5 minutes.

Nothing processes that report for 4 more minutes. Then instance A grabs it — but instance B's pod restarted and it's ALSO trying to run it now. Both instances processing the same job.

Worse: a slow GC pause on instance A makes it hold the lock past TTL. Redis quietly hands the lock to instance C. Now A thinks it owns the lock. C also thinks it owns the lock. Both write to the same output.

What's the right pattern here?

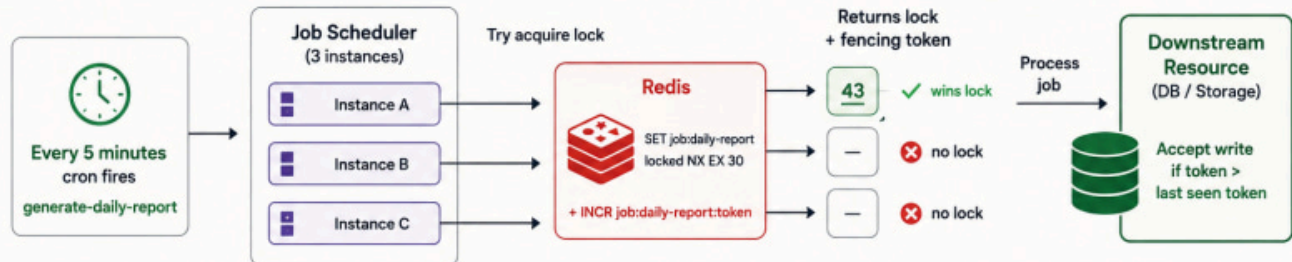
- A** Redis SETNX + short TTL + fencing token — monotonically increasing token the downstream resource checks on every write.
- B** Redlock — acquire the lock across 5 independent Redis nodes, majority wins.
- C** DB-based pessimistic lock — SELECT ... FOR UPDATE on a jobs table, transaction holds the lock.
- D** Optimistic concurrency — no lock. Each instance tries to insert the result with a unique key; DB rejects duplicates.

Answer 1

✓ CORRECT ANSWER

A — SETNX + short TTL + fencing token ✓

System Design Labs

Monotonically increasing token that the downstream resource checks on every write.**Why this survives failures**

- 1 If the lock holder dies or is paused, the TTL expires.
- 2 Another instance acquires the lock and gets a higher token (e.g., 44).
- 3 If the old instance wakes up and tries to write with token 43, the resource rejects it. Only the highest token wins.

Key guarantee

The downstream resource is the source of truth, not the lock service.

Why the others fail in the scenario**B****Redlock**

✗

- Still time-based and unsafe under process pauses and partitions.
- Martin Kleppmann showed how it can grant the lock to multiple clients.

C**DB-based pessimistic lock**

✗

- If the process dies, the DB lock is released — good.
- But a long GC pause can still overlap work, causing duplicate side effects. No fencing = resource can't tell.

D**Optimistic concurrency**

✗

- Prevents duplicate final results, but all instances still do the work.
- Wastes resources and can overwhelm downstream systems.



Only A (SETNX + short TTL + fencing token) provides correctness under pauses, failures, and partitions.

The fencing token makes the downstream resource the final authority.

Answer: A — SETNX + short TTL + fencing token ✓

Here's why, and why the other three trick smart engineers:

Why A wins (SETNX + TTL + fencing token):

A lock alone can't protect a distributed resource. You need to assume the lock holder might be dead, paused, or partitioned — and the lock might get reassigned while they still think they own it. This is exactly the scenario above: GC pause, TTL expires, Redis gives the lock to someone else, original owner wakes up and keeps writing.

The fencing token fixes it. Every time a lock is acquired, Redis (or the lock service) hands out a monotonically increasing number — 42, 43, 44. The downstream resource (the DB, the storage system, the output queue) remembers the highest token it's seen and rejects any write with a lower token.

So when instance A wakes up from its GC pause holding token 42, and instance C is already writing with token 43, A's write gets rejected by the DB itself. The resource is the source of truth — not the lock service.

This is what Google's Chubby does. It's what Martin Kleppmann recommends in "How to do distributed locking." It's the only pattern on this list that's actually safe against process pauses.

Answer 2

Why B is the trap answer (Redlock):

Redlock is the pattern most senior engineers reach for. It looks rigorous. Martin Kleppmann wrote a now-famous post (2016) showing Redlock is unsafe in the exact scenario above: any process can pause long enough to lose its lock without noticing.

More importantly — Redlock without a fencing token has the same fundamental flaw as SETNX. If your resource can't verify a token on write, Redlock doesn't save you. You're maintaining 5 Redis clusters for a problem a fencing token solves on 1.

Answer 3

Why C is wrong (DB pessimistic lock):

SELECT ... FOR UPDATE inside a transaction works — for tiny critical sections. The lock auto-releases when the transaction ends or the connection dies.

But the lock lives inside a single DB transaction. If your job is "generate a daily report" that takes 60 seconds of S3 writes and external API calls, you're holding a row lock on your primary DB for 60 seconds per job. Connection pool starved. Every other query blocked behind it.

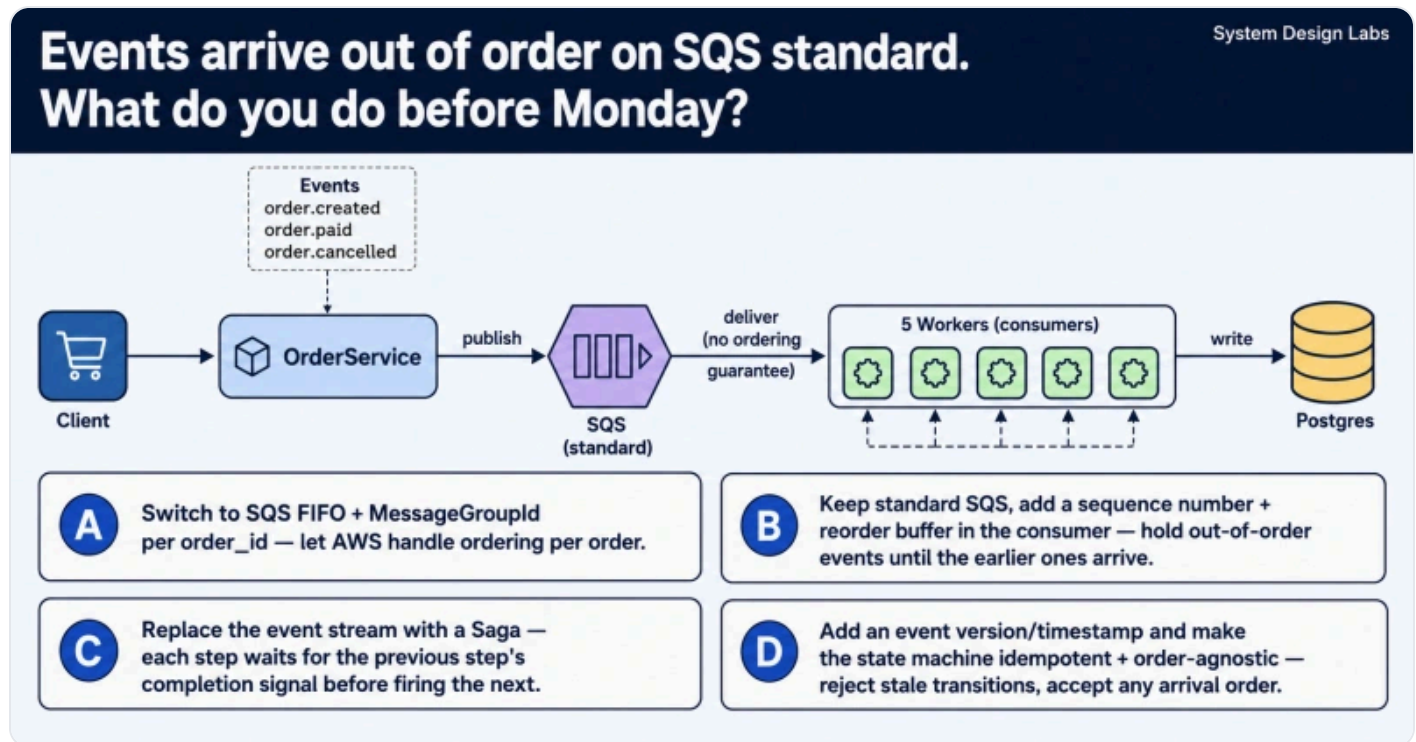
Answer 4

Why D is wrong here (optimistic concurrency):

Optimistic concurrency is beautiful when it fits — no lock service, no lock holder, no stale-lock problem. Each instance tries the operation; the DB unique constraint rejects losers.

It breaks when the work isn't idempotent or isn't a single write. Three instances racing to do all of that means triple the S3 writes, triple the Slack pings, triple the compute cost.

Event Ordering in a Message Queue



Your order service publishes 3 events per order: created, paid, cancelled.

Standard SQS queue, 5 consumers, ~2K orders/minute at peak.

Last night, prod alerted: 47 orders stuck in "cancelled" state with no "created" record. Customers got refunded for orders that were never marked as placed. Finance is not happy.

You dig in. The order was placed, paid, and cancelled within 400ms. Three messages hit SQS in order. Three consumers picked them up in parallel. "cancelled" got processed first. State machine rejected "created" as invalid. Data is now wrong.

Here's the setup:

OrderService → SQS (standard) → 5 workers → Postgres

Events: order.created → order.paid → order.cancelled

Problem: workers process in parallel, SQS standard doesn't guarantee order, downstream state machine breaks when events arrive out of sequence.

You have until Monday. What do you do?

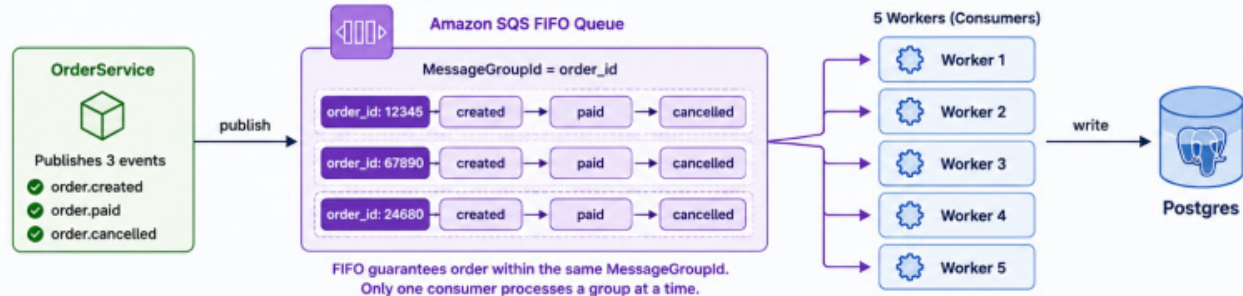
- A** Switch to SQS FIFO + MessageGroupId per order_id — let AWS handle ordering per order.
- B** Keep standard SQS, add a sequence number + reorder buffer in the consumer — hold out-of-order events until the earlier ones arrive.
- C** Replace the event stream with a Saga — each step waits for the previous step's completion signal before firing the next.
- D** Add an event version/timestamp and make the state machine idempotent + order-agnostic — reject stale transitions, accept any arrival order.

Answer 1

✓ CORRECT ANSWER

Answer: A — SQS FIFO + MessageGroupId per order_id

Per-order ordering guaranteed. High throughput. Minimal changes. Ship it Monday.



✓ Answer: A — SQS FIFO + MessageGroupId per order_id

Why A wins:

You don't need global ordering — you need *per-order* ordering. Set `MessageGroupId = order_id`. FIFO guarantees messages with the same group ID are delivered in exact send order, with only one consumer processing that group at a time. Your 2K orders/min still fan out across 5 workers — throughput is unchanged. Order #12345's `created` → `paid` → `cancelled` always lands in sequence. Cost: a few ms latency. Ship it Monday.

Answer 2

Why B is the trap (reorder buffer):

Fools senior engineers on the whiteboard. You'd end up building: in-memory buffer per `order_id` + TTL logic + dead-letter path for gaps + crash recovery + stuck buffer monitoring. You just rebuilt FIFO in app code. With more bugs. And you own every edge case. B is valid only when you *can't* use FIFO (e.g. Kafka with a taken partition key). That's not this situation.

Answer 3

Why C is wrong (Saga):

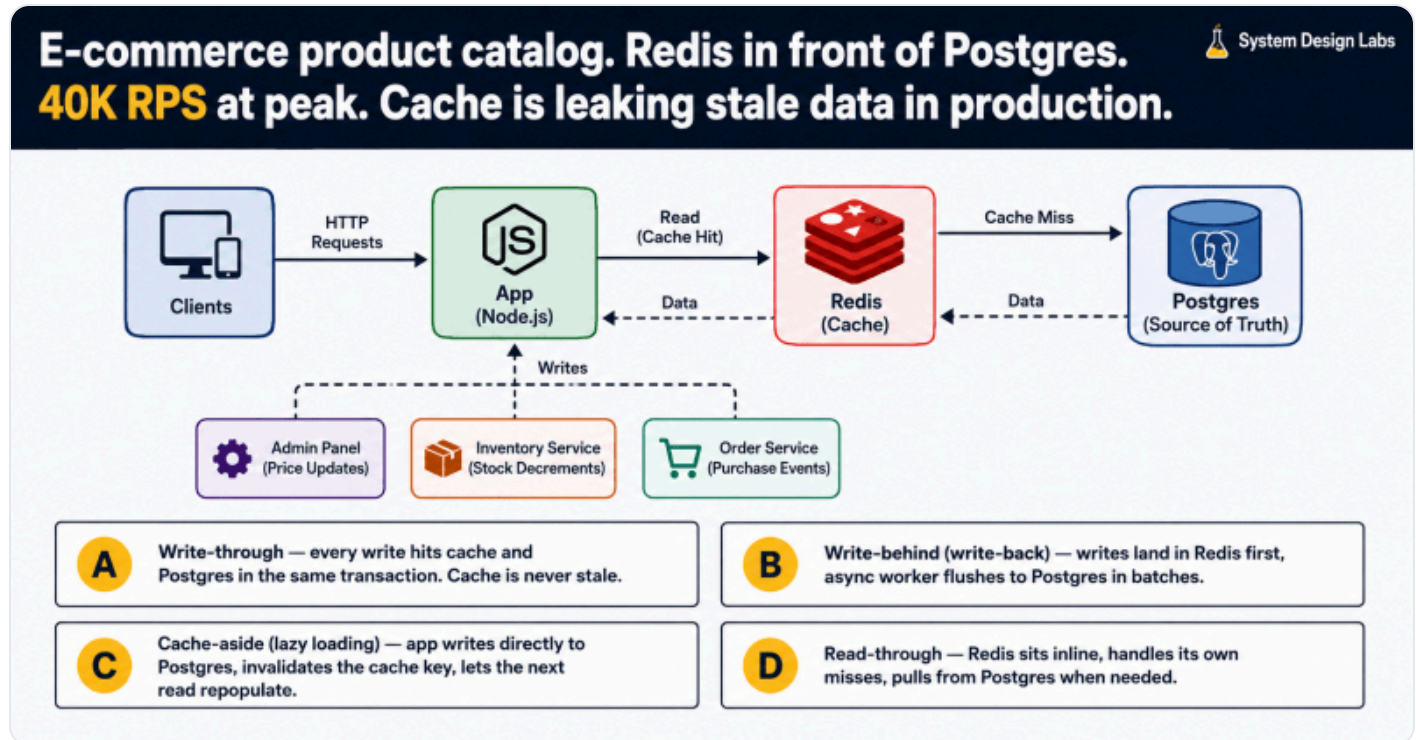
Saga is for long-running distributed transactions with compensating actions (book flight → charge card → reserve hotel). Your problem is a *state stream for one entity* — not a multi-service transaction. Forcing it into a Saga turns async events into sync RPC, increases coupling, and still doesn't fix ordering.

Answer 4

Why D is wrong (event versioning):

Partially works — but you'd *drop* legitimate events. "cancelled" arrives first, moves state forward, then "created" gets rejected. DB says order is cancelled with no creation record. To make D work correctly you need full event sourcing (6-month rewrite, not a Monday fix).

Keeping Cache and Database in Sync



E-commerce product catalog. Redis in front of Postgres. 40K RPS at peak.

In staging, cache hit ratio sits at 94%. Response times under 20ms. Everything looks clean.

Ship to prod. Within 48 hours, support tickets start rolling in. Customers seeing wrong prices. Old stock counts. Products showing "available" that shipped out three hours ago.

Here's the setup:

App (Node.js) → Redis (cache) → Postgres (source of truth)

Writes come from: admin panel (price updates), inventory service (stock decrements), order service (purchase events).

Reads come from: product pages, search results, checkout flow.

The cache is leaking stale data in production. Your team is debating the invalidation strategy. What do you ship?

- A** Write-through — every write hits cache and Postgres in the same transaction. Cache is never stale.

B Write-behind (write-back) — writes land in Redis first, async worker flushes to Postgres in batches.

C Cache-aside (lazy loading) — app writes directly to Postgres, invalidates the cache key, lets the next read repopulate.

D Read-through — Redis sits inline, handles its own misses, pulls from Postgres when needed.

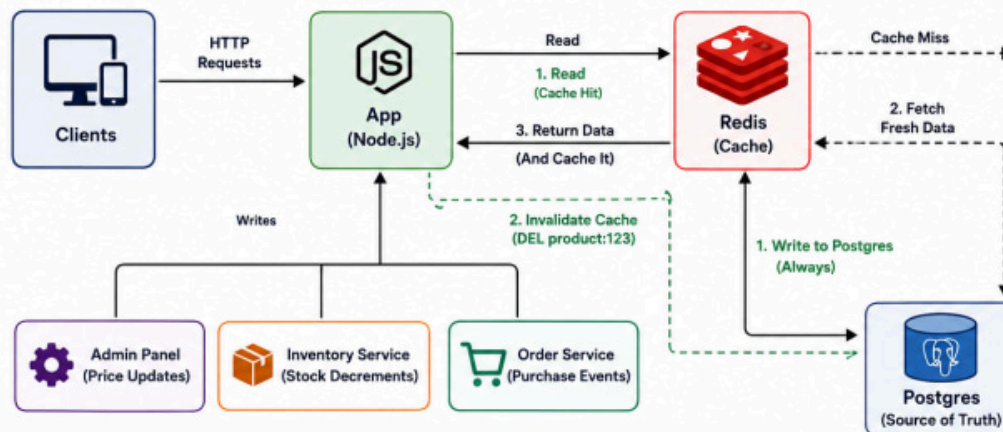
Answer 1

✓ CORRECT ANSWER

ANSWER: C — Cache-aside (lazy loading)

System Design Labs

App writes to Postgres → invalidates Redis → next read repopulates.

**Why C Wins**

- ✓ Multiple writers update Postgres independently.
- ✓ Each writer invalidates Redis. Next read repopulates fresh data.
- ✓ Postgres remains the single source of truth.
- ✓ Redis is disposable. If it dies, the app still works (correct, just slower).
- ✓ Battle-tested by Shopify, Etsy, and AWS reference architectures.

✓ Answer: C — Cache-aside

Why C wins:

Multiple write paths (admin panel, inventory service, order service) all hit Postgres independently. Cache-aside fits perfectly: each writer updates Postgres → invalidates the cache key (DEL product:123) → next read misses → fetches fresh data → repopulates Redis. Postgres stays the uncontested source of truth. Redis is just a disposable read accelerator. If Redis dies, the app still works — slower, but correct. This is what Shopify, Etsy, and most AWS reference architectures run in prod.

Answer 2**Why A is the trap (Write-through):**

"We write to both atomically" — except there's no distributed transaction between Redis and Postgres. You write to Postgres, then write to Redis. If the Redis write fails (network blip, timeout), you have consistent Postgres + stale Redis. The exact bug you were trying to fix. Worse, write-through couples every service to Redis — now your inventory service and order service both need Redis credentials. One Redis outage takes down your entire write path. Works great for single-service/single-writer systems. Falls apart with multiple writers.

Answer 3

Why B is wrong (Write-behind):

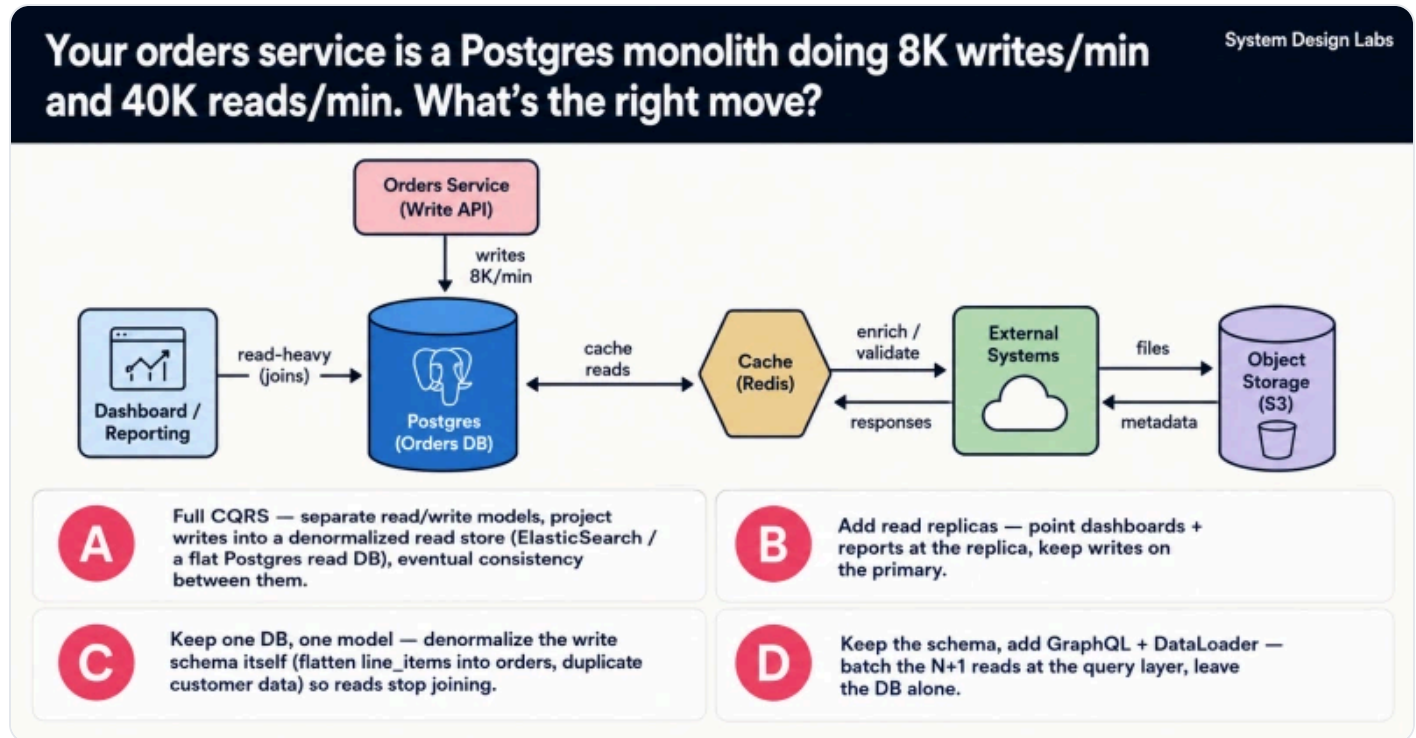
Redis becomes your source of truth. Redis loses data during failovers, OOM kills, AOF lag. "We might lose the last few price updates" is not defensible in a postmortem when real money is involved. Write-behind belongs in metrics pipelines and analytics counters — not e-commerce write paths.

Answer 4

Why D is wrong (Read-through):

Only solves the read side. Your writes are still going through 3 different services with nothing telling the cache. Read-through doesn't fix stale data from multiple writers — you'd still need invalidation on top, which is just cache-aside with extra steps.

Splitting Read and Write Data Models



Your orders service is a Postgres monolith doing 8K writes/min and 40K reads/min.

Same tables. Same schema. Everything's grinding.

The write side needs a normalized schema — orders, line_items, payments, shipments, addresses. Clean FKs, ACID, no duplicated data. The read side wants the opposite — the dashboard joins 7 tables to render one order card, and the reporting queries are lighting up the CPU at 85% every morning at 9am.

You've already tuned indexes. You've already added caching. The fundamental problem isn't hardware — the read model and the write model want **different shapes of the same data**, and you've been pretending they're the same thing for two years.

Here's the decision on the table:

- A** Full CQRS — separate read/write models, project writes into a denormalized read store (ElasticSearch / a flat Postgres read DB), eventual consistency between them.
- B** Add read replicas — point dashboards + reports at the replica, keep writes on the primary.

C Keep one DB, one model — denormalize the write schema itself (flatten line_items into orders, duplicate customer data) so reads stop joining.

D Keep the schema, add GraphQL + DataLoader — batch the N+1 reads at the query layer, leave the DB alone.

Answer 1

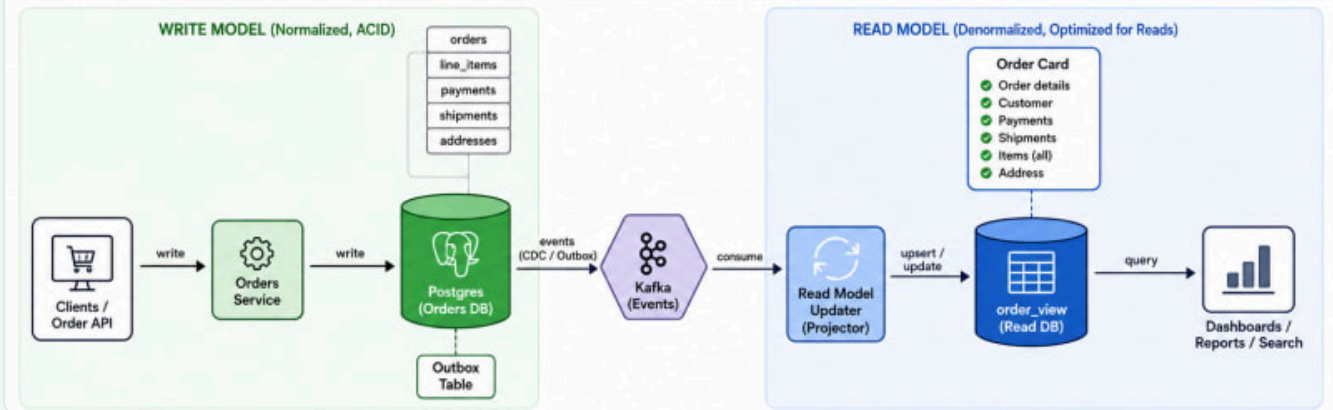
✓ CORRECT ANSWER

A

Full CQRS — Separate Read/Write Models

System Design Labs

Project writes into a denormalized read store. Eventual consistency.



✓ Answer: A — Full CQRS

Why A wins:

The pain isn't "reads are slow" — it's that the write side wants a normalized schema and the read side wants a denormalized one, and you're forcing them to share. No amount of hardware fixes a schema shape mismatch. Write side stays clean: orders, line_items, payments, all normalized, ACID, FK-enforced. Read side is a projection: a flat order_view table (or Elasticsearch index) where every order card renders from one row, zero joins. Projector keeps them in sync (outbox → Kafka → read-model updater, or CDC). You eat eventual consistency — the dashboard might lag by a few hundred ms. For 95% of reads (dashboards, reports, search), that's invisible. For the 5% that needs strong consistency (user's own order confirmation), read from the write side directly.

Answer 2**Why B is the trap (Read replicas):**

B fixes read *load*. Not read *shape*. The 7-table join is still a 7-table join — it just runs on different hardware. You've moved the CPU bill, not removed it. Latency drops for 6 months while traffic grows, then you're back where you started with 3 replicas all running the same expensive joins. B is right when the joins are structurally fine but the primary is overwhelmed. Wrong when the joins themselves are the problem.

Answer 3

Why C is wrong (Denormalize the write model):

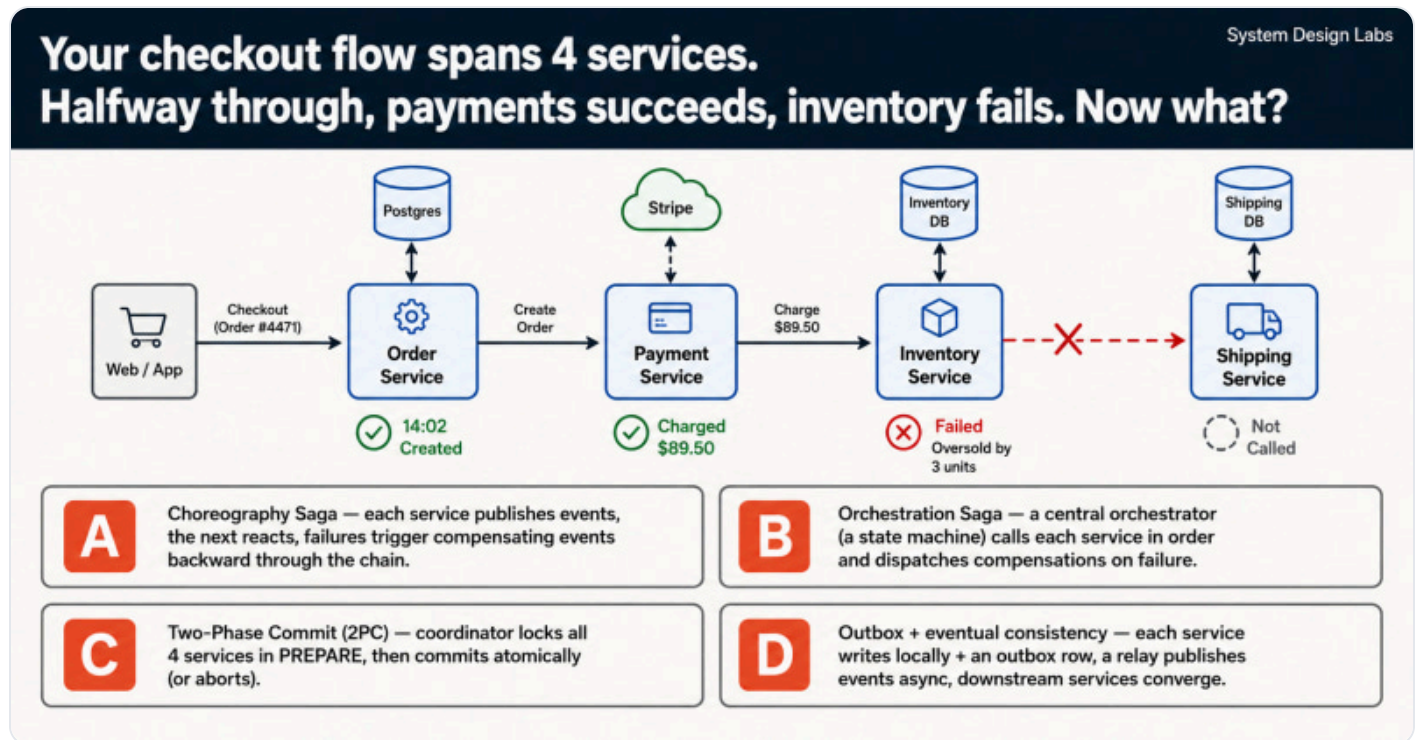
Every line-item change now rewrites the order row. Every address update cascades through every order ever placed. Write amplification compounds at 8K writes/min. Schema ossifies — adding a field to line_items means migrating millions of denormalized rows. Classic "avoid the architecture conversation" move. Buys a quarter, mortgages two.

Answer 4

Why D is wrong (GraphQL + DataLoader):

DataLoader batches API calls, not joins. The 7-table join happens in the database, not in the application. 3 months of GraphQL migration later, reads are exactly as slow as before. Right answer when clients are the problem. Wrong answer when the DB schema is.

Distributed Transactions Across Services



Order #4471 just hit your system at 14:02. The flow:

1. OrderService creates the order (Postgres) ✓
2. PaymentService charges \$89.50 to Stripe ✓
3. InventoryService tries to reserve the SKU → fails (oversold by 3 units)
4. ShippingService never gets called

You're now holding the customer's money, no inventory to ship, and an order stuck in PENDING. In a monolith you'd wrap all 4 in a transaction and ROLLBACK. You can't do that across 4 services with 4 databases.

What do you reach for?

- A** Choreography Saga — each service publishes events, the next reacts, failures trigger compensating events backward through the chain.
- B** Orchestration Saga — a central orchestrator (a state machine) calls each service in order and dispatches compensations on failure.

C Two-Phase Commit (2PC) — coordinator locks all 4 services in PREPARE, then commits atomically (or aborts).

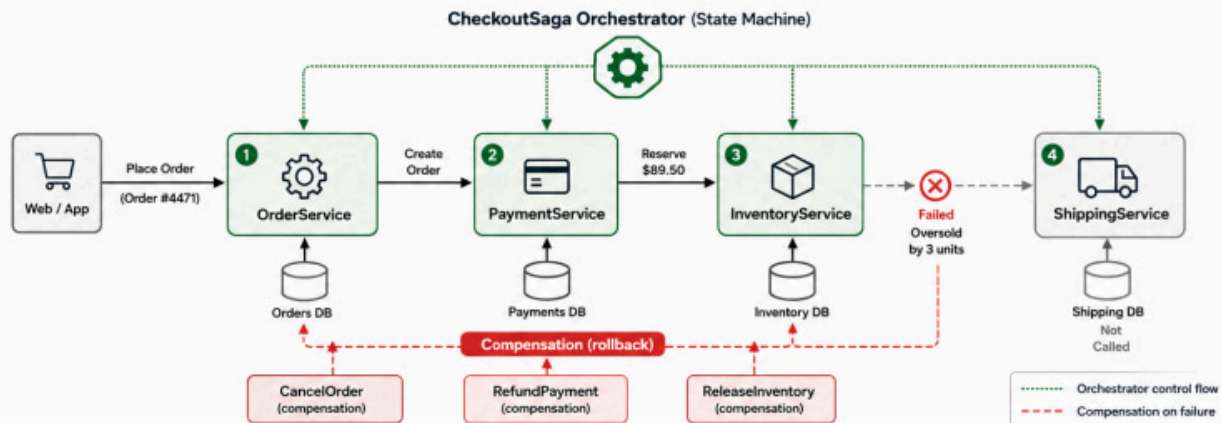
D Outbox + eventual consistency — each service writes locally + an outbox row, a relay publishes events async, downstream services converge.

Answer 1 ✓ CORRECT ANSWER

ANSWER: B Orchestration Saga

System Design Labs

Central orchestrator calls each service in order and dispatches compensations on failure.



CheckoutSaga Execution — Order #4471

Step	Service	Status	Started At	Finished At	Duration	Result
1	OrderService	✓ SUCCEEDED	14:02:01	14:02:01	120ms	Orderid: ord_4471
2	PaymentService	✓ SUCCEEDED	14:02:01	14:02:03	1.2s	Charged: ch_9f82 Amount: \$89.50
3	InventoryService	✗ FAILED	14:02:03	14:02:04	850ms	Oversold by 3 units
4	ShippingService	○ PENDING	—	—	—	Not called

Compensation Actions (executed in reverse order)			
✓ ReleaseInventory	SUCCEEDED	14:02:04	Released reservation
✓ RefundPayment	SUCCEEDED	14:02:06	Refundid: rf_12ab
✓ CancelOrder	SUCCEEDED	14:02:06	Order marked CANCELED

Saga finished
Order #4471
ROLLED BACK

✓ Answer: B — Orchestration Saga

Why B wins:

4-step checkout with strict ordering + per-step compensations = exactly what an orchestrator is built for. Stand up a CheckoutSaga state machine (Temporal, AWS Step Functions, Camunda, or hand-rolled). It calls each service in order: OrderService → PaymentService → InventoryService → ShippingService. Each step has a registered compensation: CancelOrder, RefundPayment, ReleaseInventory. When Inventory fails, the orchestrator walks backward and refunds automatically. State is durable, observable, replayable. At 3am you open the orchestrator UI and see exactly which step failed with full inputs/outputs.

Answer 2

Why A is the trap (Choreography):

Elegant on paper. Falls apart at 4+ steps with branching compensations. The business logic for "what happens when inventory fails after payment succeeded" gets smeared across 3 services that all need to know each other's failure modes. Nobody owns the flow. No single place to see "what is the state of order #4471?" At 3am you're piecing it together from events across 4 Kafka topics. Choreography is right for loose, fire-and-forget side effects — not sequential checkouts with specific compensations.

Answer 3

Why C is wrong (2PC):

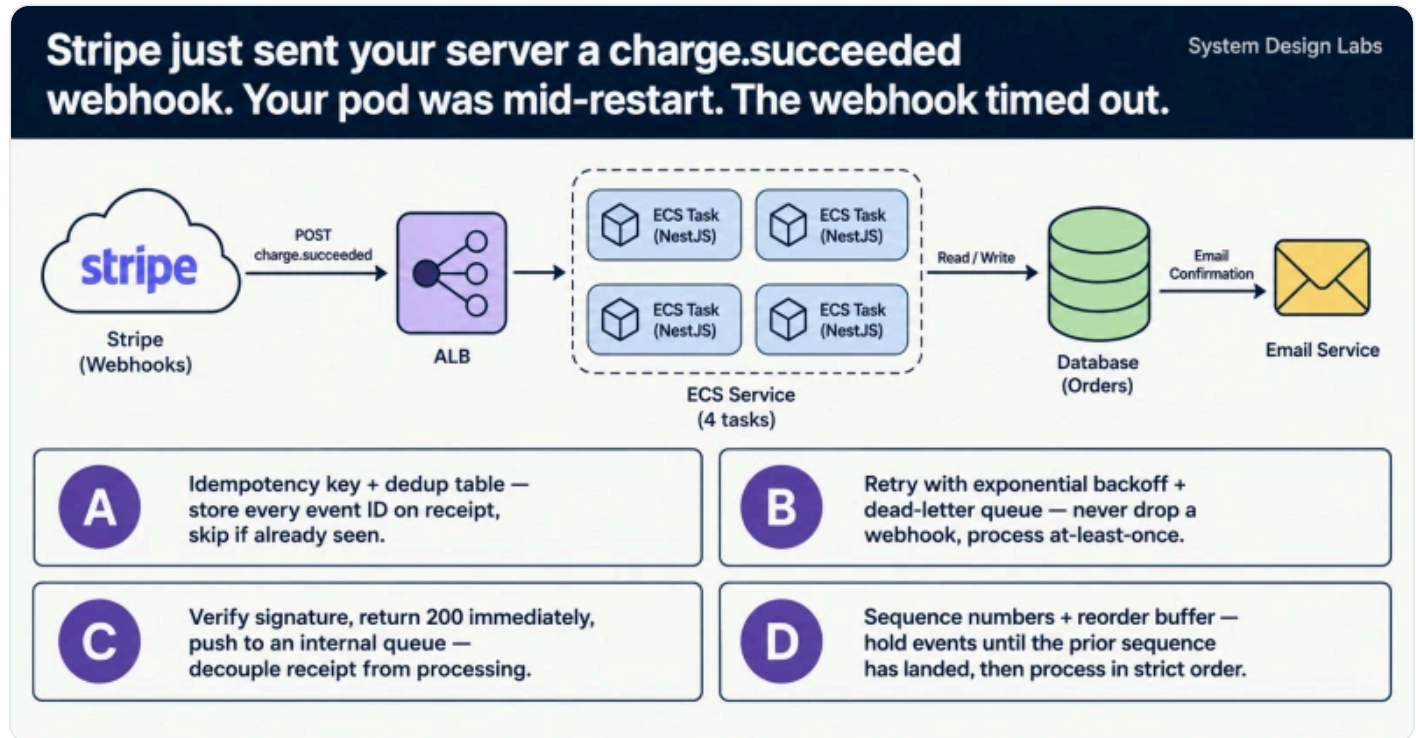
Stripe doesn't speak 2PC. Neither does any shipping carrier API. Beyond that, 2PC holds locks during PREPARE — one slow service stalls everything. At checkout volumes, the lock contention alone takes you down. Textbook answer that dies on contact with reality.

Answer 4

Why D is wrong (Outbox):

Outbox is infrastructure, not a saga. It solves how to reliably publish events after a local DB commit. It doesn't define step ordering, doesn't define compensations, doesn't answer what happens when Inventory fails. You'd still need a saga on top of it.

Handling Webhook Retries Idempotently



Stripe just sent your server a charge.succeeded webhook.

Your pod was mid-restart. The webhook timed out. Stripe will retry — but now you're racing.

Here's the setup:

NestJS API on ECS, 4 tasks behind an ALB. Stripe POSTs charge.succeeded events to /webhooks/stripe. You have ~10s to return a 200 before Stripe times out and queues a retry. On Black Friday you're handling ~80 webhooks/sec.

Three failure modes are already happening in prod:

- Pod restarts mid-deploy → webhook missed → Stripe retries → you process it, but a second pod also processed the original → **double charge confirmation email sent**
- DB is slow at peak → handler takes 12s → Stripe times out → retries → now you're processing the same event twice while the first one is still running
- Stripe sends charge.succeeded then charge.refunded 200ms apart → your queue delivers them out of order → you mark the order paid AFTER it was already refunded

The handler is a 30-line function that does the right thing 99% of the time. The 1% is killing you.

What do you do?

- A** Idempotency key + dedup table — store every event ID on receipt, skip if already seen.
- B** Retry with exponential backoff + dead-letter queue — never drop a webhook, process at-least-once.
- C** Verify signature, return 200 immediately, push to an internal queue — decouple receipt from processing.
- D** Sequence numbers + reorder buffer — hold events until the prior sequence has landed, then process in strict order.

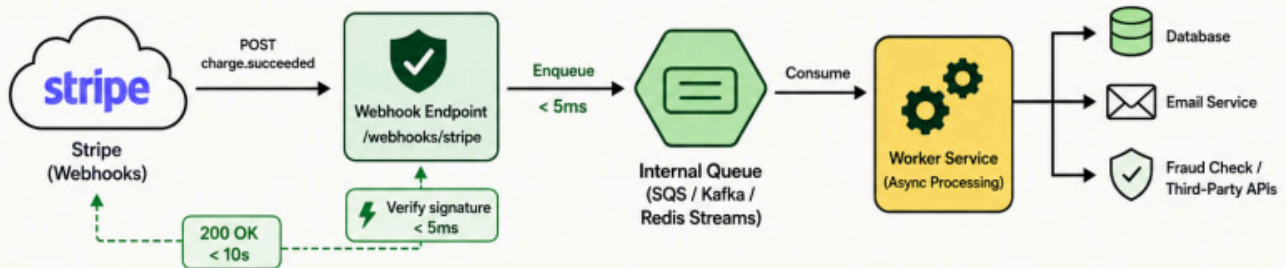
Answer 1

✓ CORRECT ANSWER

**Answer: C**

Verify signature, return 200 immediately, push to an internal queue.

System Design Labs

**C**

Verify signature, return 200 immediately, push to an internal queue.

- ✓ ACK to Stripe in < 10s (usually < 10ms).
- ✓ Decouples receipt from processing.
- ✓ Survives pod restarts and DB slowness.
- ✓ Events are durable; workers handle retries and scale independently.

A

Idempotency key + dedup table — store every event ID on receipt, skip if already seen.

- Great for preventing duplicates, but does not solve slow processing or timeouts.
- Still keeps heavy work in the webhook request path.

B

Retry with exponential backoff + dead-letter queue — never drop a webhook, process at-least-once.

- Useful, but it's still processing inside the webhook path.
- Doesn't address 10s timeout, pod restarts, or ordering problems.

D

Sequence numbers + reorder buffer — hold events until the prior sequence has landed, then process in strict order.

- Helps ordering, but increases latency and complexity.
- Does not solve timeouts, retries, or duplicate execution.

Answer: C — Verify signature, return 200 immediately, push to an internal queue ✓

Here's why, and why the other three trick smart engineers:

Why C wins (Sync receipt + async processing):

The fundamental problem is that Stripe's clock and your handler's clock are decoupled. Stripe gives you ~10 seconds to ACK. Your DB write, your fraud check, your email send — none of that should live inside that 10-second window. Ever.

The pattern: validate the HMAC signature in <5ms, write the raw payload to SQS / Kafka / Redis Streams, return 200. Now your handler is bounded by network + signature verification, not by whatever your downstream services are doing today. Pod restart mid-processing? The message is already durable in the queue — a worker picks it up. DB slow? The webhook endpoint doesn't care — it returned 200 in 8ms. This is what Stripe, Shopify, and GitHub literally tell you to do in their own webhook docs.

Answer 2

Why A is the trap answer (Idempotency key + dedup table):

This is the answer that fools senior engineers in interviews — and it's not wrong, it's just incomplete.

Yes, you absolutely need idempotency. Stripe will deliver the same event ID more than once. A dedup table keyed on [event.id](#) with a unique constraint is table stakes. But it doesn't solve the original pain: the handler is still synchronous, still on the critical 10-second path, still getting killed by pod restarts mid-write.

If you ship A alone, you've prevented double-processing — but you're still timing out, still triggering retries, still creating the conditions for the race. Idempotency is a *property your handler must have*, not an *architecture*. C is the architecture; A lives inside it. The senior engineer trap is treating idempotency as the whole answer when it's one of three layers.

Answer 3

Why B is wrong as the answer (Retry + DLQ):

Stripe is *already doing* the retry with exponential backoff. That's their job, not yours. Stripe retries failed webhooks for up to 3 days with backoff. You don't need to build that for inbound webhooks — you'd be duplicating provider behavior.

Retry + DLQ is the right pattern for *your internal queue* (the one C creates), not for the webhook endpoint itself. If your worker fails to process a queued event, sure, retry it and DLQ after N attempts. But that's a downstream concern. Putting "retry logic" at the webhook receiver level confuses inbound and outbound retry semantics.

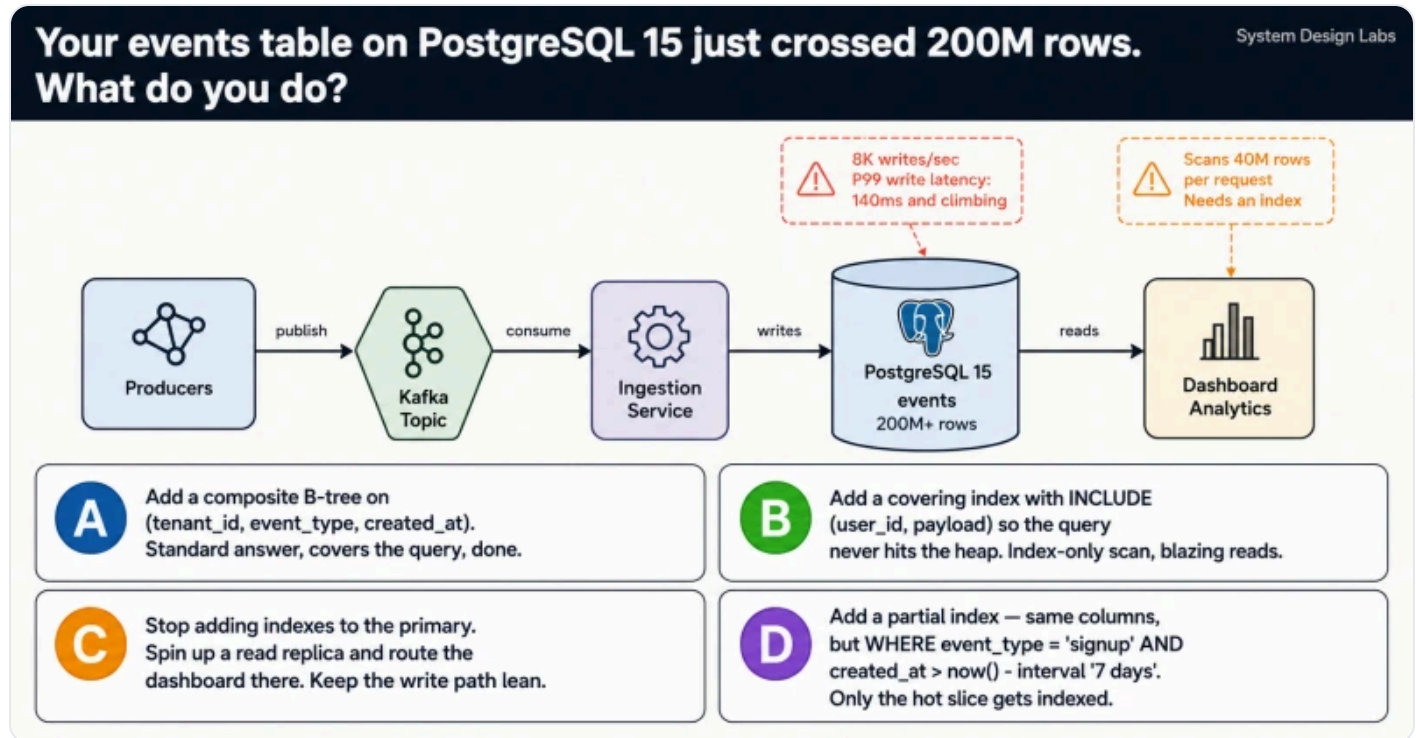
Answer 4

Why D is wrong (Sequence numbers + reorder buffer):

This sounds rigorous and it's the answer engineers reach for after they've been burned by ordering once. The problem: most webhook providers don't give you sequence numbers. Stripe doesn't. GitHub doesn't. Even when they do (Shopify's X-Shopify-Webhook-Id), the gaps aren't dense — you can't tell "missing event 47" from "47 was filtered."

More importantly: ordering at the receiver is the wrong layer. You handle ordering at the *domain level* — when you process `charge.refunded`, you check the current state of the order before transitioning. If the order is already in refunded, you no-op. State machines beat reorder buffers because they handle the case where events arrive out of order *AND* the case where one of them never arrives at all. A reorder buffer just means you stall forever when an event gets dropped upstream.

Indexing a High-Ingest Table



Your events table on PostgreSQL 15 just crossed 200M rows.

Ingestion runs at 8K writes/sec from a Kafka consumer. P99 write latency was 12ms — now it's 140ms and climbing.

The dashboard team is screaming. Their query filters on (tenant_id, event_type, created_at) and scans 40M rows per request. They want an index. Yesterday.

You check pg_stat_user_indexes. The table already has 4 indexes. Each new write touches every one of them. Add a fifth and your ingestion lag becomes ingestion failure.

But 92% of dashboard queries hit a single tenant's last 7 days of signup events. A tiny slice of a huge table.

What do you do?

- A) Add a composite B-tree on (tenant_id, event_type, created_at). Standard answer, covers the query, done.
- B) Add a covering index with INCLUDE (user_id, payload) so the query never hits the heap. Index-only scan.
- C) Stop adding indexes to the primary. Spin up a read replica and route the dashboard there. Keep writes lean.

D) Add a partial index — same columns, but WHERE event_type = 'signup' AND created_at > now() - interval '7 days'. Only the hot slice gets indexed.

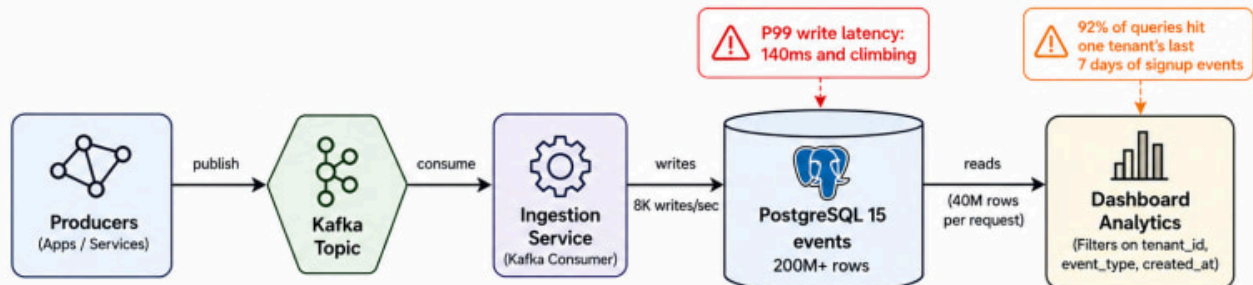
Answer 1

✓ CORRECT ANSWER

D**Correct Answer**

**Add a partial index — same columns,
WHERE event_type = 'signup' AND created_at > now() - interval '7 days'.**

System Design Labs

**D) Partial Index (Correct)**

Best Balance



Only indexes rows where event_type = 'signup' AND created_at > now() - interval '7 days'.

- ✓ Indexes only the hot slice (~0.2% of table)
- ✓ Inserts outside the predicate pay zero cost
- ✓ Planner uses it for the dashboard query
- ✓ Fast reads, minimal write amplification



Trade-off: if the dashboard needs 30 days instead of 7, you rebuild the index. Known cost, not a hidden one.

**A) Composite B-tree Index**

- ✗ Indexes all rows (200M+)
- ✗ Every write updates it
- ✗ High write amplification
- ✗ Will worsen write latency

**B) Covering Index INCLUDE (user_id, payload)**

- ✗ Even more writes per insert
- ✗ Much larger index
- ✗ Higher memory & I/O
- ✗ Write path takes the hit

**C) Read Replica**

- ✗ Does not fix slow writes on primary
- ✗ Still need the index somewhere for fast reads
- ✗ Replication lag won't help real-time dashboard
- ✗ Adds infrastructure and ops complexity

Answer: D — Partial Index ✓

Here's why, and why the other three trick smart engineers.

Why D wins (Partial Index)

92% of your queries hit one tenant's last 7 days of signup events. A partial index only indexes rows that match its WHERE clause — everything else writes to the table without touching this index at all.

The math: if signup events are ~4% of the stream and the 7-day window covers ~5% of the table, your partial index covers roughly 0.2% of rows. Inserts that don't match the predicate cost you nothing on this index. The write amplification is almost entirely gone.

PostgreSQL's planner uses the partial index for any query whose WHERE clause is a subset of the index predicate. Your dashboard query gets full coverage. Reads stay fast, writes stay alive.

The catch: when the dashboard team wants 30 days instead of 7, you rebuild. That's a known cost, not a hidden one.

Answer 2

Why B is the trap (Covering Index with INCLUDE)

This fools senior engineers more than any other option. An index-only scan looks gorgeous in EXPLAIN. The problem is what you stuffed into INCLUDE.

That payload column is probably a JSONB blob averaging a few KB. Every insert now writes the row to the heap *and* writes a fat index entry containing the full payload. Every UPDATE to `user_id` or payload rewrites the index entry. You didn't reduce write cost — you multiplied it.

At 8K writes/sec on a table that's already index-heavy, a covering index with JSONB inside is how you turn 140ms p99 into 800ms p99. INCLUDE is the right call for narrow columns on read-heavy tables. This is neither.

Answer 3

Why A is wrong (Full composite B-tree)

A (`tenant_id`, `event_type`, `created_at`) index will make the dashboard fast. It will also index every single one of your 8K writes/sec across all tenants and all event types. You're paying full write cost to serve a query that hits 0.2% of the data. That's the definition of a bad tradeoff.

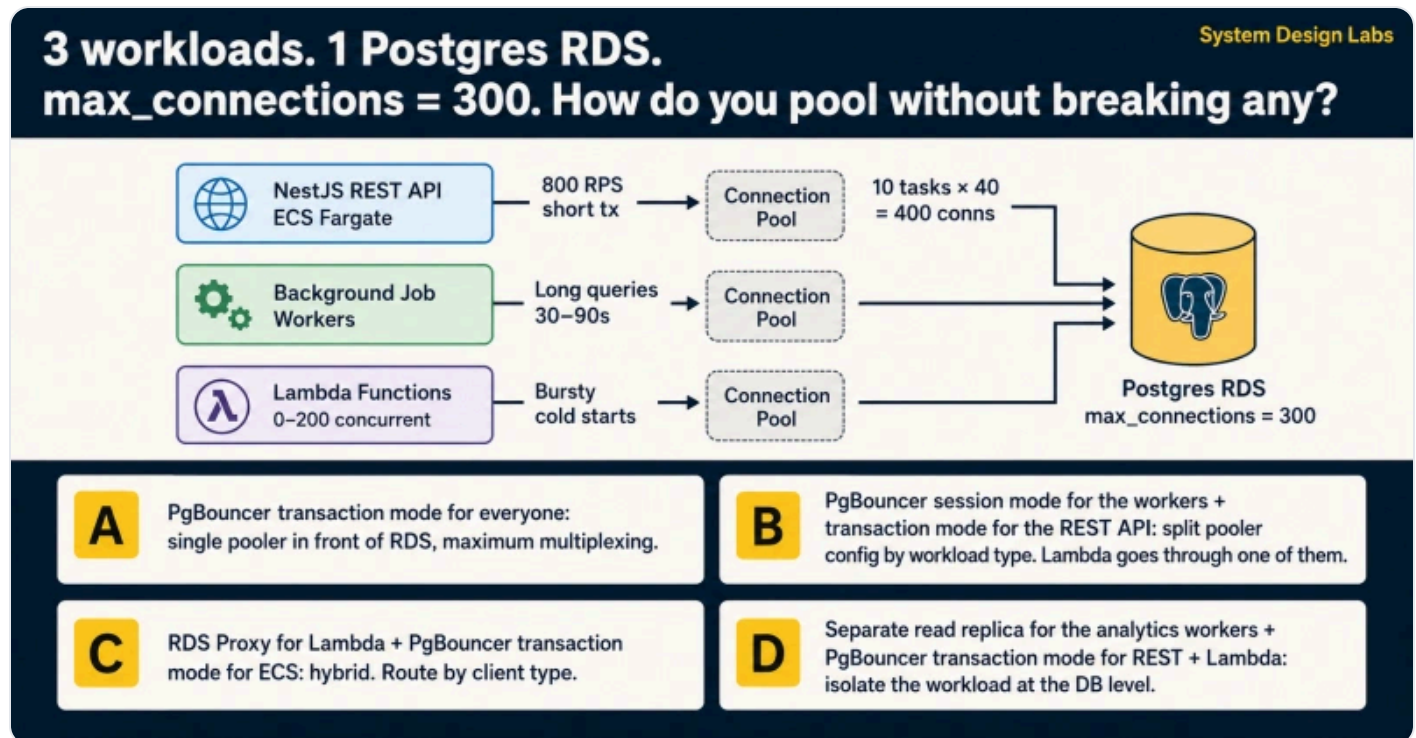
Answer 4

Why C is wrong (Read replica)

The indexes still have to exist somewhere. Put them on the replica and it falls behind replaying WAL while maintaining indexes. Put them on the primary and you're back to the original problem.

Read replicas solve "my primary can't serve all the reads." They don't solve "my write path is bottlenecked by index maintenance."

Managing a Shared Connection Pool



You're running 3 workloads against the same Postgres RDS instance. `max_connections = 300`. They all need a connection pool. None of them want to share.

Here's the setup:

→ NestJS REST API on ECS Fargate. Short transactions, 800 RPS, 10 tasks × 40 pool = **400 connections requested**.

→ Background job workers. Long-running analytics queries. Hold a connection for **30–90 seconds** each.

→ Lambda functions. Bursty, cold-start heavy, **0–200 concurrent invocations**.

Three clients. One database. 300 max connections. Math doesn't math.

You need to pool without breaking any of them. Pick your poison:

A **PgBouncer transaction mode for everyone:** single pooler in front of RDS, maximum multiplexing.

B **PgBouncer session mode for the workers + transaction mode for the REST API:** split pooler config by workload type. Lambda goes through one of them.

C RDS Proxy for Lambda + PgBouncer transaction mode for ECS: hybrid. Route by client type.

D Separate read replica for the analytics workers + PgBouncer transaction mode for REST + Lambda: isolate the workload at the DB level.

Answer 1 ✓ CORRECT ANSWER

B**Answer: B — PgBouncer session mode for workers + transaction mode for REST API ✓**

System Design Labs

WHY B WINS**API gets multiplexing**

Short transactions are perfect for transaction mode.

**Workers keep their session**

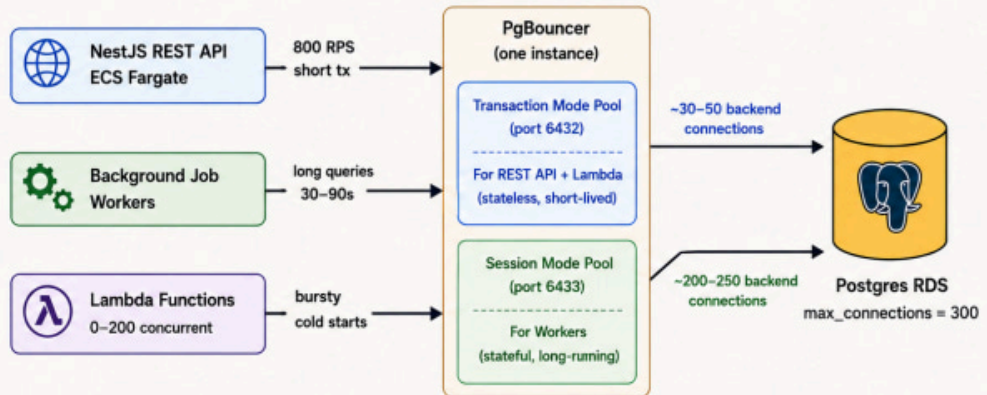
Session mode preserves state across long-running queries.

**No one starves**

Each workload gets what it needs without stepping on each other.

**Stays under 300 connections**

Backend connections to Postgres remain within the limit.

**HOW THIS SOLVES IT**

PgBouncer multiplexes the API and Lambda connections in transaction mode. 400 client connections collapse to ~30-50 backend connections.



Workers use session mode. One client connection equals one backend connection for the entire session. State is preserved, queries don't break.



Lambda traffic is stateless and short-lived, same shape as the API. It uses the transaction-mode pool.



Total backend connections stay under 300. API scales, workers run their long queries, and nothing breaks at 3am.

Answer: B — PgBouncer session mode for workers + transaction mode for REST API ✓

This one separates engineers who've actually run PgBouncer in prod from ones who've only read the docs. Here's the breakdown:

Why B wins (split pooler config by workload type):

The whole point of PgBouncer is multiplexing — one backend Postgres connection serves many client connections, but **only when it's safe to hand the connection back between transactions**. That's transaction mode.

Transaction mode is *perfect* for the NestJS REST API: short transactions, 800 RPS, no session state, no SET LOCAL, no advisory locks held across queries, no prepared statements that need to survive. The REST API's 400 requested client connections collapse down to ~30-50 actual backend connections to Postgres. Math suddenly maths.

Transaction mode is *catastrophic* for the analytics workers. Their queries run 30-90 seconds and span multiple transactions — some hold session state, some use temp tables, some use SET search_path, some use cursors. Hand the connection back between transactions and that state is gone. Worse: at 800 RPS from the API side, the worker's chance of getting the *same* backend connection back on the next transaction is basically zero. The worker silently breaks.

So you give the workers their own PgBouncer pool in **session mode** — one client connection = one backend connection for the whole session. They get fewer connections, but they hold them, which is what they actually need.

The Lambda traffic goes through the REST API's transaction-mode pool (it's stateless, short-lived — same shape as the API). One pooler instance, two pool configs, two listening ports. PgBouncer supports this natively.

End state: API gets multiplexing, workers don't starve, total backend connections to Postgres stay under 300, nobody breaks. That's the boring correct answer that wins on call.

Answer 2

...Why A is the trap (PgBouncer transaction mode for everyone):

This is the answer most engineers reach for because "transaction mode = best multiplexing." It works beautifully for the API. It silently destroys the workers.

The moment an analytics query needs anything that doesn't fit in one transaction — a temp table, a session variable, a SET LOCAL, a prepared statement, a cursor that outlives a transaction — PgBouncer hands the connection back to the pool. Next query? Different backend connection. State gone. Query fails or worse, returns wrong results.

The failure mode is the cruelest kind: it works in dev (low concurrency, same connection by luck), it works in staging (light load), it dies in prod when the API is hammering the pool and the workers can't get a stable connection. You'll find this at 3am during a financial close.

Answer 3

Why C is wrong (RDS Proxy for Lambda + PgBouncer for ECS):

Defensible on paper. RDS Proxy is genuinely good for Lambda — it handles the cold-start connection storm, manages IAM auth, integrates with the VPC. If the question were just "Lambda + Postgres," C is right.

But C doesn't fix the workers. They're still long-running, they still need session state, and you've now introduced a second pooling product to operate, monitor, and pay for. RDS Proxy is also *more expensive than PgBouncer* per connection-hour, and it doesn't support session pinning across transactions the way session-mode PgBouncer does.

C solves the easiest of the three problems and leaves the hardest one untouched.

Answer 4

Why D is the senior-engineer trap (read replica for workers):

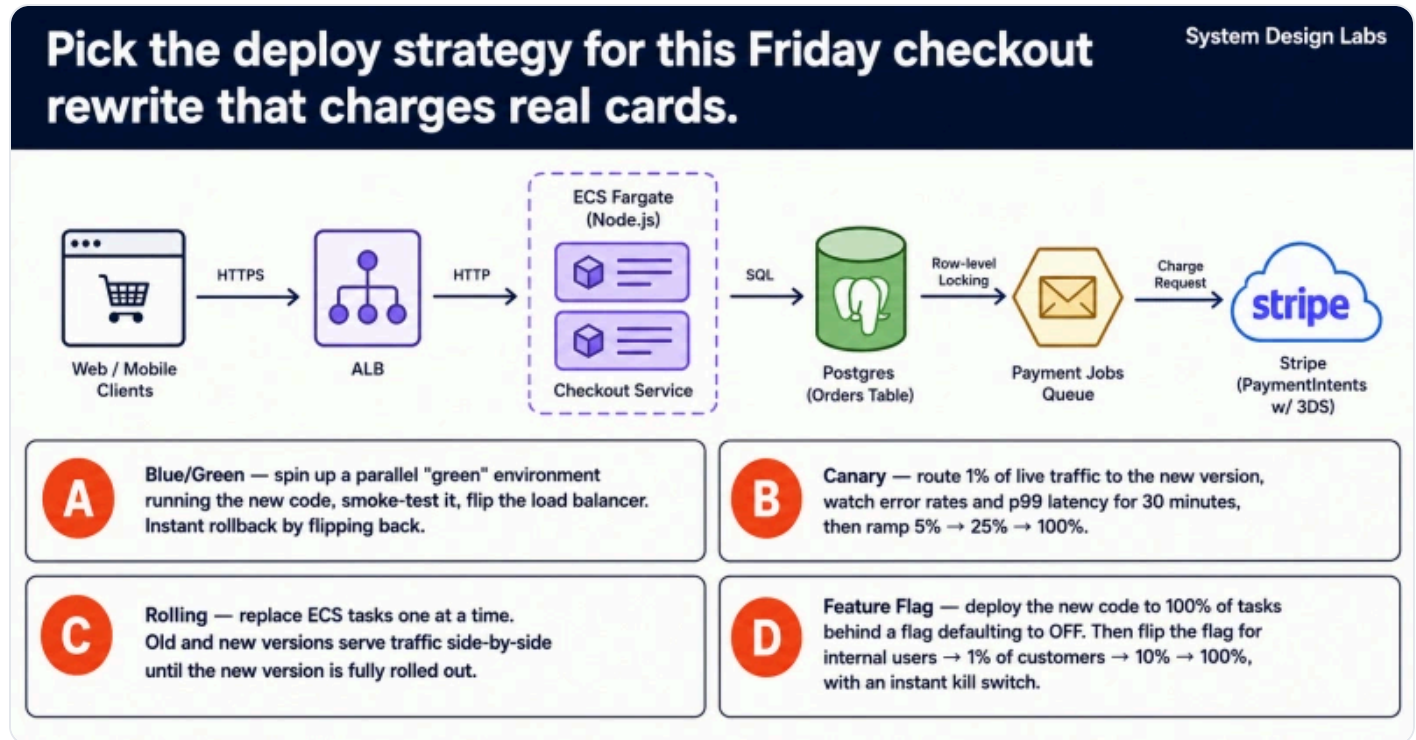
This is the option that fools experienced engineers, because it pattern-matches to "isolate the workload." And isolating analytics from OLTP at the DB level is genuinely a great idea — for a *different* problem.

D solves **read scaling** and **noisy neighbor on query plans / cache / IO**. It does **not** solve the connection math. Your read replica still has its own max_connections ceiling. Your workers still hold connections for 30–90 seconds. You've just moved the connection exhaustion problem from the primary to the replica — and now you're paying for an extra RDS instance.

If the problem statement was "analytics queries are starving the OLTP workload," D would be right. The problem statement is "we don't have enough connections." Different pain. Different pattern.

D is what you do *after* you have B working and you realize analytics is also competing for IOPS and buffer cache.

Safely Rolling Out a Risky Change



You're shipping a rewrite of your checkout write path on Friday.

Node.js on ECS Fargate. ~3,000 RPS at peak. Postgres with row-level locking on the orders table. The change touches the line of code that actually charges the customer's card — old path uses Stripe Charges API, new path uses PaymentIntents with 3DS.

The QA env is green. Load test passed. Your staff engineer signs off.

But it's *checkout*. If this breaks, money breaks. And you can't roll back a card that already got charged.

Pick the deploy strategy:

- A** Blue/Green — spin up a parallel "green" environment running the new code, smoke-test it, flip the load balancer. Instant rollback by flipping back.
- B** Canary — route 1% of live traffic to the new version, watch error rates and p99 latency for 30 minutes, then ramp 5% → 25% → 100%.

C Rolling — replace ECS tasks one at a time. Old and new versions serve traffic side-by-side until the new version is fully rolled out.

D Feature Flag — deploy the new code to 100% of tasks behind a flag defaulting to OFF. Then flip the flag for internal users → 1% of customers → 10% → 100%, with an instant kill switch.

Answer 1

✓ CORRECT ANSWER

ANSWER: D — FEATURE FLAG ✓

System Design Labs

**Why D wins (Feature Flag):**

Feature flags decouple deploy from release. The new code ships to 100% of your ECS tasks — but it's dormant. Nothing executes until you flip the flag.



Rollback is a config push, not a deploy.

Sub-second.
No ECS restarts,
no LB drain,
no DNS TTL.



You target who sees the new path, not what % of random traffic.

\$0.99 customer and \$40K B2B customer shouldn't get coin-flipped equally — that's not a strategy, that's hope.



You can keep the old code path warm for weeks.

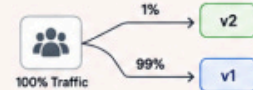
3DS edge case on a specific issuing bank in Brazil? Flip those users back while keeping everyone else on v2.



Why the other options look right — but quietly fail when the change touches money

A Blue/Green

Big bang. You flip everyone at once. If the new payment flow has a flaw that only appears under real money conditions, you learn about it at 3AM with 100% impact.

B Canary

Random % is the wrong targeting. High-value customers, specific banks, or regions might be in the 1% — and the blast radius is still real money.

C Rolling

Old and new code run side-by-side. You can't control which customers hit which version. Shared DB + money writes = inconsistent behavior and hard-to-debug issues.

D Feature Flag

- ✓ Deploy once. Release safely.
- ✓ Target precisely. Roll back instantly.
- ✓ Keep old path ready as a safety net.

Answer: D — Feature Flag ✓

Here's why, and why the other three look right but quietly fail when the change touches money:

Why D wins (Feature Flag):

Feature flags decouple **deploy** from **release**. The new code ships to 100% of your ECS tasks — but it's dormant. Nothing executes until you flip the flag.

- **Rollback is a config push, not a deploy.** Sub-second. No ECS restarts, no LB drain, no DNS TTL.
- *You target who sees the new path, not what % of random traffic.* \$0.99 customer and \$40K B2B customer shouldn't get coin-flipped equally — that's not a strategy, that's hope.
- **You can keep the old code path warm for weeks.** 3DS edge case on a specific issuing bank in Brazil? Flip *those* users back while keeping everyone else on v2.

Answer 2

Why B is the trap (Canary):

1. Canary routes by request, not by customer. A B2B billing job hits your 1% canary 50 times in an hour. Bug on their card type? You've charged them 50 times before dashboards catch it.
2. Payment failures don't surface as 5xx errors — they surface as successful 200s that charged the wrong amount. Your error-rate alarm never fires.

Answer 3

Why A is wrong (Blue/Green):

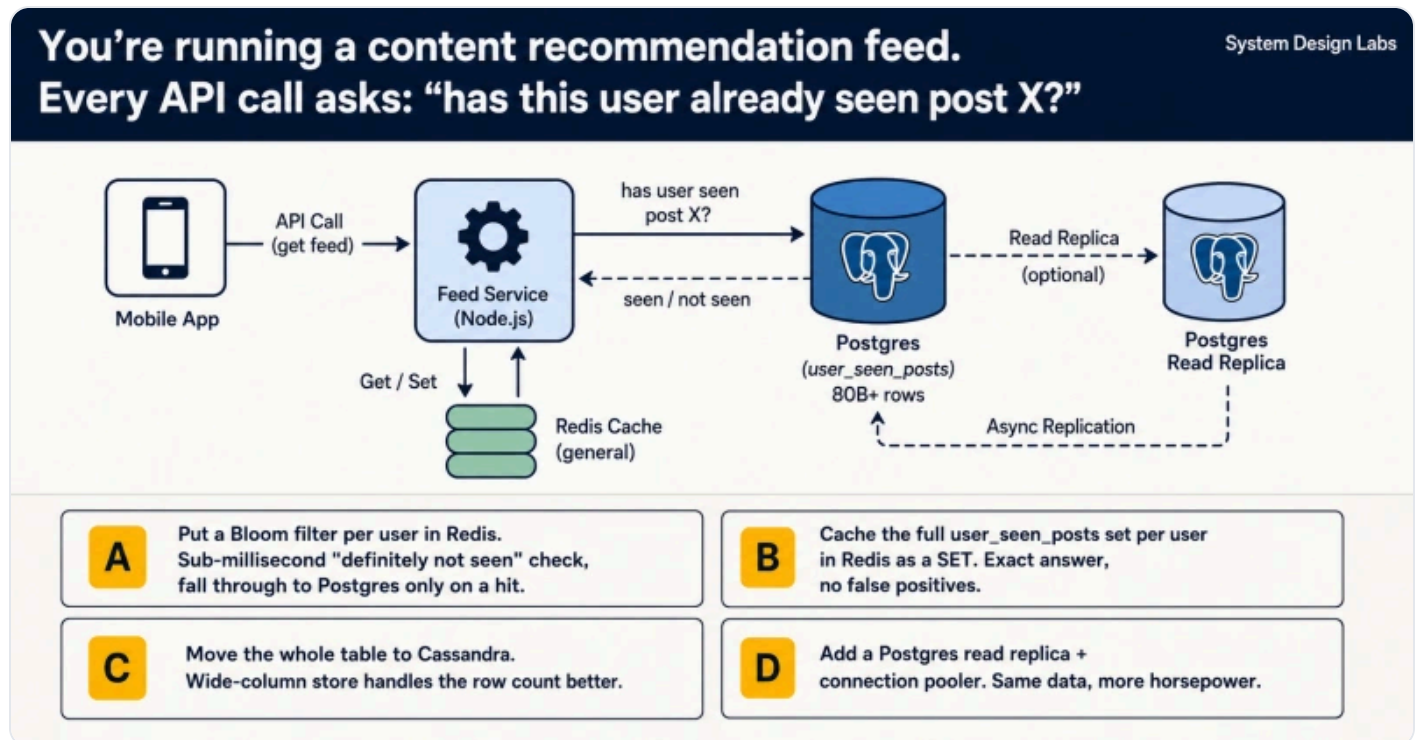
"Instant cutover" is the problem. The moment you flip the LB, 100% of live traffic hits new code — including Stripe webhooks confirming PaymentIntents *created* by the old code. Cross-version race condition on real money. Rollback doesn't undo a charge.

Answer 4

Why C is wrong (Rolling):

For 5–10 minutes, old and new tasks serve checkout simultaneously. "Create order" hits v1. "Confirm payment" webhook 800ms later hits v2. Inconsistent state written by code that was never tested together.

Fast “Has the User Seen This?” Checks



You're running a content recommendation feed. 50M users.

Every API call asks: "has this user already seen post X?"

Right now you're hitting Postgres on every check. The user_seen_posts table has 80B rows and growing. p99 latency on the feed endpoint just crossed 600ms and your DBA is sending you screenshots of CPU graphs at 2am.

Here's the setup:

- Feed service: Node.js, ~120K RPS at peak
- "Already seen" check: SELECT against Postgres on every recommendation
- Table: user_seen_posts(user_id, post_id, seen_at) (partitioned, indexed, still slow)
- Cache hit ratio is fine. The problem is the long tail of cold lookups
- False positives on "already seen" are tolerable. False negatives (showing the same post twice) are not great but survivable

Product wants p99 under 100ms. You have one sprint. What do you do?

A Put a Bloom filter per user in Redis. Sub-millisecond "definitely not seen" check, fall through to Postgres only on a hit.

B Cache the full user_seen_posts set per user in Redis as a SET. Exact answer, no false positives.

C Move the whole table to Cassandra. Wide-column store handles the row count better.

D Add a Postgres read replica + connection pooler. Same data, more horsepower.

Answer 1

✓ CORRECT ANSWER

■ Author's diagram unavailable (source image returned 404)

A — Bloom Filter per user in Redis (THE RIGHT ANSWER)

Bloom filters give one ironclad guarantee: "*this item is DEFINITELY NOT in the set.*" If the filter says no → skip Postgres entirely. If it says maybe → fall through to confirm.

Why it wins here:

- ~97% of feed checks are for posts the user has *never* seen → sub-millisecond "definitely no" from Redis, Postgres never touched
- **Memory math:** 10K posts per user in a Bloom filter at 1% false-positive = **12 KB**. Same data as a Redis SET = **80 KB+**, and grows linearly
- At 50M users: Bloom filter = ~600 GB. Redis SET = ~4 TB. That's the difference
- Real-world users: Medium's "have you read this", Cassandra's SSTable short-circuit, Bitcoin's SPV wallets

The 1% false positive here means you *occasionally* skip showing a post the user hasn't seen. Product said that's tolerable. Bloom filters never produce false negatives (showing the same post twice) — which would be the dealbreaker.

Answer 2**B — Redis SET per user (SENIOR ENGINEER TRAP)**

Looks strictly better on paper — same $O(1)$ speed, *plus* perfect accuracy, no false positives. SISMEMBER is battle-tested.

The trap: **memory explodes on power users**. A user with 50K posts seen = 3–4 MB in a Redis SET. Your top 5% users blow up the Redis cluster. You'll either over-provision (expensive) or start evicting hot users (defeats the whole point).

Rule of thumb: Redis SET when cardinality is small (<1K per key) or false positives are unacceptable. Bloom filter when cardinality is large and "probably not" is good enough.

Answer 3

C — Move to Cassandra (RIGHT PATTERN, WRONG PROBLEM)

Cassandra handles wide-column data and massive row counts fine. But the problem isn't *storage* — Postgres with proper partitioning handles 80B rows. The problem is **read latency under load**.

Switching storage engines doesn't fix the read pattern. You're still doing a point lookup per recommendation. Now you've added a multi-quarter migration + new ops skillset to solve what's fixable in a sprint with 12 KB per user in Redis.

Cassandra is the answer when the bottleneck is write throughput or wide-row scans. Not for 120K point lookups per second on a hot path.

Answer 4

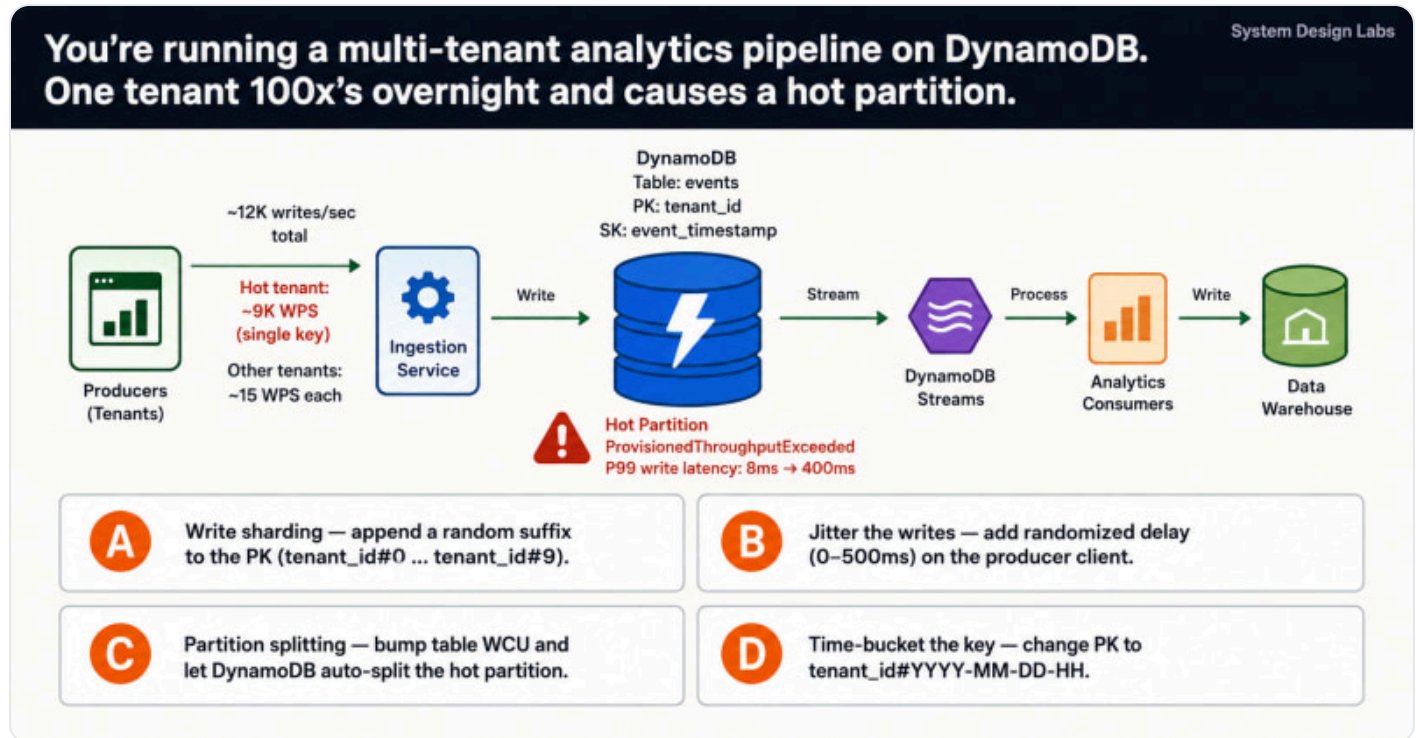
D — Postgres read replica + connection pooler (CAPACITY ≠ COST)

Replicas buy horsepower, not efficiency. You're still hitting disk on every check, still doing a B-tree lookup, still paying round-trip per request. 3 replicas → maybe 2.5x read throughput before replication lag and connection limits bite.

To hit p99 <100ms at 120K RPS, you don't need *more database* — you need to **not hit the database** for the 97% of requests where the answer is "no."

This is the option chosen by teams who treat every performance problem as a capacity problem.

Taming a Hot Partition Key



You're running a multi-tenant analytics pipeline on DynamoDB. 200 tenants, 12K writes/sec total. Everything is fine — until it isn't.

One tenant onboards a massive customer overnight. Their event volume 100x's. Now that one tenant is hitting 9K writes/sec on a single partition key. The other 199 tenants sit idle.

ProvisionedThroughputExceeded errors start firing. P99 write latency spikes from 8ms to 400ms. Your on-call gets paged. Everyone is getting throttled because of one key.

Here's the setup:

- Table: events (DynamoDB, on-demand capacity)
- PK: tenant_id · SK: event_timestamp
- Hot tenant: ~9K WPS · All other tenants: ~15 WPS each
- Single partition key absorbing all 9K writes — DynamoDB's per-partition limit is the wall

Classic hot partition. What do you do?

- A** Write sharding — append a random suffix to the PK (tenant_id#0 ... tenant_id#9).

B Jitter the writes — add randomized delay (0–500ms) on the producer client.

C Partition splitting — bump table WCU and let DynamoDB auto-split the hot partition.

D Time-bucket the key — change PK to tenant_id#YYYY-MM-DD-HH.

Answer 1

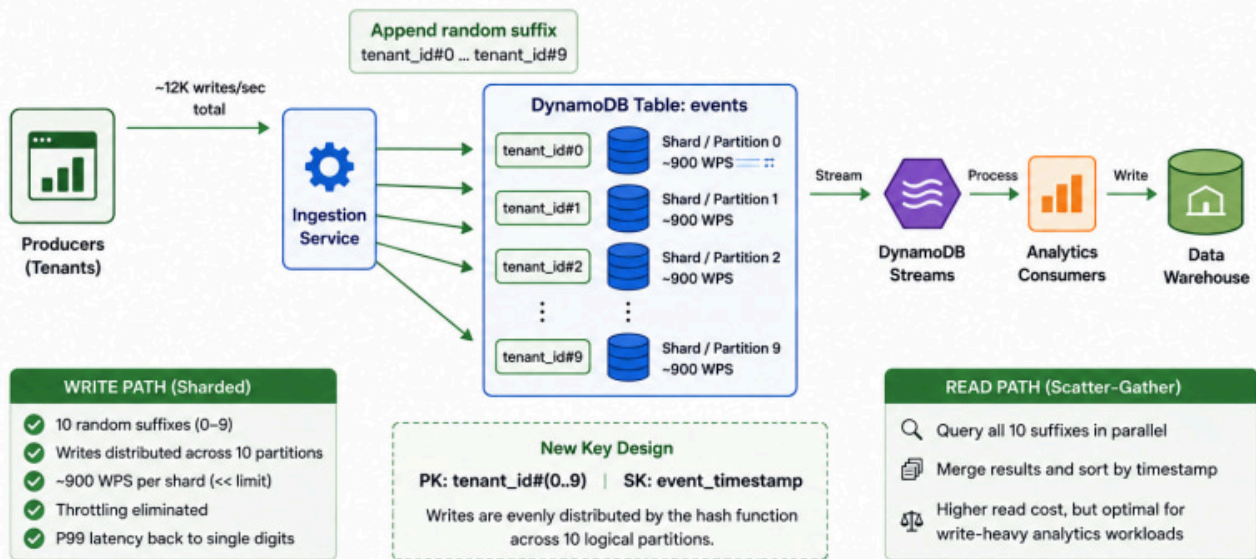
✓ CORRECT ANSWER

ANSWER: A — WRITE SHARDING (THE RIGHT ANSWER)

System Design Labs

**Append a random suffix to the PK to spread writes across multiple partitions.**

10 shards ≈ 900 WPS each | Throttling stops | P99 latency back to single digits



This is the canonical fix for hot partitions — and the pattern AWS documents.

A — Write Sharding (THE RIGHT ANSWER)

DynamoDB hashes the partition key to decide which physical shard handles the write. One PK value = one shard = one wall. When that tenant hits 9K WPS, the shard caps out.

The fix: append a random suffix at write time — tenant_id#0 through tenant_id#9. Now the hash function sees 10 different keys and distributes writes across 10 logical partitions (~900 WPS each, well under the limit). Throttling stops, P99 drops back to single digits.

The read cost: scatter-gather across all 10 suffixes to query that tenant's data. For a write-heavy analytics pipeline, that's the right tradeoff.

This is literally the pattern AWS documents as the canonical fix for hot partitions.

Answer 2

C — Partition Splitting (SENIOR ENGINEER TRAP)

DynamoDB *does* auto-split partitions — so this sounds right. But the split is based on **throughput**, not **key cardinality**. If one PK value generates 9K WPS, all those writes still hash to the same destination after the split. You can't split a single key across two physical shards.

You'll see the partition split in CloudWatch, assume the problem is fixed, and then get paged again 10 minutes later. It's the answer that survives the design doc and dies in the war room.

Answer 3

B — Jitter the writes

Jitter is a real pattern — it solves the **thundering herd** problem, where N clients fire requests at the same millisecond and create a spike. Spreading those over 500ms smooths the burst.

But a hot partition is a **sustained** 9K WPS, not a burst. Adding delay doesn't reduce the rate — DynamoDB still sees 9K WPS on the same partition. You just made every write slower for zero benefit.

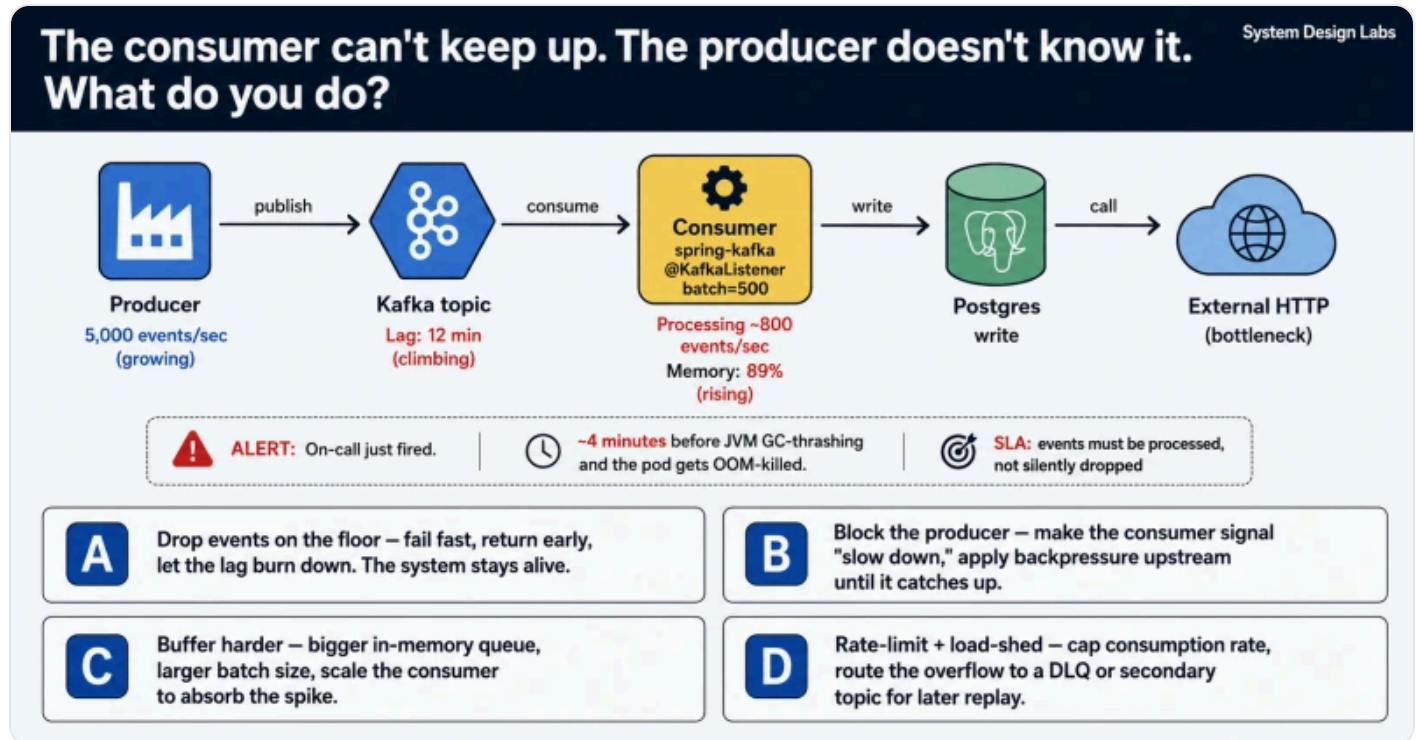
Answer 4

D — Time-bucket the key

`tenant_id#2026-05-21-14` is a real pattern for time-series data — it keeps per-bucket size manageable and enables efficient TTL expiry.

But at any given hour, you *still* have one partition key absorbing all 9K WPS for that tenant. You renamed the hot partition to a hot time bucket. Same problem, new name. Plus now your reads have to query 24 different PKs for a day's worth of data.

Backpressure on an Overwhelmed Consumer



Your Kafka consumer is processing 800 events/sec.

The producer just hit 5,000 events/sec and it's not slowing down.

Lag chart: 12 minutes behind and climbing. Consumer memory: 89% and rising. The on-call alert just fired. You have ~4 minutes before the JVM starts GC-thrashing and the pod gets OOM-killed.

Here's the setup:

Producer → Kafka topic (5K events/sec, growing)

Consumer → spring-kafka @KafkaListener, batch=500, processes ~800 events/sec

Downstream → Postgres write + external HTTP call (the real bottleneck)

SLA → events must be processed, not silently dropped

The consumer can't keep up. The producer doesn't know it. What do you do?

- A** Drop events on the floor — fail fast, return early, let the lag burn down. The system stays alive.

B Block the producer — make the consumer signal "slow down," apply backpressure upstream until it catches up.

C Buffer harder — bigger in-memory queue, larger batch size, scale the consumer to absorb the spike.

D Rate-limit + load-shed — cap consumption rate, route the overflow to a DLQ or secondary topic for later replay.

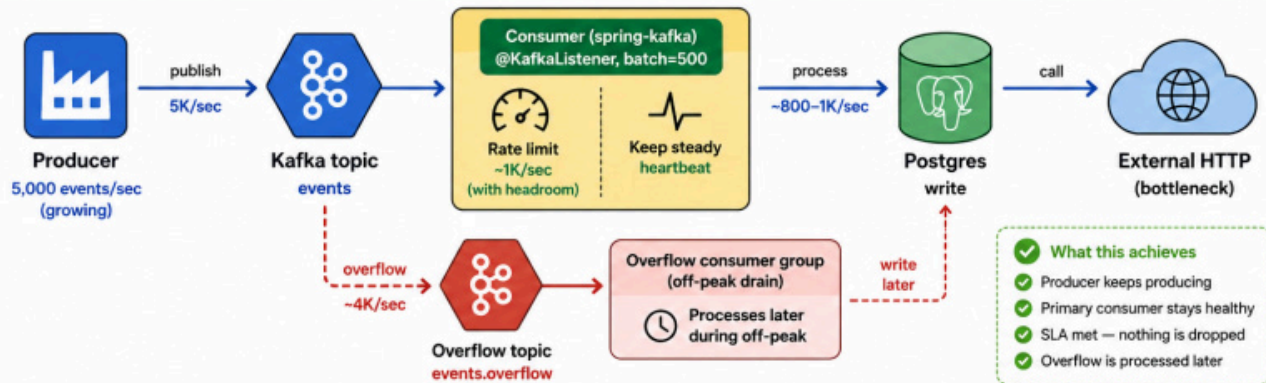
Answer 1

✓ CORRECT ANSWER

✓ ANSWER: D — Rate-limit + load-shed

System Design Labs

🎯 Goal: Keep the system healthy, meet the SLA, and process overflow later — not drop, not block.



★ Why D wins:

The SLA says events **must be processed** — not dropped, not blocked. That kills **A and B** immediately.

Cap the consumer's intake at ~1K/sec with headroom, route overflow to a durable secondary topic (**events.overflow**) with its own consumer group that drains during off-peak.

Producer keeps producing. Primary consumer keeps a steady heartbeat.

Overflow gets processed later. That's **graceful degradation**.



Stripe and Shopify publish architecture posts on this exact pattern.

Why others lose:

A Drop events on the floor
Violates the SLA. Events are lost forever.

B Block the producer
Kafka doesn't provide consumer → producer backpressure. You'd need custom acks/transactions — not this stack.

C Buffer harder
More memory = faster OOM. Larger batches increase latency and worsen recovery time.

Answer: D — Rate-limit + load-shed ✓

Why D wins:

The SLA says events must be processed — not dropped, not blocked. That kills A and B immediately. Cap the consumer's intake at ~1K/sec with headroom, route overflow to a durable secondary topic (**events.overflow**) with its own consumer group that drains during off-peak. Producer keeps producing. Primary consumer keeps a steady heartbeat. Overflow gets processed later. That's graceful degradation. Stripe and Shopify publish architecture posts on this exact pattern.

Answer 2

Why B is the trap answer:

This is what fools senior engineers. Backpressure-as-blocking works in Reactor, RxJava, Akka — because the protocol carries the signal. Kafka is pull-based and decoupled. The producer doesn't know your consumer exists. There's no socket-level backpressure signal flowing upstream. Engineers from gRPC streaming or RxJava reach for B because it worked there. In a brokered, decoupled, log-based system — it doesn't apply.

Answer 3

Why A is wrong:

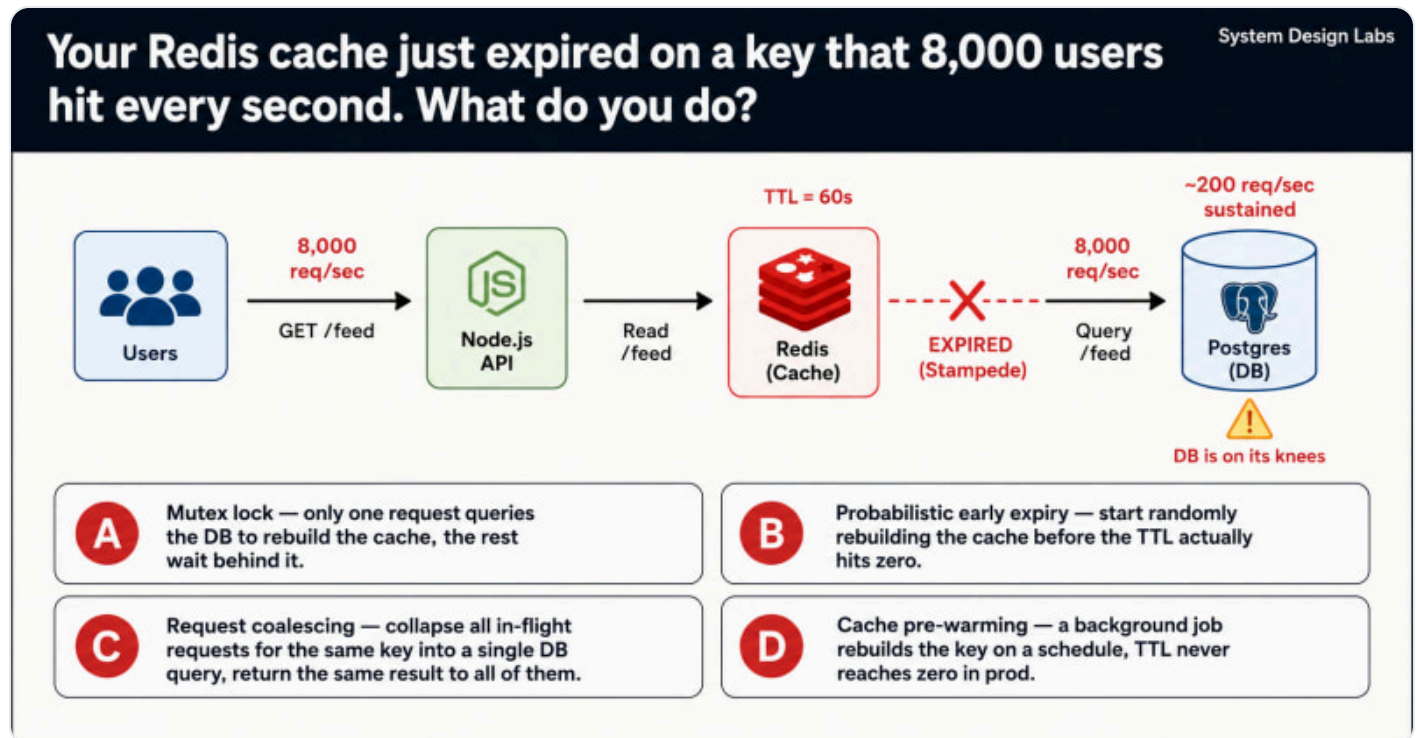
Drops events. SLA says "must be processed, not silently dropped." Done.

Answer 4

Why C is wrong:

Buffering is a delay tactic. A bigger buffer absorbs a 30-second spike — not a sustained 5K/sec input on an 800/sec consumer. The math: you're 4,200 events behind per second. A 1M-event buffer buys ~4 minutes. Then you're back where you started, but now with a memory exhaustion problem. And the real bottleneck is the external HTTP call — scaling consumers 5x doesn't help if downstream rate-limits you at request #51.

Surviving a Cache Stampede



Your Redis cache just expired on a key that 8,000 users hit every second.

Every single one of those requests is now flying straight at your database.

This is the thundering herd. You didn't have a traffic problem — you had a cache problem. Now you have both.

Here's the setup:

Service → Node.js API, 8,000 req/sec on the /feed endpoint

Cache → Redis, TTL = 60s on the feed key

DB → Postgres, comfortable at ~200 req/sec sustained

What happened → TTL expired at peak traffic, all 8,000 req/sec hit Postgres simultaneously

The DB is on its knees. You have minutes before it falls over. And the next TTL expiry is in 60 seconds.

What do you do?

A Mutex lock — only one request queries the DB to rebuild the cache, the rest wait behind it.

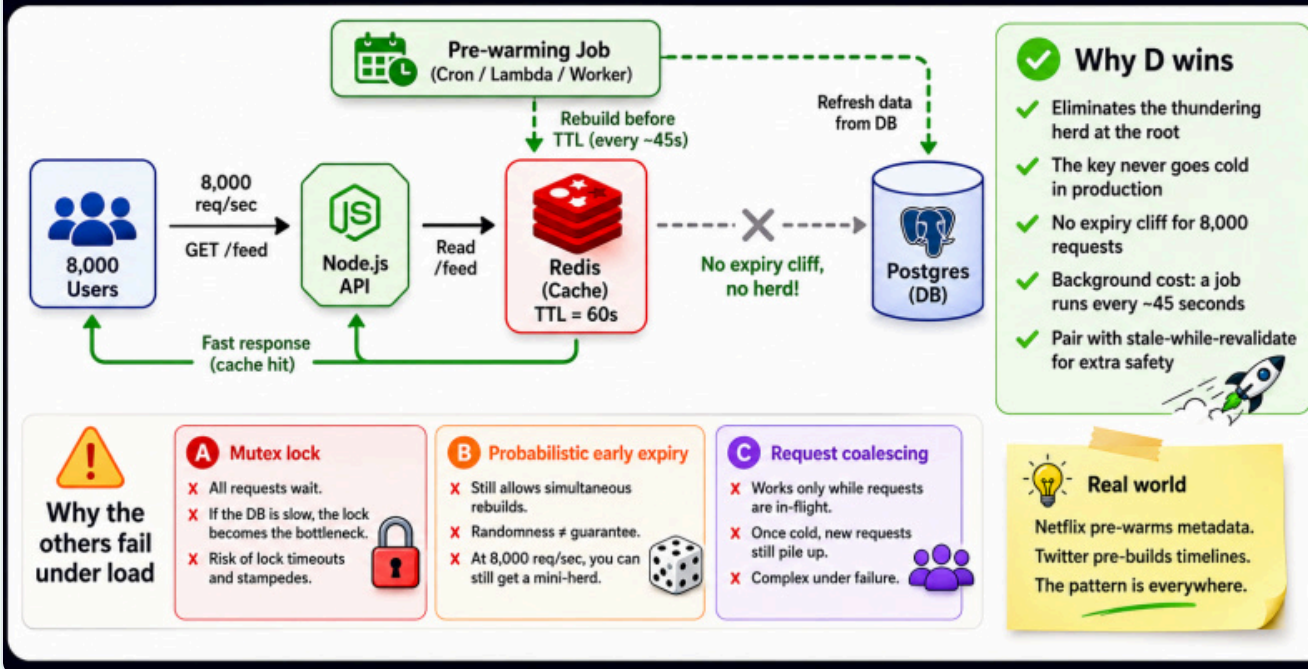
- B** Probabilistic early expiry — start randomly rebuilding the cache before the TTL actually hits zero.
- C** Request coalescing — collapse all in-flight requests for the same key into a single DB query, return the same result to all of them.
- D** Cache pre-warming — a background job rebuilds the key on a schedule, TTL never reaches zero in prod.

Answer 1

✓ CORRECT ANSWER

D Answer: D — Cache pre-warming ✓Pre-warm before TTL expires. **No cold key, no herd.**

System Design Labs

**Answer: D — Cache pre-warming** ✓

Here's why, and why the other three trick smart engineers:

Why D wins (Cache pre-warming):

Pre-warming eliminates the thundering herd at the root. A background job — a cron, a scheduled Lambda, a Sidekiq worker — rebuilds the cache key on a fixed schedule that's shorter than the TTL. The key never goes cold in production. There's no expiry cliff for 8,000 requests to fall off.

You know the data is hot. You know when it expires. You have the compute to rebuild it proactively. The cost is a background job running every ~45 seconds; the benefit is your database never sees the spike. Netflix pre-warms content metadata. Twitter pre-builds timelines for high-follower accounts. The pattern is everywhere — it's just not glamorous enough to make it into most architecture posts.

One detail that matters: pair it with stale-while-revalidate. Serve the stale value while the background job refreshes, so a slightly-late rebuild never causes a miss.

Answer 2

Why C is the trap answer (Request coalescing):

Coalescing sounds most sophisticated — collapse all concurrent requests for the same key into one DB query, return the same result to all of them. No stampede, no lock contention. Elegant.

Here's where it breaks: coalescing requires in-process coordination. It works beautifully on a single server. On 50 Node.js instances behind a load balancer, each instance coalesces independently — so now you have 50 concurrent DB queries instead of 8,000. Better, but still a 50x spike at every TTL expiry. To coalesce across instances you need a distributed coordination layer, and now you've built something nearly as complex as a mutex without the simplicity.

Right answer for a single-process server. Partial solution at scale that engineers convince themselves is complete.

Answer 3

Why A is wrong (Mutex lock):

Mutex looks clean: one request gets the lock, rebuilds the cache, everyone else waits. No duplicate DB queries.

The problem is "everyone else waits." Under 8,000 req/sec, waiting means API threads blocked, request queue filling up, p99 latency spiking to seconds. You've traded a DB overload for an application-layer backup. The database survives; the user experience dies. And if the lock holder is slow — DB is struggling, rebuild takes 500ms — every request in the system stalls for that window.

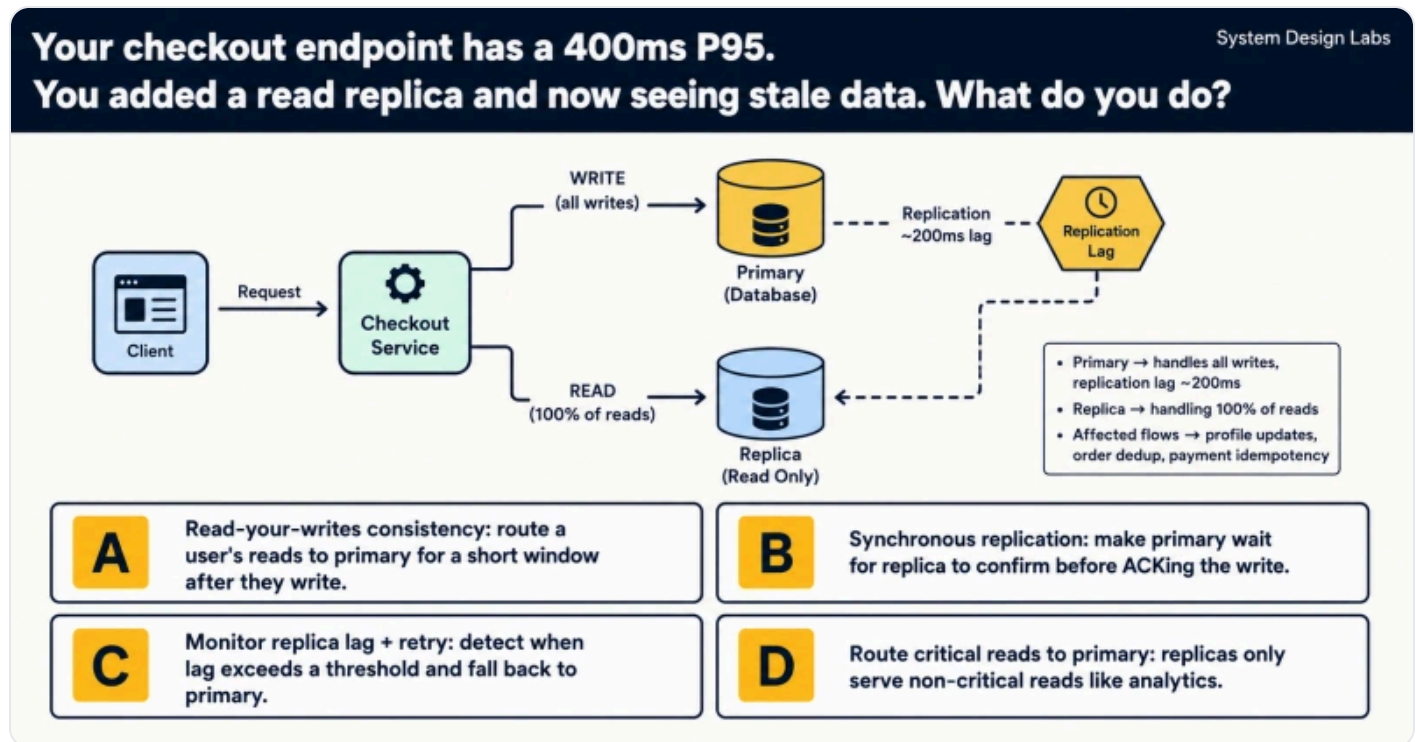
Answer 4

Why B is wrong (Probabilistic early expiry):

XFetch / jitter-based expiry is clever: as the TTL counts down, each cache read has a small random chance of triggering a rebuild early. No lock, no coordination layer. Elegant math.

The problem is it's probabilistic, not guaranteed. Under the exact wrong timing (low traffic right before TTL, then a sudden spike) the key still expires cold. You've reduced the probability of a thundering herd without eliminating it. For an endpoint running at 8,000 req/sec 24/7, you want a deterministic guarantee — not good odds.

Read-Your-Writes with Read Replicas



Your checkout endpoint has a 400ms P95. Profiling shows 70% of that is DB reads.

You add a read replica and point all SELECT queries at it. P95 drops to 90ms. The team celebrates.

Two hours later, support tickets flood in. Customers update their shipping address but see the old one on the confirmation screen. One customer gets charged twice because the "order already exists" check read stale data and missed the duplicate.

Here's the setup:

- Primary → handles all writes, replication lag ~200ms
- Replica → handling 100% of reads
- Affected flows → profile updates, order dedup, payment idempotency

The replica is working exactly as designed. That's the problem.

What do you do?

- A** Read-your-writes consistency: route a user's reads to primary for a short window after they write.

B Synchronous replication: make primary wait for replica to confirm before ACKing the write.

C Monitor replica lag + retry: detect when lag exceeds a threshold and fall back to primary.

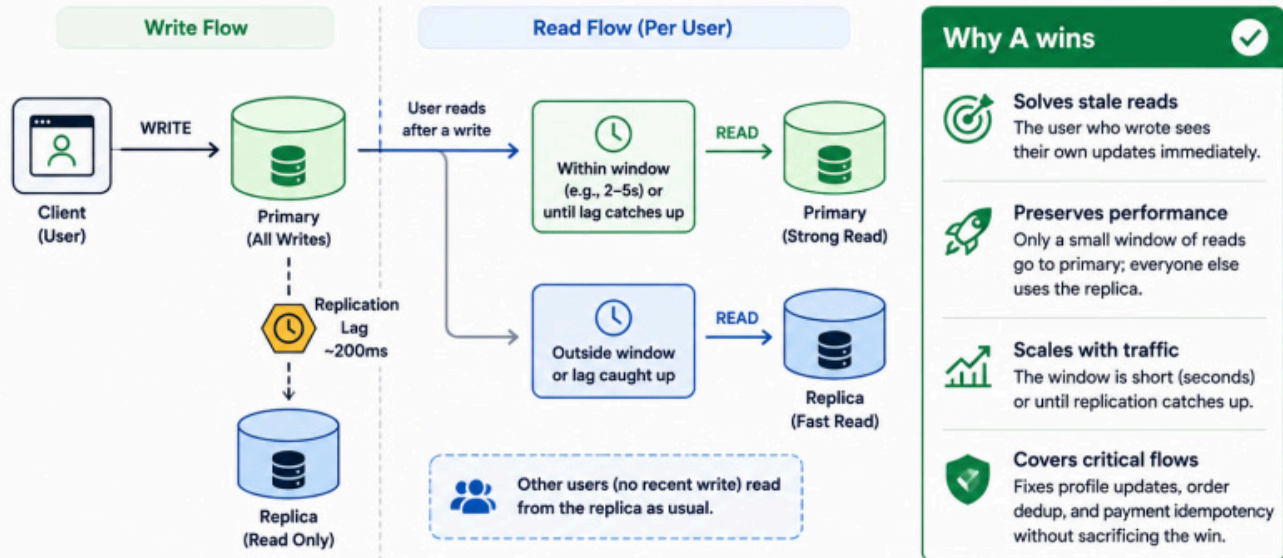
D Route critical reads to primary: replicas only serve non-critical reads like analytics.

Answer 1

✓ CORRECT ANSWER

ANSWER: A — Read-your-writes consistency:
route a user's reads to primary for a short window after they write.

System Design Labs



Key idea: After a user performs a write, their subsequent reads are routed to the primary for a short window: typically a few seconds, or until replica lag catches up. Everyone else reads from the replica. The performance win is preserved for the vast majority of traffic.

Why A wins:

After a user performs a write, their subsequent reads are routed to the primary for a short window: typically a few seconds, or until replica lag catches up. Everyone else reads from the replica. The performance win is preserved for the vast majority of traffic.

Answer 2

Why D is the trap:

"Critical" is not a stable category. The primary read list quietly shrinks, stale read bugs come back, and you spend the next quarter playing whack-a-mole with consistency bugs. You've traded a systematic solution for a per-feature judgment call.

Answer 3

Why B is wrong:

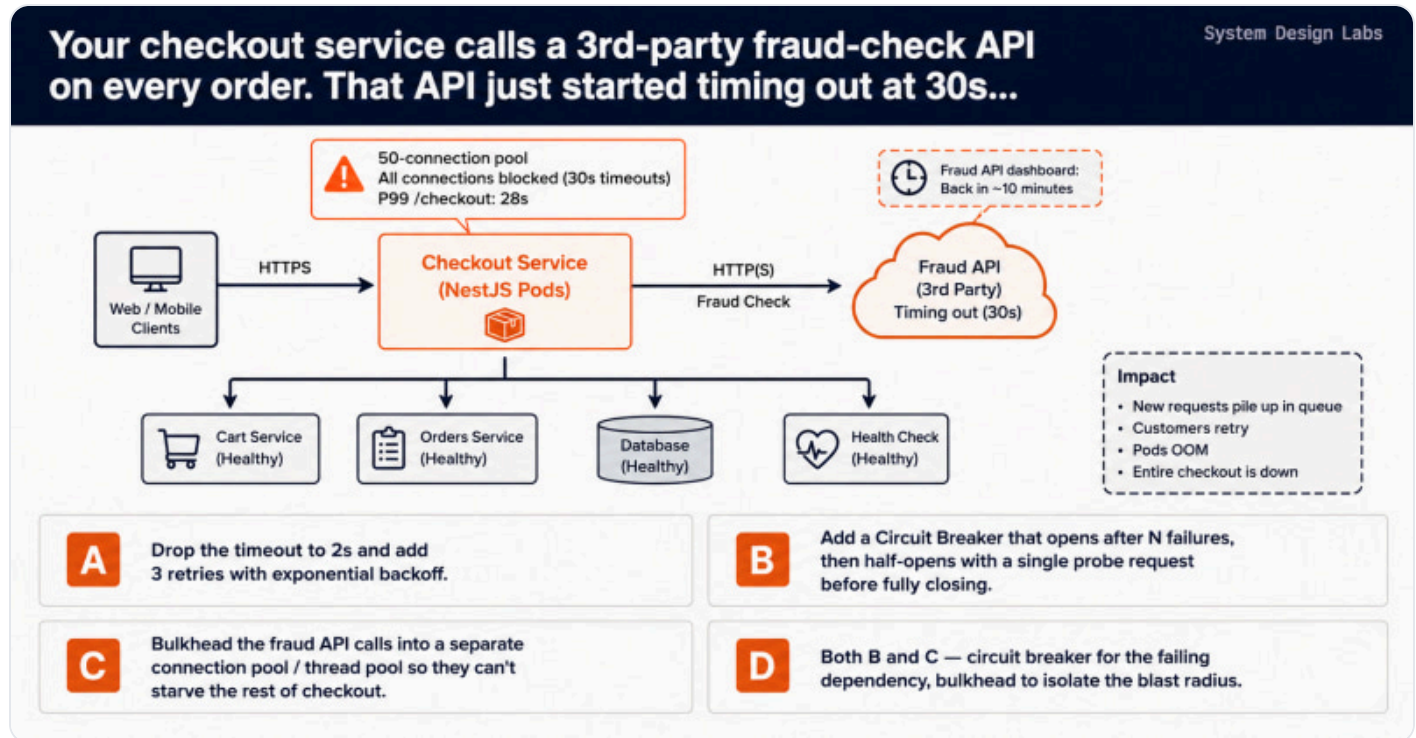
Synchronous replication means the primary waits for the replica to confirm every write. Replica lag goes to zero, but your write P95 goes from 20ms to 80–120ms. Worse, you've coupled your primary's availability to your replica's health.

Answer 4

Why C is wrong:

Average lag isn't your problem. Your problem is that a user reads their own write within 200ms of making it. The lag metric looks fine (200ms is "normal"), but the data the user needs is still in transit.

Containing a Failing Downstream Dependency



Your checkout service calls a 3rd-party fraud-check API on every order.

That API just started timing out at 30s instead of its usual 200ms.

Your Node.js checkout pods have a 50-connection pool. Within 90 seconds, every connection is parked waiting on the fraud API. New checkout requests pile up in the queue. P99 latency on /checkout goes from 300ms to 28s. Customers retry. Pods OOM. The fraud API is degraded — your entire checkout is down.

Here's the setup:

- Checkout (NestJS) → Fraud API (3rd party) — 30s timeouts
- Same pods also handle /cart, /orders, /health — all healthy dependencies
- Fraud API's own dashboard says it'll be back in ~10 minutes
- Your SLO budget for the quarter is about to evaporate

You need to stop the bleeding without losing the rest of checkout. What do you do?

A Drop the timeout to 2s and add 3 retries with exponential backoff.

B Add a Circuit Breaker that opens after N failures, then half-opens with a single probe request before fully closing.

C Bulkhead the fraud API calls into a separate connection pool / thread pool so they can't starve the rest of checkout.

D Both B and C — circuit breaker for the failing dependency, bulkhead to isolate the blast radius.

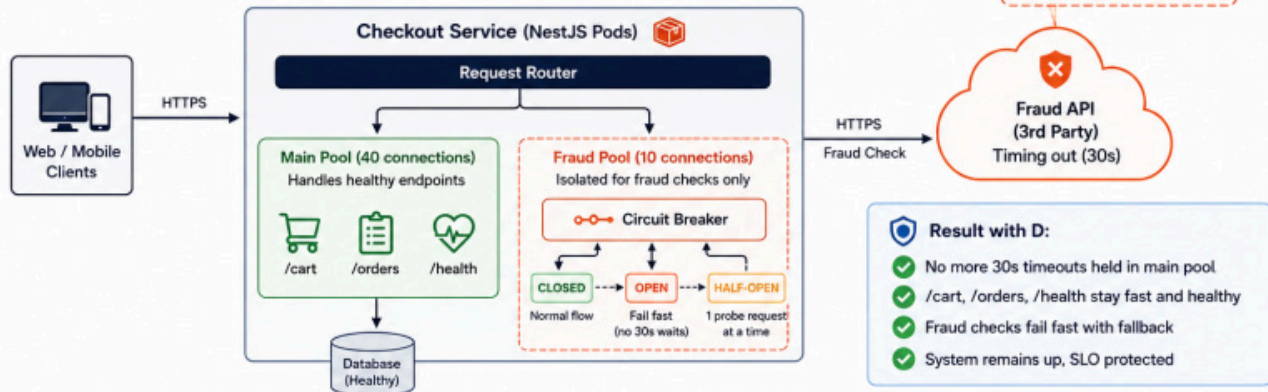
Answer 1 ✓ CORRECT ANSWER

Correct Answer: D – Circuit Breaker + Bulkhead, together ✓

System Design Labs



Why D wins: Circuit Breaker stops the bleeding. Bulkhead contains the blast radius. Together, they keep the rest of checkout healthy while the fraud API is degraded.

**A**

Drop the timeout to 2s and add 3 retries with exponential backoff.

✗ Worse: More requests, more load, more retries. You amplify the outage and burn resources faster.

B

Add a Circuit Breaker that opens after N failures, then half-opens with a single probe request before fully closing.

⚠ Better, but not enough: Without bulkhead, the fraud pool can still starve the rest of checkout.

C

Bulkhead the fraud API calls into a separate connection pool / thread pool so they can't starve the rest of checkout.

⚠ Better isolation, but without a circuit breaker you'll still wait full 30s timeouts and exhaust the fraud pool.

D

Both B and C — circuit breaker for the failing dependency, bulkhead to isolate the blast radius.

✓ Best outcome: Stop hammering the downed dependency + protect the rest of your service. This is what ships.

Answer: D — Circuit Breaker + Bulkhead, together ✓

Here's why, and why the other three trick smart engineers:

Why D wins (Circuit Breaker + Bulkhead):

These two patterns solve two *different* failure modes, and you need both.

The **Circuit Breaker** stops you from hammering a dead dependency. After N consecutive failures (or a failure rate threshold over a rolling window), it flips to OPEN — every subsequent call to the fraud API fails instantly with a fallback. No 30s wait. No connection held hostage. After a cooldown (say 30s), it goes HALF-OPEN: it allows exactly *one* probe request through. If that probe succeeds, the breaker closes and traffic resumes. If it fails, back to OPEN for another cooldown. Half-open is the part most tutorials gloss over — it's what prevents the thundering herd from re-killing a service that's just coming back up.

The **Bulkhead** is the part most engineers forget exists until they get burned. It isolates resource pools. Your fraud API gets its own dedicated pool of, say, 10 connections — separate from the 40 connections that serve /cart, /orders, /health. When the fraud API hangs, it can saturate its 10 connections, but

the other 40 stay free. /cart keeps working. /health keeps working. The blast radius stops at the fraud feature. The ship doesn't sink because one compartment flooded — that's literally where the name comes from.

Resilience4j, Polly (.NET), and Hystrix (legacy) all ship both. AWS App Mesh + Envoy give you bulkheads at the proxy layer. Netflix wrote Hystrix specifically because they learned this lesson the hard way: a single slow dependency cascading through shared thread pools took down recommendations during peak hours.

Answer 2

Why B alone is the staff-engineer trap answer:

A circuit breaker without a bulkhead is the answer that *sounds* complete in an interview. It's the most-mentioned pattern in resilience talks. And it does fix the specific failure described — once the breaker opens, fraud calls fail fast and stop tying up connections.

But here's the production reality: between failure #1 and the breaker tripping (failure #N), every one of those N requests is still holding a connection from the shared pool. If your threshold is 20 failures over 10 seconds, you can still saturate a 50-connection pool before the breaker even notices. You've reduced the outage window, not eliminated the cascade.

Bulkheads make the breaker's threshold *latency-tolerant*. They give you slack to detect and react without the rest of the system already being on fire.

This is the trap because B is right *and* incomplete. Senior engineers stop at B in design reviews because it sounds sufficient. Staff engineers ship D because they've seen B fail in production at 3am.

Answer 3

Why C alone is partial:

Bulkhead alone is better than nothing — it contains the damage. But your 10 dedicated fraud connections will still spend 30 seconds each waiting on doomed calls. You're paying full latency cost on every checkout, just on a smaller pool. Customers still see slow checkouts; you just don't lose /cart with them.

Bulkhead without a breaker = "the leak is contained, but the room is still flooding."

Answer 4

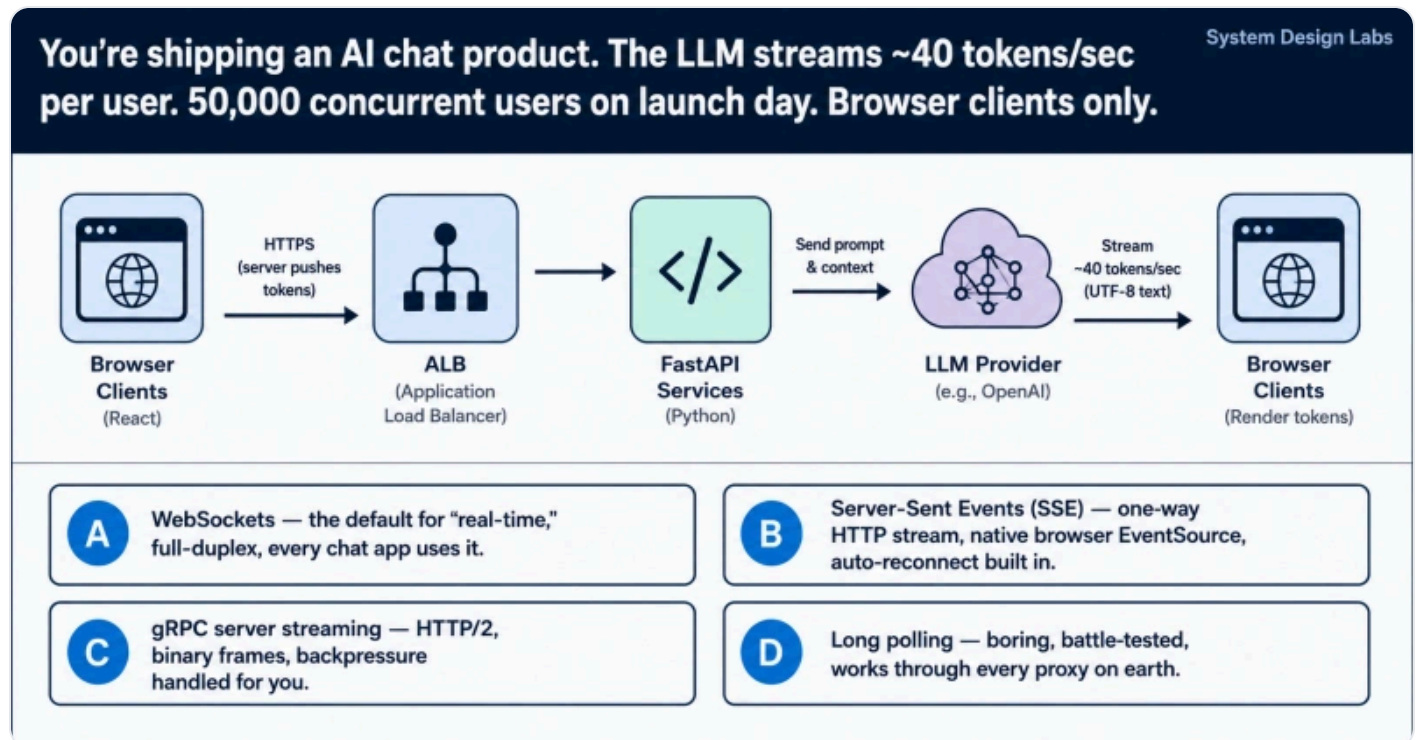
Why A is wrong (and dangerous):

Lower timeout + retries with backoff is the single worst thing you can do to a degraded dependency. It's the canonical anti-pattern.

The fraud API is already struggling. You just decided to send it 3x the traffic. Every retry storm from every checkout pod converges on a service that needs *less* load to recover. This is how a partial outage becomes a total outage — and how a 10-minute incident becomes a 4-hour incident. Retries belong on transient errors (a single 503, a connection reset), not on systemic degradation. And they always need a circuit breaker upstream of them to cap the blast.

If you've ever read a postmortem with the phrase "retry storm contributed to extended recovery time" — this is what they meant.

Choosing a Real-Time Streaming Transport



You're shipping an AI chat product. The LLM streams ~40 tokens/sec per user.

50,000 concurrent users on launch day. Browser clients only. Tokens flow one way: server → user.

Your team meets to pick the transport. Everyone shows up with a strong opinion.

Here's the setup:

- Frontend: React in the browser
- Backend: Python (FastAPI) behind an ALB
- Payload: UTF-8 text tokens, ~5–20 bytes each
- Direction: server pushes, client just renders
- Reconnects must be invisible (mobile networks drop constantly)

The team lead says "WebSockets, obviously." The platform engineer pushes back. What do you ship?

A WebSockets — the default for “real-time,” full-duplex, every chat app uses it.

B Server-Sent Events (SSE) — one-way HTTP stream, native browser EventSource, auto-reconnect built in.

C gRPC server streaming — HTTP/2, binary frames, backpressure handled for you.

D Long polling — boring, battle-tested, works through every proxy on earth.

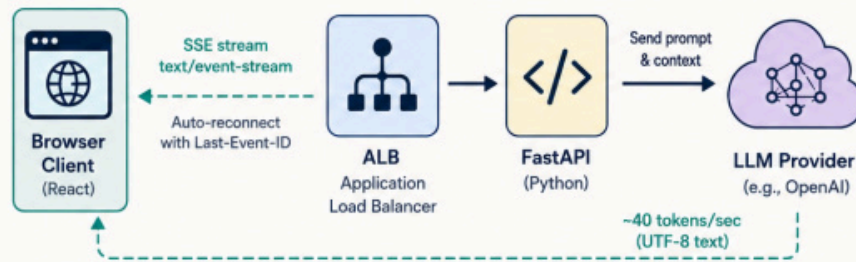
Answer 1

✓ CORRECT ANSWER

RIGHT ANSWER: B) SSE

System Design Labs

Server-Sent Events (SSE) — one-way HTTP stream, native browser EventSource, auto-reconnect built in.



- ✓ **One-way traffic.** Server pushes tokens, client renders. Perfect match for SSE.
- 🔄 **Built-in auto-reconnect.** EventSource reconnects and sends Last-Event-ID so the server can resume from there.
- 🛡️ **Simpler and more reliable.** No protocol upgrade. No new framing. No new auth path. Less code. Fewer bugs.

A WebSockets the default for "real-time," full-duplex, every chat app uses it.

- ✗ Overkill for one-way traffic
- ✗ Manual reconnect & resume logic
- ✗ Higher infra and operational cost

B Server-Sent Events (SSE) one-way HTTP stream, native browser EventSource, auto-reconnect built in.

- ✓ Built for one-way server push
- ✓ Automatic reconnect with Last-Event-ID
- ✓ Simple, efficient, and reliable

C gRPC server streaming HTTP/2, binary frames, backpressure handled for you.

- ✗ Browser support is poor
- ✗ Extra complexity for text tokens
- ✗ Harder to operate at scale

D Long polling boring, battle-tested, works through every proxy on earth.

- ✗ High overhead and latency
- ✗ Inefficient at scale
- ✗ Not real-time enough

Why B wins (SSE):

The traffic is one-way. Server pushes tokens, client renders. That's it. The client never sends a token back mid-stream — user input goes through a separate POST. So you're paying for full-duplex on a half-duplex problem.

SSE is built for exactly this shape. One long-lived HTTP connection, text/event-stream, server writes, browser's native EventSource reads. No protocol upgrade. No new framing. No new auth path.

The killer feature: **automatic reconnect with Last-Event-ID**. Browser drops, mobile switches from WiFi to LTE — EventSource reconnects on its own and tells the server the last event it saw. You replay from there. With WebSockets, you write that logic yourself, and you write it wrong the first three times.

OpenAI's streaming API? SSE. Anthropic's streaming API? SSE.

Answer 2

Why A is the trap (WebSockets):

The word "chat" tricks people. User-to-user chat (Slack, WhatsApp) is bidirectional. An AI chat app is a unidirectional token stream with a separate POST for the prompt. Different shape, different transport.

The cost at 50K connections:

- Sticky sessions on the ALB
- Hand-rolled reconnect + replay logic
- Heartbeats and ping/pong to detect dead connections
- Higher per-connection memory for buffers you'll never use

Every one of those turns into an on-call page.

Answer 3

Why C is wrong (gRPC streaming):

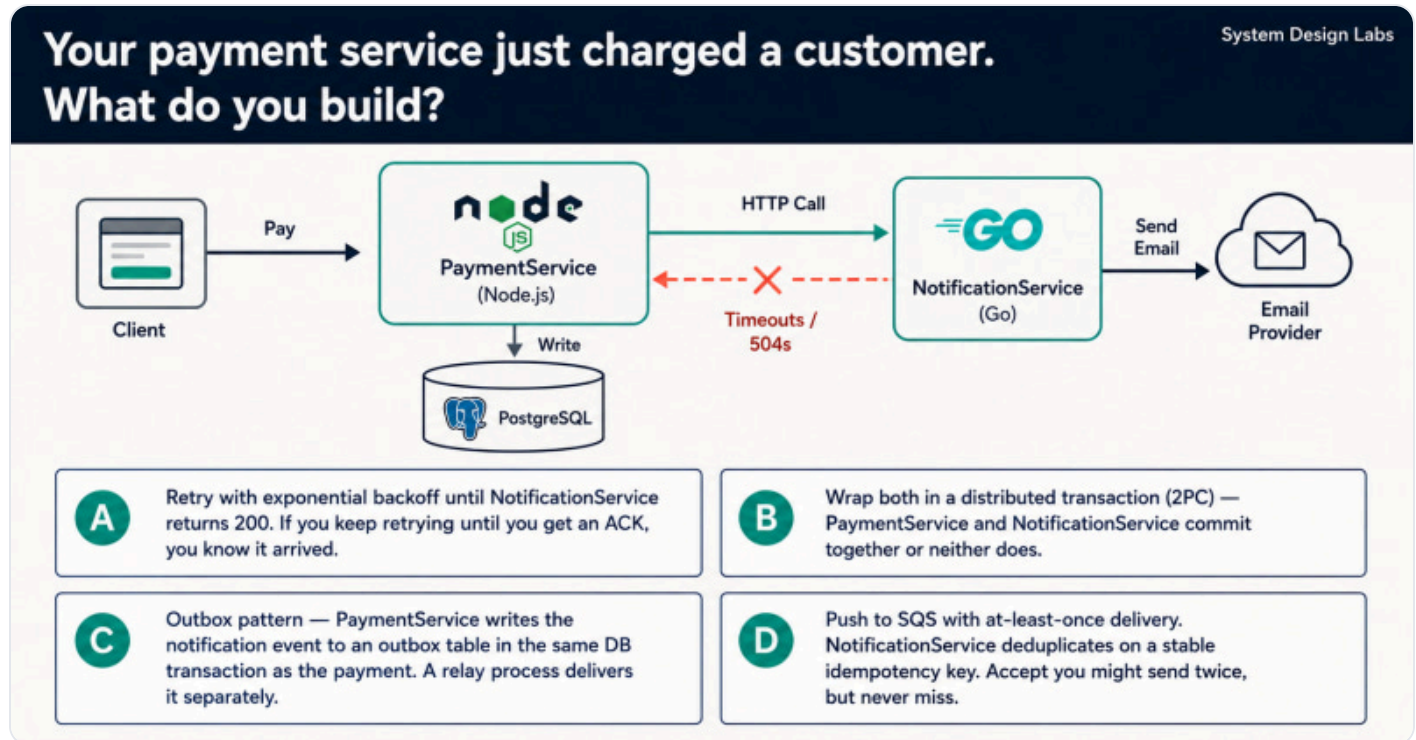
Browsers don't speak gRPC. You'd need Envoy + gRPC-Web, which downgrades the streaming model and adds a proxy hop. You're now operating Envoy and debugging Protobuf frames in the network tab — to ship UTF-8 tokens. Great for service-to-service. Wrong tool for browsers.

Answer 4

Why D is wrong (Long polling):

At 40 tokens/sec, every poll cycle burns a full HTTP request. $50K \text{ users} \times 40 \text{ req/sec} = \text{2M RPS just to render text}$. It's a valid fallback when SSE/WS are blocked. It's not your primary transport in 2026.

Reliable Messaging Across Services



Your payment service just charged a customer.

It writes to the DB. Now it needs to tell the notification service: "Send the confirmation email."

The HTTP call times out. Did it arrive? You don't know. You retry. Customer gets two emails.

You've just hit the Two Generals Problem. It's not a bug. It's a proof.

No protocol over an unreliable channel can guarantee both sides agree on the final message. Not HTTP. Not TCP. Not your retry loop. The uncertainty is mathematically irreducible.

Here's the setup:

PaymentService (Node.js, PostgreSQL) → NotificationService (Go)

~40ms p99 latency, occasional 504s under load.

You need to send exactly one confirmation email per payment — no double-sends, no missed sends.

What do you build?

A) Retry with exponential backoff until NotificationService returns 200. If you keep retrying until you get an ACK, you know it arrived.

B) Wrap both in a distributed transaction (2PC) — PaymentService and NotificationService commit together or neither does.

C) Outbox pattern — PaymentService writes the notification event to an outbox table in the same DB transaction as the payment. A relay process delivers it separately.

D) Push to SQS with at-least-once delivery. NotificationService deduplicates on a stable idempotency key. Accept you might send twice, but never miss.

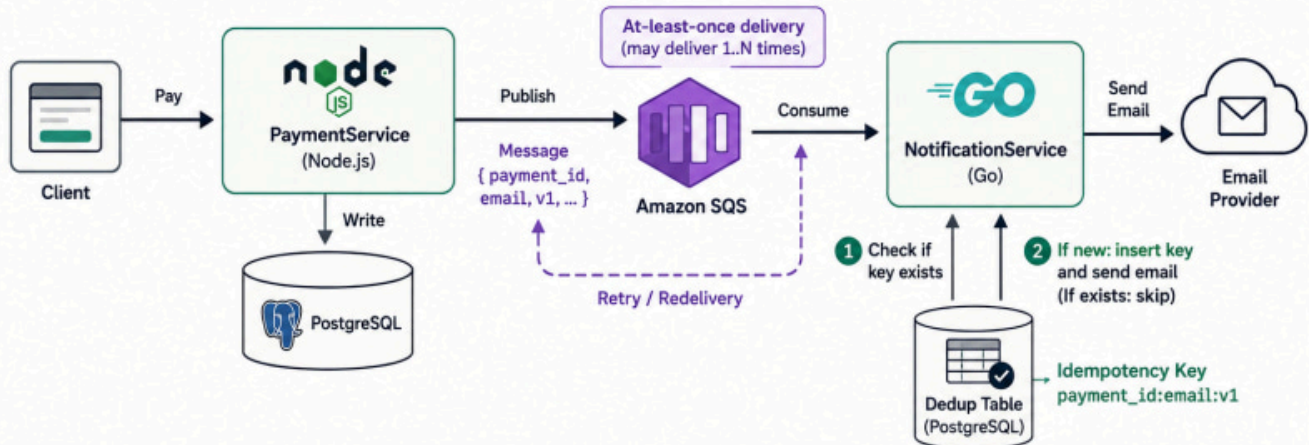
Answer 1

✓ CORRECT ANSWER

D

Push to SQS with at-least-once delivery.
NotificationService deduplicates on a stable idempotency key.
Accept you might send twice, but never miss.

✓ CORRECT



Accept the reality
 Network + systems are unreliable. Messages will be delivered — maybe more than once.



Queue absorbs uncertainty
 SQS durably stores the message and guarantees at-least-once delivery.



Idempotency absorbs duplicates
 Dedup table with key (`payment_id:email:v1`) makes replays a no-op.



Outcome
 No missed sends. No double-sends in practice.

★ This is how Stripe and AWS handle it: Queue + Idempotency = Exactly-once effects in practice.

D — SQS + at-least-once + idempotency key (CORRECT)

The only answer that accepts the impossibility and designs around it. At-least-once means the message will arrive — maybe twice. Idempotency on the consumer side (`payment_id:email:v1` in a dedup table) makes the second delivery a no-op. No missed sends, no double-sends in practice. This is exactly how Stripe and AWS handle it. The queue absorbs the uncertainty. Idempotency absorbs the duplicates.

Answer 2

A — Retry until ACK (SENIOR ENGINEER TRAP)

Feels airtight. It's not. What if the 200 response is what timed out — not the request?

NotificationService sent the email AND returned 200, but you never saw the response. So you retry. Second email sent. Now you need to confirm your ACK arrived too... which is the Two Generals recursion. No finite number of retries closes this loop. More handshakes = more latency and more failure surfaces, not more certainty.

Answer 3

B — 2PC (WRONG)

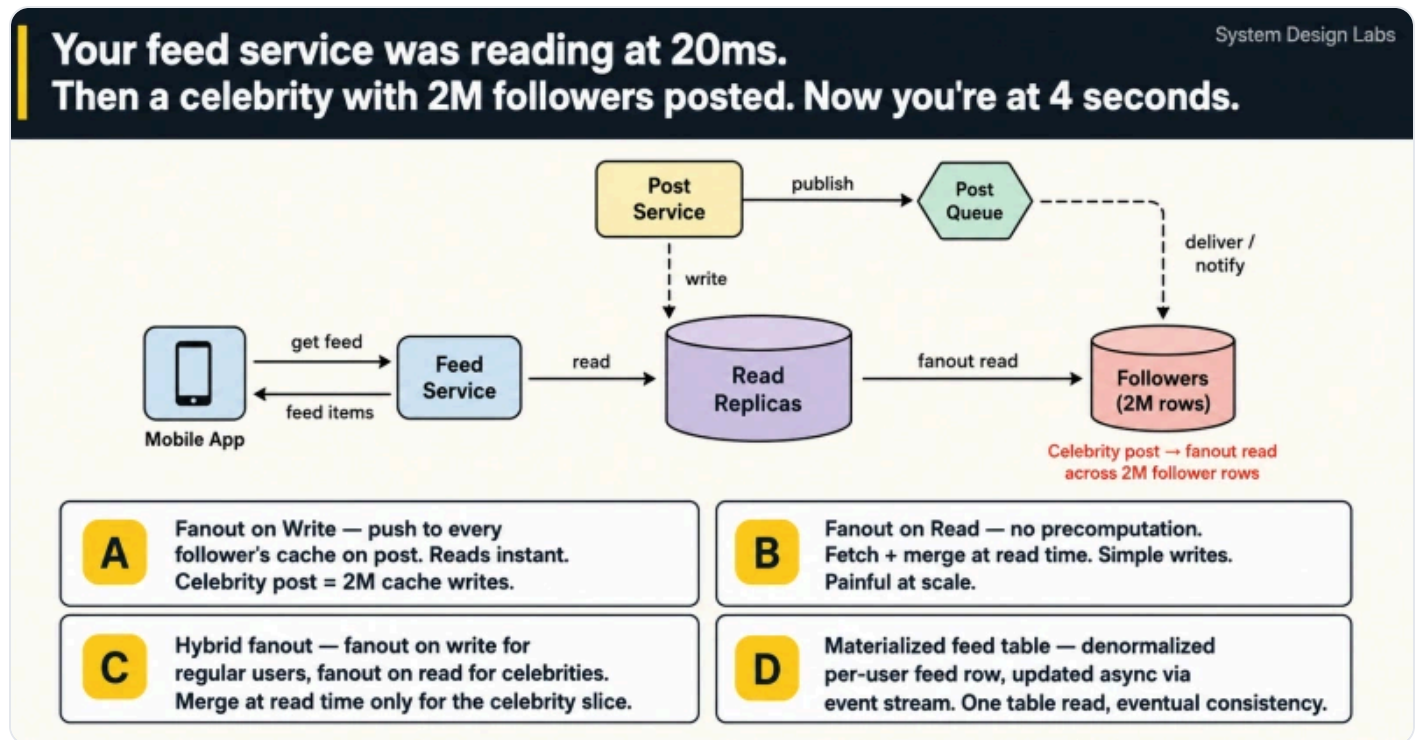
The coordinator is a single point of failure. If it crashes between Phase 1 (prepare) and Phase 2 (commit), both services are stuck with locks held, waiting for a decision that never comes. You've traded message uncertainty for coordinator-failure uncertainty — same problem, more complexity.

Answer 4

C — Outbox Pattern (CORRECT but heavier)

Actually solid — atomicity at the DB level, event and payment always in sync. But you're now running a relay process + CDC pipeline + outbox cleanup. For most teams with SQS already in place, D gets 95% of the same guarantees with a fraction of the infrastructure. C shines at scale or when strict ordering matters. D is the right answer for most teams today.

Feed Fanout for Celebrity Accounts



Your feed service was reading at 20ms.

Then a celebrity with 2M followers posted.

Now you're at 4 seconds. P99 is on fire.

Here's the setup:

10M users. ~50k posts/day. One account: 2M followers.

Every time that account posts, your feed query fans out across 2M follower rows, sorts by timestamp, and buries your read replicas.

Your team is debating how to fix the delivery model — not the query, the architecture.

A) Fanout on Write — push to every follower's cache on post. Reads instant. Celebrity post = 2M cache writes.

B) Fanout on Read — no precomputation. Fetch + merge at read time. Simple writes. Painful at scale.

C) Hybrid fanout — fanout on write for regular users, fanout on read for celebrities. Merge at read time only for the celebrity slice.

D) Materialized feed table — denormalized per-user feed row, updated async via event stream. One table read, eventual consistency.

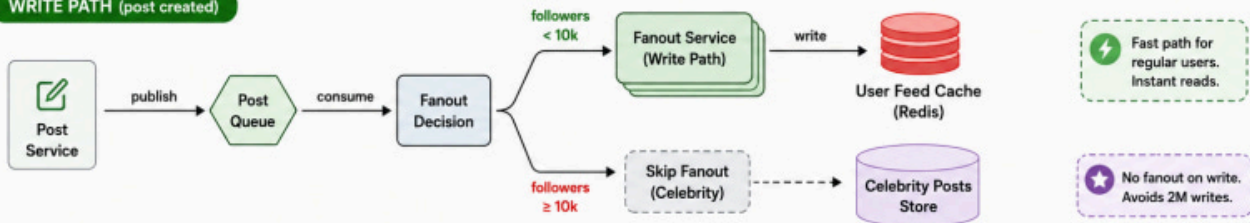
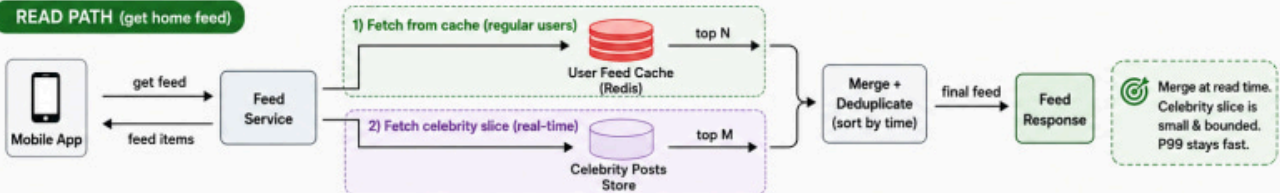
Answer 1 ✓ CORRECT ANSWER

ANSWER: C — Hybrid fanout ✓

Fanout on write for regular users, fanout on read for celebrities.
Merge at read time only for the celebrity slice.

Why C wins

- ✓ Avoids 2M synchronous writes
- ✓ Keeps reads fast (P99)
- ✓ Bounded fanout at read time
- ✓ Used by Twitter at scale

WRITE PATH (post created)**READ PATH (get home feed)**

Key Insight
Hybrid is honest about both failure modes:
Fanout-on-write is great for most users.
Fanout-on-read is reserved for the few heavy hitters.



< 10k followers
Fanout on write
(instant reads)



≥ 10k followers
Fanout on read
(bounded cost)



Result
Fast P50 / P99
at scale

Answer: C — Hybrid fanout**Why C wins:**

This is how Twitter actually solved it — they published the architecture. The insight: fanout-on-write is fast for readers but catastrophic for high-follower accounts. Fanout-on-read is simple but destroys latency at scale. Hybrid is honest about both failure modes.

When a post is created → check follower count. Under 10k? Fan out immediately to all followers' Redis feed caches. Over 10k (celebrity)? Skip the fanout. At read time, fetch from two sources: precomputed cache (regular accounts) + real-time query for celebrity accounts. Merge + deduplicate. The celebrity slice is small and bounded. Result: P99 stays fast, no 2M synchronous writes on celebrity posts.

Answer 2

Why A is the trap (Fanout on Write):

Looks perfect. Reads are instant. Until Cristiano Ronaldo posts and you're writing to 500M caches in parallel. At 2M followers, even 1ms per write = 2,000 seconds of write work. Cache cluster saturates. Write queue backs up. Thundering herd on every celebrity post.

Teams implement this, it works great at 1M users, then hits the celebrity problem when it's already in production. That's the trap.

Answer 3

Why B is wrong (Fanout on Read):

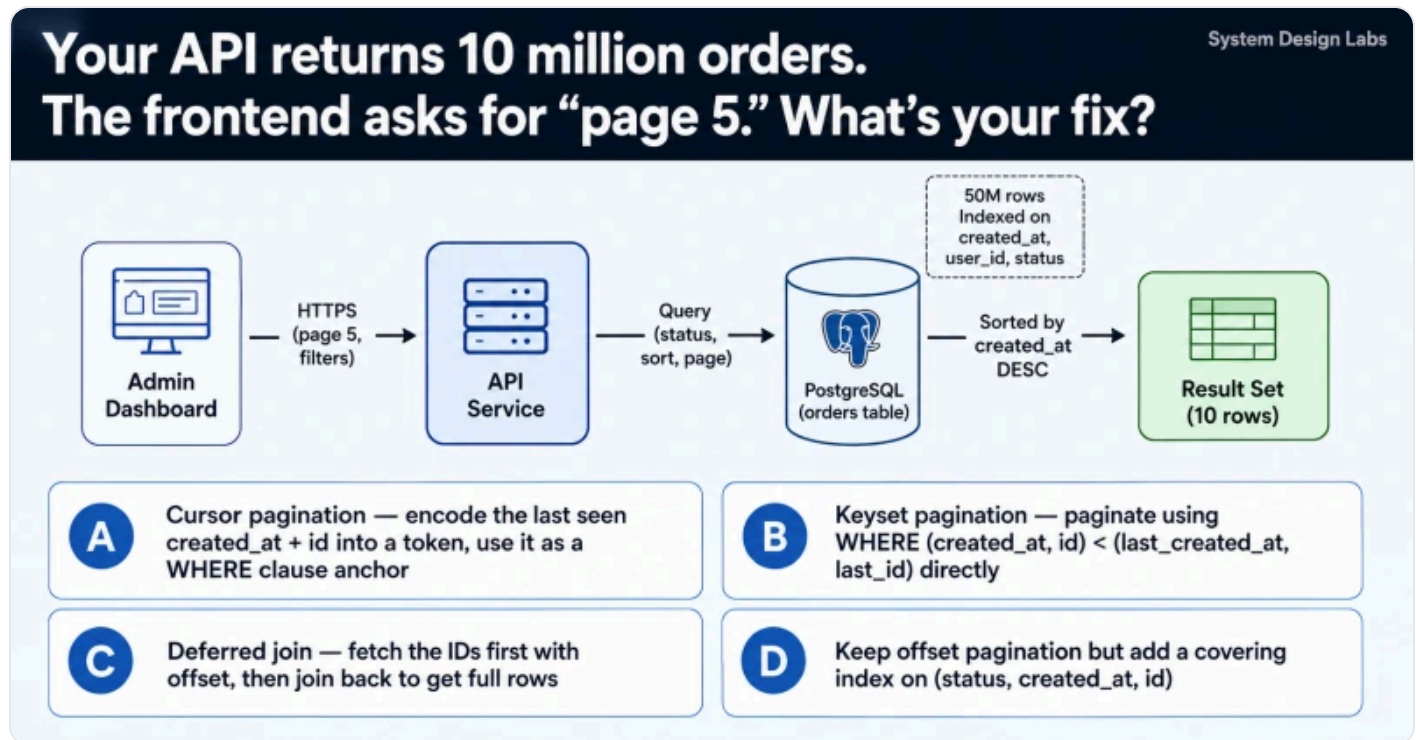
500 accounts followed = 500 queries (or one giant JOIN), sorted in memory, before returning anything. You can cache the result — but now you're doing expensive computation to fill a cache that expires in 30s. That's just a slow fanout-on-write with extra latency. Doesn't survive 10M users.

Answer 4

Why D is wrong (Materialized feed table):

Sounds clean — denormalized row per user, updated async via CDC/event stream. But a celebrity post still triggers 2M row updates, just asynchronously. You haven't solved write amplification, you've deferred it. Plus: operational complexity of an event stream + idempotent consumers + reconciliation logic. More moving parts for the same problem.

Paginating Large Result Sets Efficiently



Your API returns 10 million orders.

The frontend asks for "page 5."

Offset 40, limit 10. Simple enough — until your DBA messages you at 2 AM.

Query time: 4.2 seconds. Table scans climbing. Prod is lagging.

Here's the stack: PostgreSQL, 50M rows in orders, sorted by `created_at` DESC. Users can filter by status. You're paginating an admin dashboard used by 200 concurrent support agents.

Offset pagination worked fine at 10K rows. At 50M, it's reading and discarding 40 million rows just to return 10.

You need to fix this before the next sprint ships the customer-facing version — which will have 10x the traffic.

Here's your setup:

- Table: orders — 50M rows, indexed on `created_at`, `user_id`, `status`
- Query: sorted by `created_at` DESC, filtered by status
- Client: needs "previous/next" navigation + jump-to-page

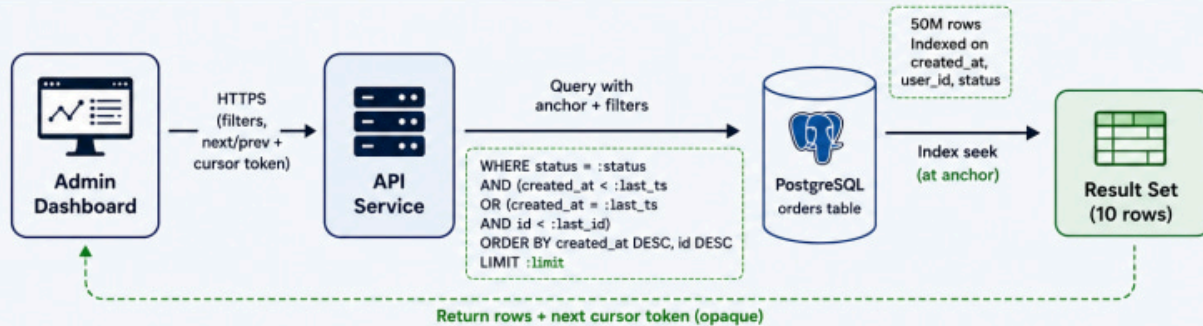
- SLA: p99 < 200ms

Four engineers propose four different approaches:

- A** Cursor pagination — encode the last seen `created_at` + `id` into a token, use it as a `WHERE` clause anchor
- B** Keyset pagination — paginate using `WHERE (created_at, id) < (last_created_at, last_id)` directly
- C** Deferred join — fetch the IDs first with offset, then join back to get full rows
- D** Keep offset pagination but add a covering index on `(status, created_at, id)`

Answer 1 ✓ CORRECT ANSWER**A****ANSWER: CURSOR PAGINATION**

Encode the last seen (created_at, id) into a token.
Use it as an anchor for the next query.

 System Design Labs
**WHY THIS WINS HERE**

- ✓ Query always hits the index at the anchor point — no scan, no discard.
- ✓ p99 stays flat whether you're on page 1 or page 50,000.
- ✓ Works with the status filter:

```
WHERE status = 'pending' AND (created_at, id) < (anchor)
```
- ✓ The cursor is position-stable: new rows inserted before your cursor don't shift your page.

HOW THE CURSOR TOKEN WORKS**UX TRADEOFF**

No jump-to-page. Only previous/next. But this is the right tradeoff for the dashboard. Support agents navigate sequentially. Nobody needs "jump to page 3,847."

**THE BOTTOM LINE**

At 50M rows with concurrent filters, cursor pagination is the only approach that hits p99 < 200ms reliably.

p99 < 200ms

**Option A — Cursor pagination** ✓ CORRECT

Encode the last seen (created_at, id) into an opaque token. On the next request, decode it and use `WHERE created_at < :last_ts OR (created_at = :last_ts AND id < :last_id)` as your anchor.

Why this wins here:

- Query always hits the index at the anchor point — no scan, no discard
- p99 stays flat whether you're on page 1 or page 50,000
- Works with the status filter: `WHERE status = 'pending' AND (created_at, id) < (anchor)`
- The cursor is position-stable: new rows inserted before your cursor don't shift your page

The UX tradeoff: no jump-to-page. Only previous/next. **But this is the right tradeoff for the dashboard.** Support agents navigate sequentially. Nobody needs "jump to page 3,847."

At 50M rows with concurrent filters, cursor pagination is the only approach that hits p99 < 200ms reliably.

Answer 2

Option B — Keyset pagination

Keyset looks identical to cursor on the surface. The difference is subtle: keyset is a *query pattern*, cursor is a *product decision*.

Keyset uses `WHERE (created_at, id) < (val1, val2)` as a SQL construct. Cursor wraps that into an encoded token the client can't inspect or manipulate.

Here's where it falls apart: **the status filter breaks row-tuple comparison in most query planners.** `WHERE status = 'pending' AND (created_at, id) < (anchor)` doesn't use a composite index efficiently in PostgreSQL when the equality column (status) isn't part of the tuple comparison. You end up with a partial index scan that degrades under high cardinality filters.

Cursor pagination handles this by encoding the full WHERE context, not just the positional anchor. That's the gap. It's subtle, and it's the thing that bites you in production three months after you ship it...

Answer 3

Option C — Deferred join

```
SELECT o.* FROM orders o

JOIN (

    SELECT id FROM orders

    WHERE status = 'pending'

    ORDER BY created_at DESC

    OFFSET 500000 LIMIT 10

) ids ON o.id = ids.id
```

This is a legitimate optimization for offset pagination. The inner query fetches only IDs (fits in index, much faster), then the outer join fetches full rows for just 10 records.

It's real. It works. Shopify uses a version of this.

But: you still pay $O(\text{offset})$ cost on the inner query at deep pages. It helps — it does not solve. At page 50,000, you're still scanning 500,000 index entries. You shaved 80% of the cost and kept 20% of the problem.

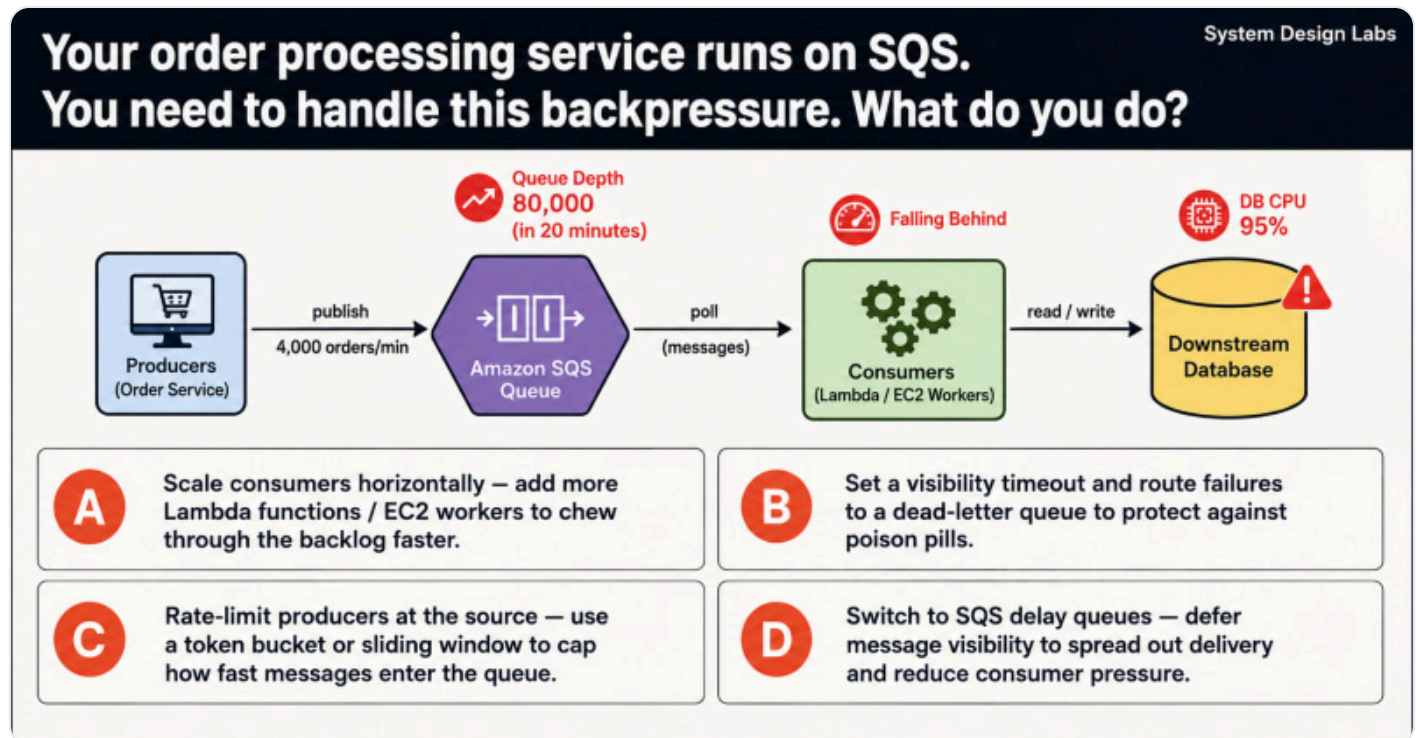
Answer 4

Option D — Covering index

Adding `CREATE INDEX ON orders(status, created_at DESC, id DESC)` is a good idea regardless of which approach you pick. It removes heap fetches for status-filtered queries.

But an index doesn't change the algorithmic complexity of `OFFSET`. It makes your table scans faster, not shorter. This is the "throw hardware at it" answer. It buys you time, not a solution.

Queue Backpressure Under a Traffic Spike



Your order processing service runs on SQS.

Normal load: 200 orders/min. Consumers keep up fine.

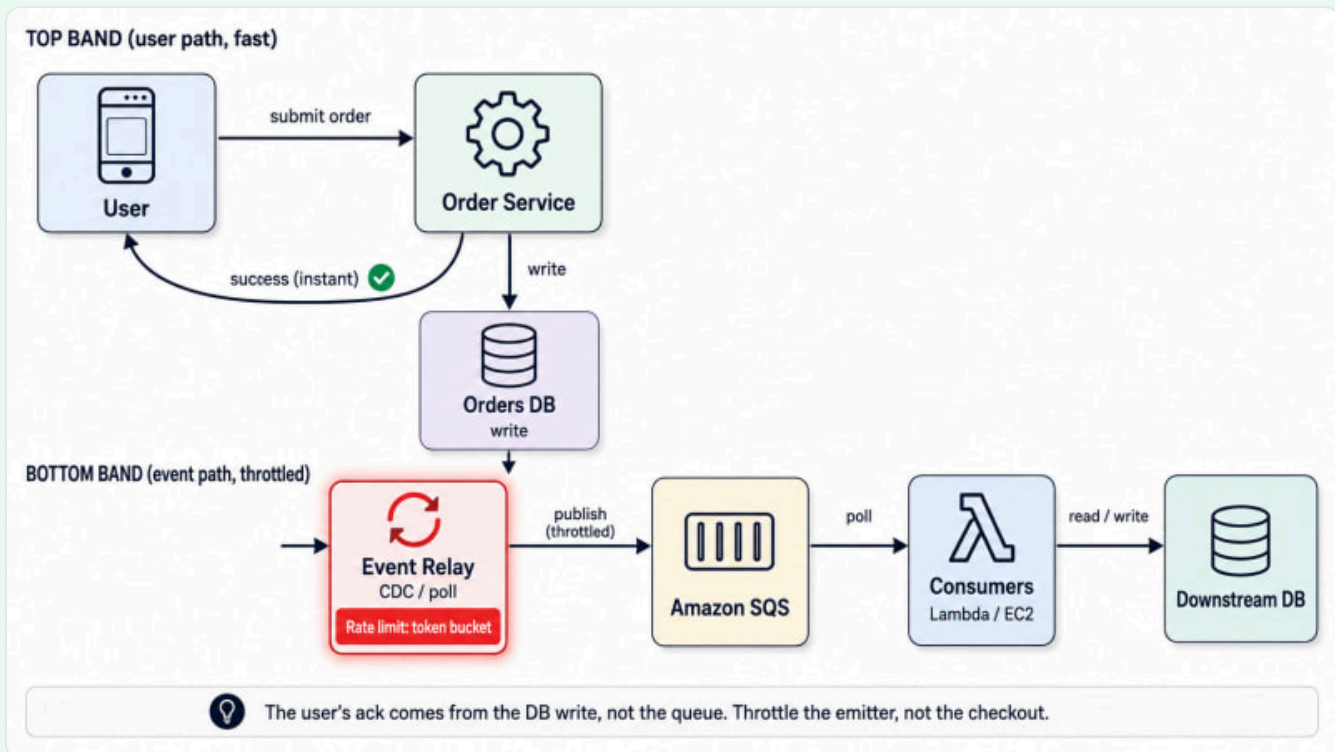
Then Black Friday hits. Producers start pushing 4,000 orders/min. Queue depth climbs to 80,000 messages in 20 minutes. Your downstream DB is at 95% CPU. Consumers are falling behind and you're watching the queue grow in real time.

You need to handle this backpressure. What do you do?

- A) Scale consumers horizontally — add more Lambda functions / EC2 workers to chew through the backlog faster.
- B) Set a visibility timeout and route failures to a dead-letter queue to protect against poison pills.
- C) Rate-limit producers at the source — use a token bucket or sliding window to cap how fast messages enter the queue.
- D) Switch to SQS delay queues — defer message visibility to spread out delivery and reduce consumer pressure.

Answer 1

✓ CORRECT ANSWER



Why C wins, and why it doesn't wreck UX

The queue grows because the producer is winning the race. 4,000 in, 200 out. You can scale consumers all day and that gap won't close on its own, especially with the DB already sitting at 95% CPU.

The fix is upstream. Rate-limit the producer with a token bucket or sliding window. In AWS that's API Gateway usage plans, Lambda reserved concurrency, or throttle middleware inside the service. Queue depth stabilizes, consumers catch up at their natural pace, and the DB stops cooking.

The mental model: slow the tap, don't just widen the drain.

Now the part the diagram glossed over: the "producer" isn't your user.

In most real systems the order flow is decoupled. A user submits an order, it's written to the DB, and they get their success response right away. A separate processor then reads those events (polling or CDC) and emits them to the queue. That emitter is the producer you throttle.

So rate-limiting never touches checkout. The user already has their ack from the write. You're capping how fast events flow into the queue, not how fast users can order. UX stays intact; the queue just stops outrunning your consumers.

That's what backpressure actually is: the downstream system telling the upstream to slow down. The queue depth is the signal. You react to it upstream.

Answer 2

Why A is the trap answer (Scale consumers):

This is the instinct answer, and it's close enough to fool senior engineers.

Yes, more consumers means more throughput. But you're treating a **rate mismatch** as a **capacity problem**. If your producer can always generate faster than your consumers can process — and on Black Friday it can — you're in an arms race you can't win. You spin up 10x Lambda functions, hit DynamoDB write throughput limits, hammer your RDS connection pool, and the queue still grows. Just slower, and now your infra bill doubled.

Consumer scaling works when your bottleneck is compute. It doesn't work when the problem is unbounded producer rate.

Answer 3

Why B is wrong (Visibility timeout + DLQ):

This is failure handling, not backpressure.

DLQ catches poison pills — messages that fail processing repeatedly and would otherwise loop forever. Visibility timeout controls how long a message stays invisible after a consumer picks it up (preventing double-processing). Neither of these slows the producer or reduces queue depth growth.

Mixing up backpressure with retry/error handling is one of the most common architecture confusions I see. They solve completely different problems.

Answer 4

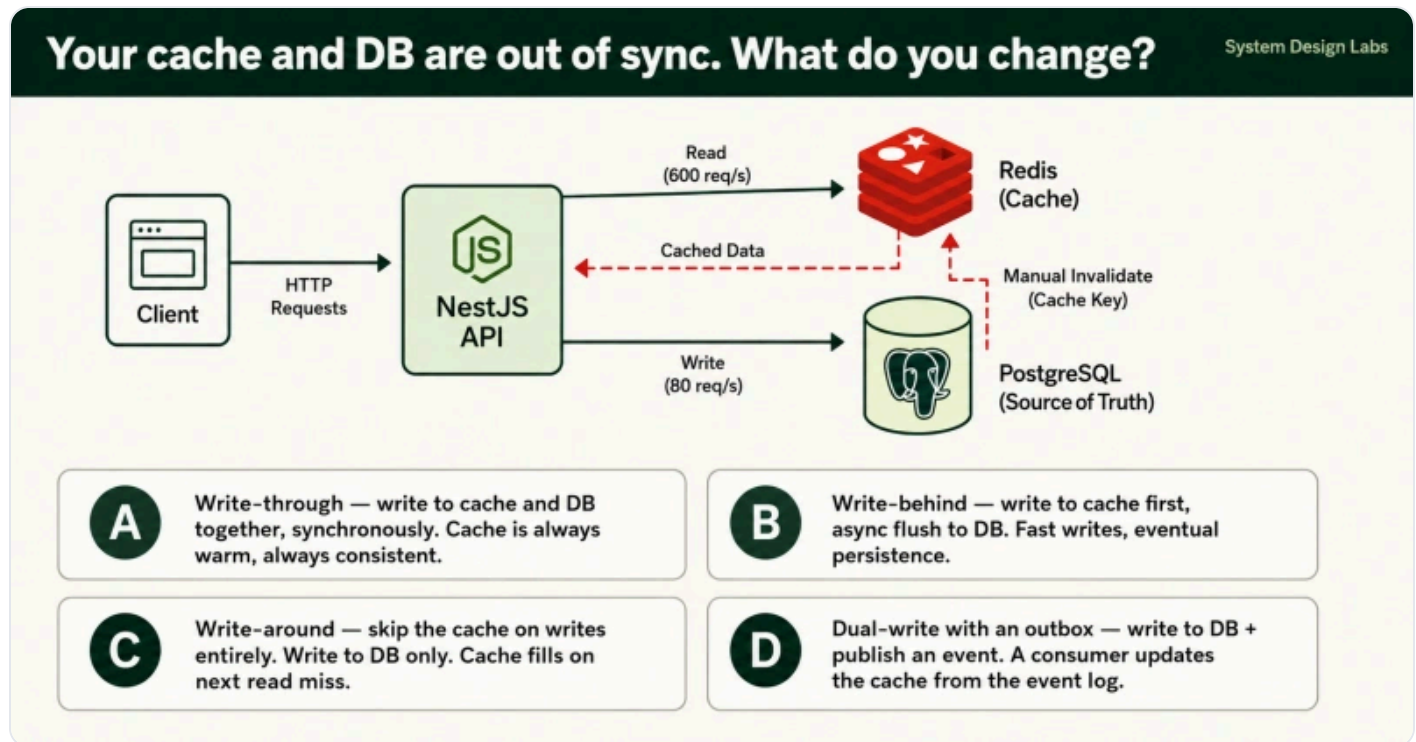
Why D is wrong (SQS delay queues):

Delay queues defer message *visibility*, not message *creation*. Your producer still pushes 4,000 msg/min. Messages pile up invisibly, then become visible in bursts when the

delay expires. You haven't reduced backpressure — you've deferred it and made the delivery pattern spikier.

This is like covering a flooding sink with a lid and calling it fixed. The water is still rising....

Write-Path Cache Consistency



Your cache and DB are out of sync. Again.

A user updates their profile. The cache still serves the old name for the next 10 minutes. Support gets a ticket. You patch it with a cache flush. It happens again next week.

You're asked to fix write consistency before it becomes a customer-facing incident.

Here's the setup:

NestJS API → PostgreSQL (source of truth) + Redis (cache)

~600 req/s reads, ~80 req/s writes at peak

Current pattern: write to DB, manually invalidate cache key on success

3 incidents this month — all traced back to stale cache after writes

You need a strategy that survives race conditions, retries, and partial failures

What do you change?

- A** **Write-through** — write to cache and DB together, synchronously. Cache is always warm, always consistent.

B Write-behind — write to cache first, async flush to DB. Fast writes, eventual persistence.

C Write-around — skip the cache on writes entirely. Write to DB only. Cache fills on next read miss.

D Dual-write with an outbox — write to DB + publish an event. A consumer updates the cache from the event log.

Answer 1

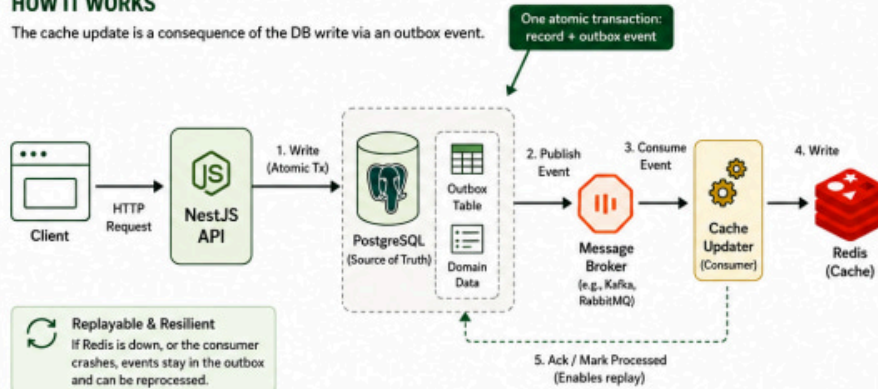
✓ CORRECT ANSWER

ANSWER: D — DUAL-WRITE WITH AN OUTBOX ✓

System Design Labs

HOW IT WORKS

The cache update is a consequence of the DB write via an outbox event.



WHY D WINS

- ✓ Eliminates the dual-write race condition: DB write and outbox event are in the same atomic transaction.
- ✓ The cache update becomes a consequence of the DB write, not a sibling operation.
- ✓ Survives crashes, retries, and partial failures.
- ✓ Provides replay: if Redis is down and comes back, re-process the outbox and catch up.

A

WRITE-THROUGH

Write to cache and DB together, synchronously.



- ✗ Two synchronous I/Os on every write.
- ✗ If Redis is slow, your write API is slow.
- ✗ If Redis is down, do you block the user? You've made Redis a hard dependency of your write path.

B

WRITE-BEHIND

Write to cache first, async flush to DB.



- ✗ Fast, but dangerous.
- ✗ If the async worker crashes before DB flush, that write is gone.
- ✗ Acceptable for analytics counters. Never acceptable for a source of truth.

C

WRITE-AROUND

Skip the cache on writes. Write to DB only.



- ✗ Solves stale cache by not writing to cache at all.
- ✗ Read-after-write is broken under replica lag.
- ✗ High-write workloads tank your cache hit rate.

D

DUAL-WRITE WITH AN OUTBOX

Write to DB + publish an event. A consumer updates the cache from the event log.



- ✓ DB write + outbox event are atomic.
- ✓ Cache update is a consequence of the DB write.
- ✓ Survives crashes, retries, and partial failures.
- ✓ Replayable and eventually consistent.

Answer: D — Dual-write with an outbox

The core problem isn't "which order to write in" — it's "what happens when one write succeeds and the other fails?"

Why D wins: Every naive dual-write has the same race condition: write to DB, write to cache, crash between the two → stale cache indefinitely. The outbox makes the cache update a *consequence* of the DB write, not a sibling operation. One atomic DB transaction: record + outbox event. A consumer updates Redis from the event. Also gives you replay — if Redis goes down and comes back, re-process the outbox.

Answer 2

Why A fails (Write-through): Looks safest. Is the most dangerous at scale. Two synchronous I/Os on every write. If Redis is slow, your write API is slow. If Redis is down, do you block the user? You've made Redis a hard dependency of your write path.

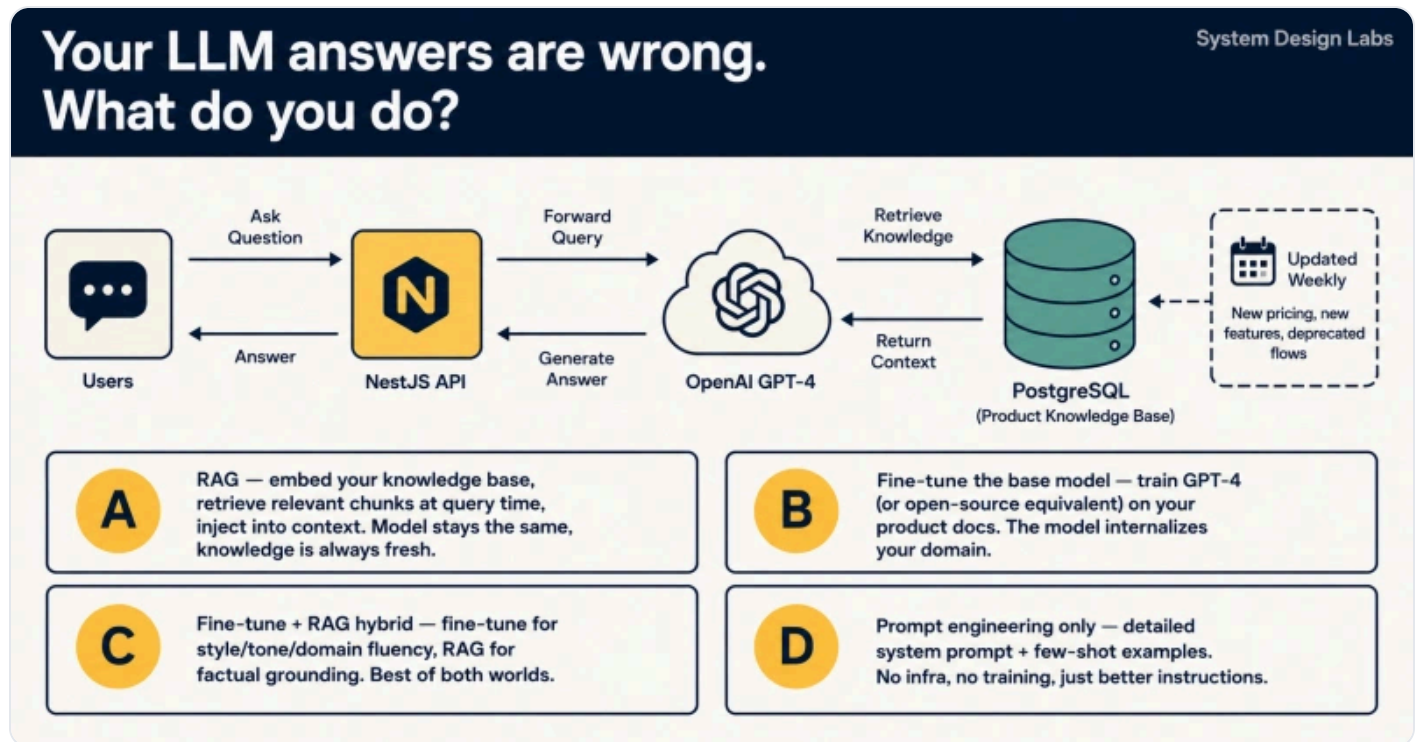
Answer 3

Why B fails (Write-behind): Fast, but: if the async worker crashes before DB flush, that write is gone. Acceptable for analytics counters. Never acceptable for a source of truth.

Answer 4

Why C is a partial fix (Write-around): Solves stale cache by not writing to cache at all. Clean — but read-after-write is broken under replica lag, and high-write workloads tank your cache hit rate.

Keeping an LLM's Answers Up to Date



Your LLM answers are wrong. Not hallucination-wrong — outdated-wrong.

You shipped a customer support bot on GPT-4. It's trained through early 2024. Your product changed 14 times since then. Every week, users get answers that were accurate 8 months ago and are flat-out wrong today.

The team is debating the fix.

Here's the setup:

NestJS API → OpenAI GPT-4 + PostgreSQL (product knowledge base)

~2,000 support queries/day, 15% return wrong answers tied to stale knowledge

Knowledge base updates weekly — new pricing, new features, deprecated flows

Budget: mid-size startup, not training custom models from scratch

You need accurate, up-to-date answers without re-training on every product update

What do you do?

A) RAG — embed your knowledge base, retrieve relevant chunks at query time, inject into context. Model stays the same, knowledge is always fresh.

- B)** Fine-tune the base model — train GPT-4 (or open-source equivalent) on your product docs. The model internalizes your domain.
- C)** Fine-tune + RAG hybrid — fine-tune for style/tone/domain fluency, RAG for factual grounding. Best of both worlds.
- D)** Prompt engineering only — detailed system prompt + few-shot examples. No infra, no training, just better instructions.

Answer 1

✓ CORRECT ANSWER

Answer: A — RAG

System Design Labs

Separate knowledge from reasoning. Keep the model frozen.**Solves the real problem**

The issue is freshness, not understanding. RAG gives the model up-to-date facts.

**No retraining cost**

Model stays frozen. No fine-tuning cycles, no redeployment.

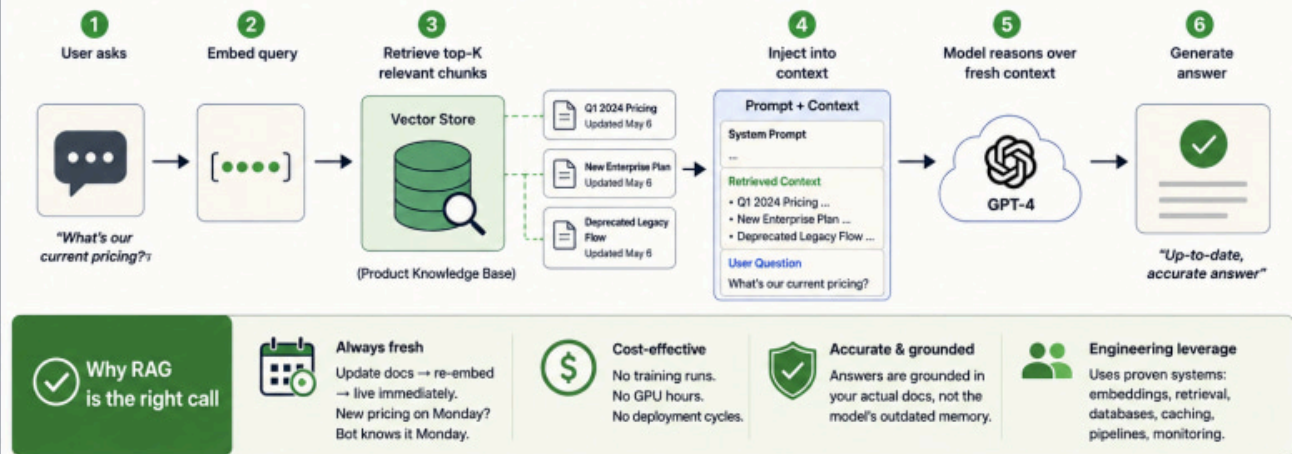
**Knowledge is decoupled**

Your knowledge lives in a vector store. Update, re-embed, done. Bot knows it immediately.

**Pipeline, not model problem**

You've turned this into a data pipeline problem — something every team already knows.

How RAG Works at Runtime

**Bottom line:** RAG gives you up-to-date, accurate answers without retraining.**Fix freshness at the data layer.** Let the model do what it does best: reason.**Answer: A — RAG**

The problem is freshness — the model doesn't know what changed, not that it doesn't understand your domain.

RAG separates knowledge from reasoning. The model stays frozen (no retraining cost, no deployment cycle). Your knowledge base lives in a vector store — update a doc, re-embed it, done. New pricing ships Monday, your bot knows it by Monday.

At runtime: user asks → embed query → retrieve top-K relevant chunks → inject into context → model reasons over fresh, accurate content.

You've turned a model problem into a data pipeline problem. Every engineering team already knows how to solve data pipelines.

Answer 2

B — Fine-tuning (the trap answer)

Feels like the obvious upgrade. Makes the problem worse.

Fine-tuning bakes knowledge into weights. Those weights are static until you retrain. Weekly product updates = weekly retraining at real cost (time + money) + real risk of catastrophic forgetting (new training can degrade recall of old content if not carefully managed).

Fine-tuning is the right answer for *behavioral* changes — tone, format, domain fluency, internal jargon. It teaches the model *how to behave*, not *what to know*. Wrong tool for a freshness problem.

Answer 3

C — Fine-tune + RAG hybrid (overkill)

Genuinely powerful architecture — but it's step 3, not step 1.

You can't productively fine-tune for style until your factual grounding is solid first. The hybrid is what you reach for *after* you've shipped RAG cleanly and hit the ceiling of what retrieval alone can do.

For a startup with a 15% wrong-answer rate due to stale knowledge, shipping a hybrid is a 3-month project when a 2-week RAG implementation fixes the actual problem.

Answer 4

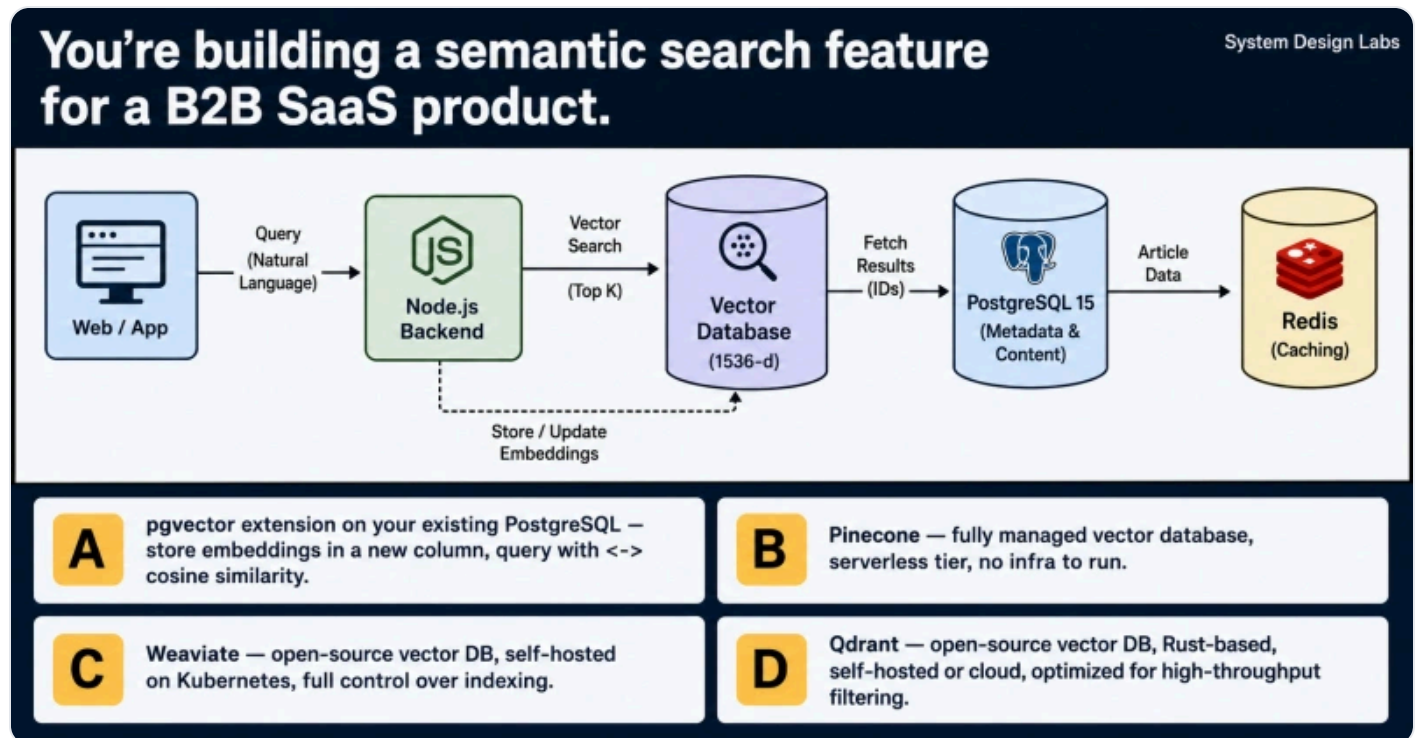
D — Prompt engineering only (hits the ceiling fast)

Fastest to ship, first to break.

A detailed system prompt can set tone and format. It cannot inject 400 pages of updated product docs without hitting token limits — and manually picking which docs to include is just RAG without the retrieval infrastructure.

Prompt engineering works when the model already knows what it needs to know. It can't fix a knowledge gap.

Choosing a Vector Store for Semantic Search



You're building a semantic search feature for a B2B SaaS product.

The corpus: 4 million support articles, docs, and user-generated tickets. Users type natural language queries. They expect Google-quality results — not keyword matching.

Your current stack: PostgreSQL 15, Redis, and a Node.js backend. The search team says ILIKE and pg_trgm aren't cutting it. Embeddings are the answer. Now you need a place to store and query 1536-dimensional vectors (OpenAI ada-002) at <100ms p99.

4 million rows. ~24GB of raw embeddings. Query volume: 300 req/s with weekend spikes to 900 req/s.

Where do you store and query those vectors?

- A** pgvector extension on your existing PostgreSQL — store embeddings in a new column, query with <-> cosine similarity.
- B** Pinecone — fully managed vector database, serverless tier, no infra to run.
- C** Weaviate — open-source vector DB, self-hosted on Kubernetes, full control over indexing.



Qdrant — open-source vector DB, Rust-based, self-hosted or cloud, optimized for high-throughput filtering.

Answer 1

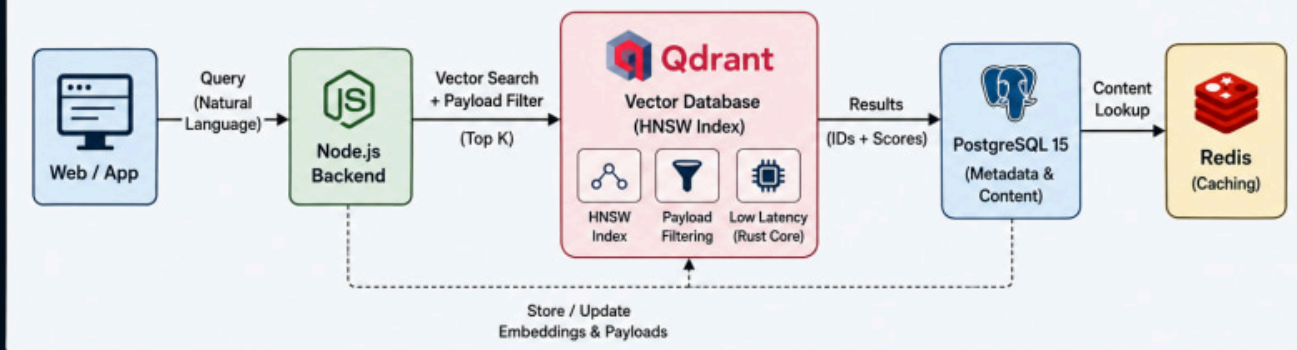
✓ CORRECT ANSWER

Why D wins: Qdrant is the right fit

System Design Labs



Qdrant handles vector search + metadata filter in one pass



Built for Scale & Low Latency

Rust core = low latency and predictable memory under load. At 300 req/s you need an index that handles concurrent ANN queries without degrading — Qdrant's HNSW implementation is tuned for this.



Killer Feature: Payload Filtering

In a B2B product, users search within a workspace, tenant, or product line — not globally. Qdrant handles vector search + metadata filter in one pass. Every other option forces *post-filtering*, which blows up recall and adds round-trips.



Full Control, Future Proof

Self-hosted gives you full control over HNSW params (m, ef_construction), memory mapping, and on-disk indexing — critical when your corpus grows past RAM.

Why D wins (Qdrant):

Qdrant is purpose-built for exactly this workload. Rust core = low latency and predictable memory under load. At 300 req/s you need an index that handles concurrent ANN queries without degrading — Qdrant's HNSW implementation is tuned for this.

The killer feature here is **payload filtering**. In a B2B product, users search within a workspace, tenant, or product line — not globally. Qdrant handles vector search + metadata filter in one pass. Every other option forces *post-filtering*, which blows up recall and adds round-trips.

Self-hosted gives you full control over HNSW params (m, ef_construction), memory mapping, and on-disk indexing — critical when your corpus grows past RAM.

Answer 2

Why A is the trap (pgvector):

pgvector is great for getting started — under ~500k vectors it's often the right call because you already own PostgreSQL. But at 4M rows and 300 req/s, the HNSW index runs inside Postgres's buffer pool, competing with your transactional workload. Latency goes non-linear under concurrency. VACUUM on a large embeddings table is painful.

Rule of thumb: pgvector under 500k vectors + low concurrency = fine. Above that, you're borrowing against future pain.

Answer 3

Why B loses (Pinecone):

Pinecone works and teams ship with it fast. But the cost model bites hard at 300 req/s sustained — serverless tier prices per query unit, you're looking at thousands of dollars/month. There's also proprietary lock-in: no standard wire protocol, so migrating means re-implementing your ingestion pipeline from scratch.

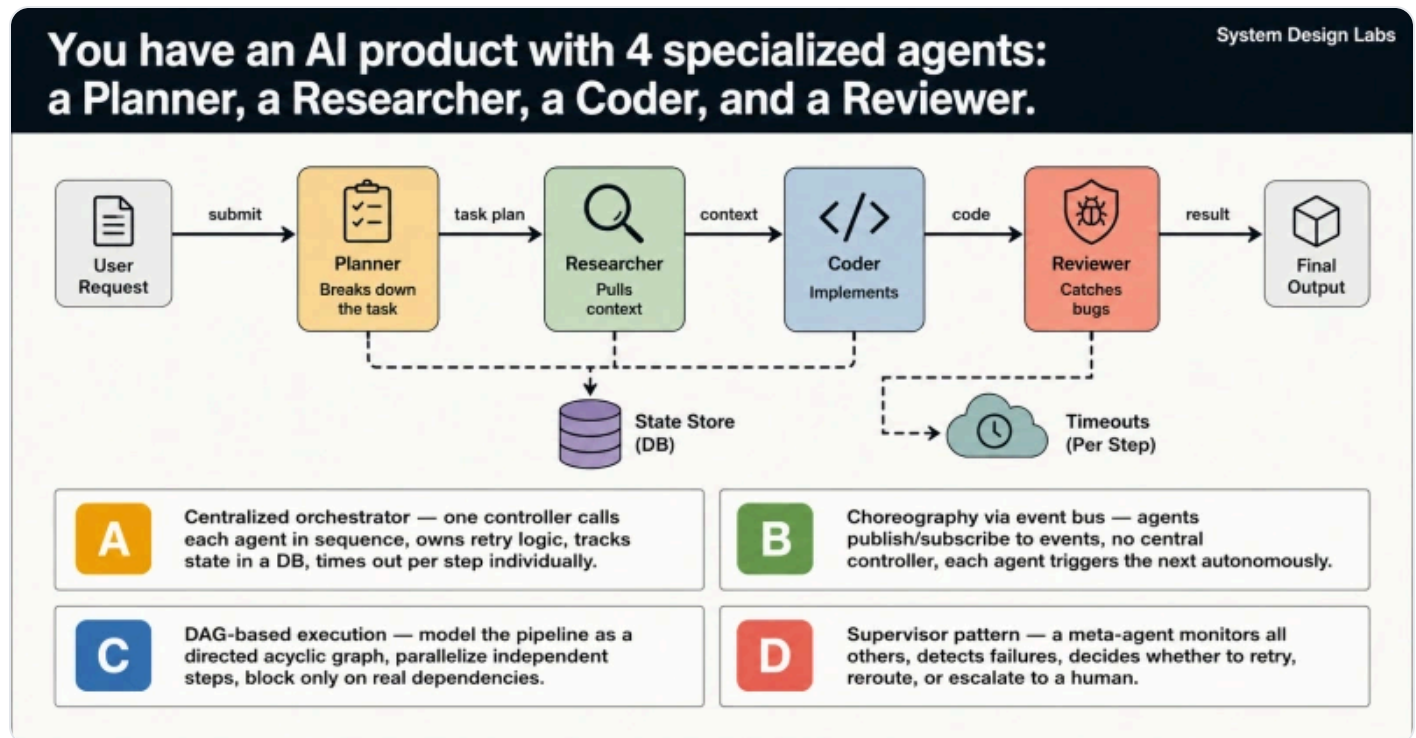
Right tool when: speed to market matters more than cost, and query volume is under ~50 req/s.

Answer 4

Why C loses (Weaviate):

Weaviate is solid with good hybrid search (BM25 + vector) and multi-modal support. But the Kubernetes footprint is heavier than Qdrant for this use case. For a team not already running K8s for their vector layer, the operational overhead is real. Weaviate shines for multi-modal search or semantic graph relationships — for pure dense vector search with filtering, Qdrant is leaner.

Coordinating a Multi-Agent Workflow



You have an AI product with 4 specialized agents: a Planner, a Researcher, a Coder, and a Reviewer.

The Planner breaks down the task. The Researcher pulls context. The Coder implements. The Reviewer catches bugs.

Simple on paper. In production, it's falling apart.

Here's what's happening:

- The Researcher sometimes returns before the Planner finishes → Coder gets incomplete context
- The Reviewer flags issues → but there's no retry loop, so bugs ship anyway
- One agent timeout hangs the entire pipeline for 40 seconds
- You have no visibility into which agent failed or why

You need to redesign the orchestration layer. What do you do?

A) Centralized orchestrator — one controller calls each agent in sequence, owns retry logic, tracks state in a DB, times out per step individually.

B) Choreography via event bus — agents publish/subscribe to events, no central controller, each agent triggers the next autonomously.

C) DAG-based execution — model the pipeline as a directed acyclic graph, parallelize independent steps, block only on real dependencies.

D) Supervisor pattern — a meta-agent monitors all others, detects failures, decides whether to retry, reroute, or escalate to a human.

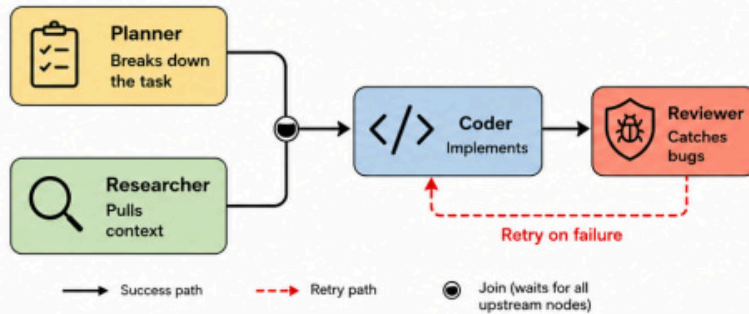
Answer 1

✓ CORRECT ANSWER

Correct Answer: C — DAG-based execution

System Design Labs

Your pipeline has hard dependencies (Coder must wait for BOTH Planner AND Researcher) AND needs retry loops (Reviewer → Coder on failure).

**What this solves directly:****Race condition**

DAG blocks Coder until both upstream nodes complete. No manual checks.

**Timeout hanging pipeline**

Each node has its own deadline. One agent timeout doesn't freeze everything.

**No retry loop**

Retry is a first-class edge in the graph, not bolted-on logic.

**No visibility**

DAG engines give you a full execution trace per run.

A

Centralized orchestrator — one controller calls each agent in sequence, owns retry logic, tracks state in a DB, times out per step individually.

B

Choreography via event bus — agents publish/subscribe to events, no central controller, each agent triggers the next autonomously.

C

DAG-based execution — model the pipeline as a directed acyclic graph, parallelize independent steps, block only on real dependencies.

D

Supervisor pattern — a meta-agent monitors all others, detects failures, decides whether to retry, reroute, or escalate to a human.



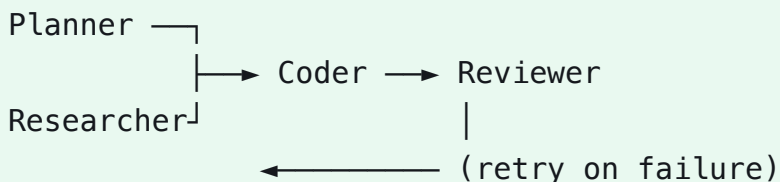
Real implementations: LangGraph, Temporal, AWS Step Functions, Prefect, Dagster.

They all converge on this model because it makes dependencies visible, failures local, and execution testable.

Correct Answer: C — DAG-based execution

Your pipeline has hard dependencies (Coder must wait for BOTH Planner AND Researcher) AND needs retry loops (Reviewer → Coder on failure).

A DAG models this explicitly:



What this solves directly:

- **Race condition** → DAG blocks Coder until both upstream nodes complete. No manual checks.
- **Timeout hanging pipeline** → each node has its own deadline. One agent timeout doesn't freeze everything.
- **No retry loop** → retry is a first-class edge in the graph, not bolted-on logic.
- **No visibility** → DAG engines give you a full execution trace per run.

Real implementations: LangGraph, Temporal, AWS Step Functions, Prefect, Dagster. They all converge on this model because it makes dependencies visible, failures local, and execution testable.

Answer 2

A — Centralized orchestrator (senior engineer trap)

This is the closest wrong answer. A central controller with per-step timeouts and DB state *feels* right — it's structured, it has retry logic, it has visibility.

The problem: it's **sequential by default**. Planner and Researcher still run one after the other unless you explicitly add parallel execution logic. You're basically hand-building a DAG imperatively, and you'll do it badly. Every new dependency becomes a code change.

Why it traps senior engineers: it looks like "proper architecture" but it's just imperative orchestration with extra ceremony.

Answer 3

B — Choreography via event bus

Works well for loosely coupled, fire-and-forget pipelines (order placed → invoice generated → email sent). But your pipeline has **tight dependencies** — Coder literally cannot start without both Planner AND Researcher finishing.

With pure choreography, there's no natural "wait for these two" primitive. You'd have to build coordination logic *inside the agents themselves*. Now your Coder is doing orchestration work. That's worse than your current problem.

Rule of thumb: choreography for independence, orchestration for dependency.

Answer 4

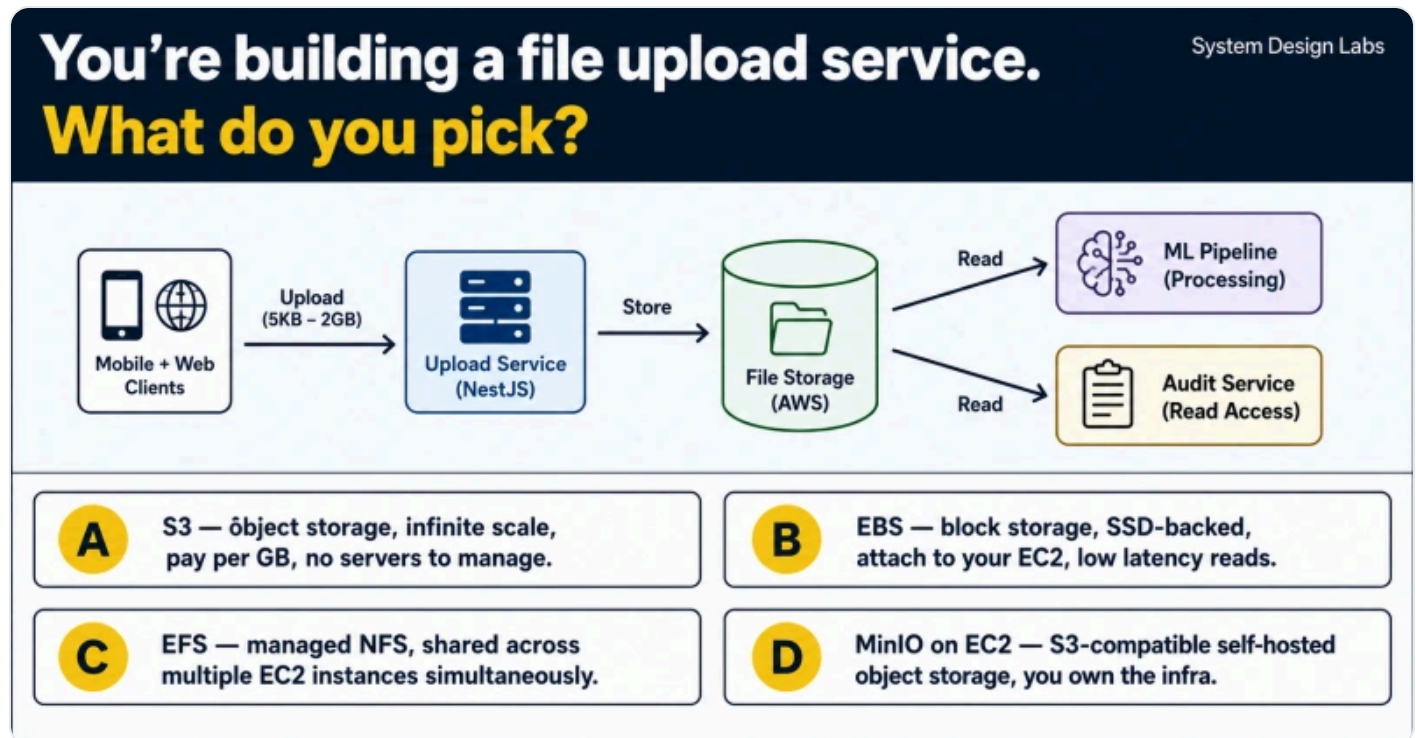
D — Supervisor pattern

A supervisor meta-agent that monitors, detects failures, and reroutes is a great **layer on top** of C — for human escalation, anomaly detection, or dynamic rerouting when the DAG can't handle it.

As a *base* orchestration model? It adds latency (every decision goes through the supervisor), a single point of failure, and doesn't solve the dependency problem at all. The supervisor still needs to know the execution order — which is just an implicit DAG inside the supervisor's brain.

Use it as resilience on top of C. Not instead of C.

Choosing a File Storage Backend



You're building a file upload service. 10TB of user files today. 100TB in 12 months.

Your team is having a fight.

The backend lead says: "Just use S3. Done."

The DevOps engineer says: "Mount an EBS volume. Simpler, faster."

The platform architect says: "We need EFS — multiple services need to read the same files."

The startup CTO says: "We can't afford cloud storage at scale. Self-host with MinIO."

All four have shipped this in production. All four have opinions backed by scars.

Here's the setup:

- Upload service (NestJS) receives files from mobile + web clients
- ML pipeline needs to read uploaded images for processing
- Audit service needs read access to the same files
- Files range from 5KB profile pics to 2GB video exports
- You're on AWS

What do you pick?

- A) S3 — object storage, infinite scale, pay per GB, no servers to manage.
- B) EBS — block storage, SSD-backed, attach to your EC2, low latency reads.
- C) EFS — managed NFS, shared across multiple EC2 instances simultaneously.
- D) MinIO on EC2 — S3-compatible self-hosted object storage, you own the infra.